

MM Adapters

Generated by Doxygen 1.9.1

1 Hierarchical Index	1
1.1 Class Hierarchy	1
2 Class Index	3
2.1 Class List	3
3 File Index	7
3.1 File List	7
4 Class Documentation	11
4.1 AddStock Class Reference	11
4.1.1 Detailed Description	12
4.1.2 Constructor & Destructor Documentation	12
4.1.2.1 AddStock()	12
4.1.3 Member Function Documentation	12
4.1.3.1 execute()	12
4.2 AskQuery Class Reference	13
4.2.1 Detailed Description	13
4.2.2 Member Function Documentation	13
4.2.2.1 careRoutine()	13
4.2.2.2 describe()	14
4.3 BasePlant Class Reference	15
4.3.1 Member Function Documentation	16
4.3.1.1 getDescription()	16
4.3.1.2 getPrice()	16
4.4 Builder Class Reference	17
4.4.1 Detailed Description	18
4.4.2 Member Function Documentation	18
4.4.2.1 build()	18
4.4.2.2 setItem()	18
4.4.2.3 setQuestion()	19
4.4.2.4 setType()	19
4.4.3 Member Data Documentation	19
4.4.3.1 item	19
4.4.3.2 question	19
4.4.3.3 type	20
4.5 CreditCardPaymentStrategy Class Reference	20
4.5.1 Detailed Description	21
4.5.2 Constructor & Destructor Documentation	21
4.5.2.1 CreditCardPaymentStrategy()	21
4.5.3 Member Function Documentation	21
4.5.3.1 getName()	21
4.5.3.2 pay()	22

4.6 Customer Class Reference	22
4.6.1 Detailed Description	22
4.6.2 Constructor & Destructor Documentation	23
4.6.2.1 Customer()	23
4.6.3 Member Function Documentation	23
4.6.3.1 getName()	23
4.6.3.2 makeQuery()	23
4.6.3.3 placeOrder()	24
4.6.3.4 setName()	24
4.7 DecorativePot Class Reference	24
4.7.1 Detailed Description	25
4.7.2 Constructor & Destructor Documentation	26
4.7.2.1 DecorativePot()	26
4.7.3 Member Function Documentation	26
4.7.3.1 getDescription()	26
4.7.3.2 getPrice()	26
4.8 DistressedState Class Reference	27
4.8.1 Detailed Description	28
4.8.2 Member Function Documentation	28
4.8.2.1 applyCare()	28
4.8.2.2 clone()	28
4.8.2.3 evaluate()	28
4.8.2.4 getName()	29
4.9 EFTPaymentStrategy Class Reference	29
4.9.1 Detailed Description	30
4.9.2 Constructor & Destructor Documentation	30
4.9.2.1 EFTPaymentStrategy()	30
4.9.3 Member Function Documentation	31
4.9.3.1 getName()	31
4.9.3.2 pay()	31
4.10 EmployeeFactory Class Reference	31
4.10.1 Detailed Description	32
4.10.2 Member Function Documentation	32
4.10.2.1 createFloorStaff()	33
4.10.2.2 createSalesStaff()	33
4.11 EWalletPaymentStrategy Class Reference	33
4.11.1 Detailed Description	34
4.11.2 Constructor & Destructor Documentation	34
4.11.2.1 EWalletPaymentStrategy()	35
4.11.3 Member Function Documentation	35
4.11.3.1 getName()	35
4.11.3.2 pay()	35

4.12 FertilizerStrategy Class Reference	36
4.12.1 Detailed Description	37
4.12.2 Member Function Documentation	37
4.12.2.1 applyFertilizer()	37
4.12.2.2 getStrategyName()	37
4.12.3 Member Data Documentation	37
4.12.3.1 currentPlant	38
4.13 FloorEmployee Class Reference	38
4.13.1 Detailed Description	39
4.13.2 Constructor & Destructor Documentation	39
4.13.2.1 FloorEmployee()	39
4.13.3 Member Function Documentation	40
4.13.3.1 reply()	40
4.13.3.2 staffCanHandle()	40
4.14 FloorManager Class Reference	41
4.14.1 Detailed Description	42
4.14.2 Constructor & Destructor Documentation	42
4.14.2.1 FloorManager()	42
4.14.3 Member Function Documentation	42
4.14.3.1 reply()	42
4.14.3.2 staffCanHandle()	43
4.15 FloorStaff Class Reference	43
4.15.1 Detailed Description	45
4.15.2 Constructor & Destructor Documentation	45
4.15.2.1 FloorStaff()	45
4.15.3 Member Function Documentation	45
4.15.3.1 handleQuery()	45
4.15.3.2 reply()	46
4.15.3.3 staffCanHandle()	46
4.16 FlowerFactory Class Reference	46
4.16.1 Detailed Description	47
4.16.2 Member Function Documentation	47
4.16.2.1 createPlant()	47
4.17 FullSunStrategy Class Reference	48
4.17.1 Detailed Description	49
4.17.2 Member Function Documentation	49
4.17.2.1 applySunlight()	49
4.17.2.2 getStrategyName()	50
4.18 GiftWrap Class Reference	50
4.18.1 Detailed Description	51
4.18.2 Constructor & Destructor Documentation	52
4.18.2.1 GiftWrap()	52

4.18.3 Member Function Documentation	52
4.18.3.1 getDescription()	52
4.18.3.2 getPrice()	52
4.19 GreenhouseUnitTest Class Reference	53
4.19.1 Detailed Description	53
4.20 HighWaterStrategy Class Reference	53
4.20.1 Detailed Description	54
4.20.2 Member Function Documentation	54
4.20.2.1 applyWater()	54
4.20.2.2 getStrategyName()	54
4.21 InorganicFertilizer Class Reference	55
4.21.1 Detailed Description	56
4.21.2 Member Function Documentation	56
4.21.2.1 applyFertilizer()	56
4.21.2.2 getStrategyName()	56
4.22 InventoryObserver Class Reference	57
4.22.1 Detailed Description	58
4.22.2 Constructor & Destructor Documentation	58
4.22.2.1 InventoryObserver()	58
4.23 InventoryProxy Class Reference	58
4.23.1 Detailed Description	59
4.23.2 Constructor & Destructor Documentation	59
4.23.2.1 InventoryProxy()	59
4.23.3 Member Function Documentation	59
4.23.3.1 addStock()	59
4.23.3.2 buyPlant()	59
4.23.3.3 removeStock()	60
4.24 ItemQueryBuilder Class Reference	60
4.24.1 Detailed Description	61
4.24.2 Member Function Documentation	61
4.24.2.1 setItem()	61
4.24.2.2 setQuestion()	62
4.24.2.3 setType()	62
4.25 LifeCycleObserver Class Reference	62
4.25.1 Detailed Description	63
4.25.2 Member Function Documentation	63
4.25.2.1 update()	63
4.26 LowWaterStrategy Class Reference	64
4.26.1 Detailed Description	65
4.26.2 Member Function Documentation	65
4.26.2.1 applyWater()	65
4.26.2.2 getStrategyName()	65

4.27 Managerfactory Class Reference	66
4.27.1 Member Function Documentation	66
4.27.1.1 createFloorStaff()	67
4.27.1.2 createSalesStaff()	68
4.28 Managerfactory Class Reference	68
4.28.1 Member Function Documentation	69
4.28.1.1 createFloorStaff()	69
4.28.1.2 createSalesStaff()	70
4.29 MatureState Class Reference	70
4.29.1 Detailed Description	71
4.29.2 Member Function Documentation	71
4.29.2.1 applyCare()	71
4.29.2.2 evaluate()	72
4.29.2.3 getName()	72
4.30 MediumWaterStrategy Class Reference	73
4.30.1 Detailed Description	73
4.30.2 Member Function Documentation	74
4.30.2.1 applyWater()	74
4.30.2.2 getStrategyName()	74
4.31 MiscQueryBuilder Class Reference	75
4.31.1 Detailed Description	76
4.31.2 Member Function Documentation	76
4.31.2.1 setItem()	76
4.31.2.2 setQuestion()	76
4.31.2.3 setType()	77
4.32 Oak Class Reference	77
4.32.1 Detailed Description	78
4.32.2 Constructor & Destructor Documentation	78
4.32.2.1 Oak()	78
4.33 Order Class Reference	79
4.33.1 Detailed Description	79
4.33.2 Constructor & Destructor Documentation	79
4.33.2.1 Order()	79
4.33.3 Member Function Documentation	80
4.33.3.1 addTransaction()	80
4.33.3.2 getCustomer()	80
4.33.3.3 getOrderId()	80
4.33.3.4 getStatus()	80
4.34 OrganicFertilizer Class Reference	81
4.34.1 Detailed Description	82
4.34.2 Member Function Documentation	82
4.34.2.1 applyFertilizer()	82

4.34.2.2 getStrategyName()	82
4.35 PartialSunStrategy Class Reference	83
4.35.1 Detailed Description	84
4.35.2 Member Function Documentation	84
4.35.2.1 applySunlight()	84
4.35.2.2 getStrategyName()	84
4.36 PaymentStrategy Class Reference	85
4.36.1 Detailed Description	85
4.36.2 Member Function Documentation	85
4.36.2.1 getName()	85
4.36.2.2 pay()	85
4.37 Plant Class Reference	86
4.37.1 Detailed Description	88
4.37.2 Constructor & Destructor Documentation	88
4.37.2.1 Plant()	88
4.37.3 Member Function Documentation	89
4.37.3.1 attach()	89
4.37.3.2 detach()	89
4.37.3.3 getCategory()	89
4.37.3.4 getColor()	89
4.37.3.5 getCurrentNutrients()	90
4.37.3.6 getCurrentSunlight()	90
4.37.3.7 getCurrentWater()	90
4.37.3.8 getDescription()	90
4.37.3.9 getGrowthProgress()	91
4.37.3.10 getMaxNutrients()	91
4.37.3.11 getMaxSunlight()	91
4.37.3.12 getMaxWater()	91
4.37.3.13 getMinNutrients()	92
4.37.3.14 getMinSunlight()	92
4.37.3.15 getMinWater()	92
4.37.3.16 getName()	92
4.37.3.17 getPrice()	93
4.37.3.18 getState()	93
4.37.3.19 setCategory()	93
4.37.3.20 setColor()	93
4.37.3.21 setCurrentNutrients()	94
4.37.3.22 setCurrentSunlight()	94
4.37.3.23 setCurrentWater()	94
4.37.3.24 setGrowthProgress()	94
4.37.3.25 setMaxNutrients()	95
4.37.3.26 setMaxSunlight()	95

4.37.3.27 setMaxWater()	95
4.37.3.28 setState()	96
4.38 PlantCareRoutine Class Reference	96
4.38.1 Detailed Description	97
4.38.2 Member Function Documentation	97
4.38.2.1 PlantCare()	97
4.39 PlantDecorator Class Reference	97
4.39.1 Detailed Description	98
4.40 PlantFactory Class Reference	99
4.40.1 Detailed Description	99
4.40.2 Member Function Documentation	99
4.40.2.1 createPlant()	99
4.41 PlantInventory Class Reference	100
4.41.1 Detailed Description	101
4.41.2 Member Function Documentation	101
4.41.2.1 addStock()	101
4.41.2.2 attach()	101
4.41.2.3 detach()	101
4.41.2.4 executeCommand()	102
4.41.2.5 findPlantIndex()	102
4.41.2.6 getInstance()	103
4.41.2.7 getInventoryReference()	103
4.41.2.8 getInventoryView()	103
4.41.2.9 getQuantity()	103
4.41.2.10 removeStock()	104
4.42 PlantLifeCycle Class Reference	104
4.42.1 Detailed Description	105
4.43 PlantState Class Reference	105
4.43.1 Detailed Description	105
4.43.2 Member Function Documentation	105
4.43.2.1 applyCare()	105
4.43.2.2 evaluate()	106
4.43.2.3 getName()	106
4.44 PlantStock Class Reference	107
4.44.1 Detailed Description	107
4.44.2 Constructor & Destructor Documentation	107
4.44.2.1 PlantStock()	107
4.44.3 Member Function Documentation	107
4.44.3.1 addStock()	107
4.44.3.2 removeStock()	109
4.45 Query Class Reference	109
4.45.1 Detailed Description	110

4.45.2 Constructor & Destructor Documentation	110
4.45.2.1 Query()	110
4.45.3 Member Function Documentation	110
4.45.3.1 getItem()	111
4.45.3.2 getQuestion()	111
4.45.3.3 getType()	111
4.46 RemoveStock Class Reference	112
4.46.1 Detailed Description	112
4.46.2 Constructor & Destructor Documentation	113
4.46.2.1 RemoveStock()	113
4.46.3 Member Function Documentation	113
4.46.3.1 execute()	113
4.47 Rose Class Reference	114
4.47.1 Detailed Description	114
4.47.2 Constructor & Destructor Documentation	115
4.47.2.1 Rose()	115
4.48 SalesEmployee Class Reference	115
4.48.1 Detailed Description	116
4.48.2 Constructor & Destructor Documentation	116
4.48.2.1 SalesEmployee()	116
4.48.3 Member Function Documentation	117
4.48.3.1 reply()	117
4.48.3.2 staffCanHandle()	117
4.49 SalesFloorObserver Class Reference	118
4.49.1 Detailed Description	119
4.49.2 Constructor & Destructor Documentation	119
4.49.2.1 SalesFloorObserver()	119
4.49.3 Member Function Documentation	119
4.49.3.1 attachStaff()	119
4.49.3.2 detachStaff()	120
4.49.3.3 getAvailableItems()	120
4.49.3.4 getNotificationHistory()	120
4.49.3.5 getPlantQuantity()	120
4.49.3.6 isPlantAvailable()	121
4.49.3.7 notifyStaff()	121
4.50 SalesManager Class Reference	121
4.50.1 Detailed Description	123
4.50.2 Constructor & Destructor Documentation	123
4.50.2.1 SalesManager()	123
4.50.3 Member Function Documentation	123
4.50.3.1 reply()	123
4.50.3.2 staffCanHandle()	124

4.51 SalesStaff Class Reference	124
4.51.1 Detailed Description	126
4.51.2 Constructor & Destructor Documentation	126
4.51.2.1 SalesStaff()	126
4.51.3 Member Function Documentation	126
4.51.3.1 handleQuery()	126
4.51.3.2 reply()	127
4.51.3.3 staffCanHandle()	127
4.52 SeedlingState Class Reference	127
4.52.1 Detailed Description	128
4.52.2 Member Function Documentation	128
4.52.2.1 applyCare()	129
4.52.2.2 evaluate()	129
4.52.2.3 getName()	129
4.53 SeedState Class Reference	130
4.53.1 Detailed Description	130
4.53.2 Member Function Documentation	131
4.53.2.1 applyCare()	131
4.53.2.2 evaluate()	131
4.53.2.3 getName()	131
4.54 Shade Class Reference	132
4.55 ShadeStrategy Class Reference	133
4.55.1 Detailed Description	134
4.55.2 Member Function Documentation	134
4.55.2.1 applySunlight()	134
4.55.2.2 getStrategyName()	134
4.56 SpecialArrangement Class Reference	135
4.56.1 Detailed Description	136
4.57 Staff Class Reference	136
4.57.1 Detailed Description	137
4.57.2 Constructor & Destructor Documentation	138
4.57.2.1 Staff()	138
4.57.2.2 ~Staff()	138
4.57.3 Member Function Documentation	138
4.57.3.1 getName()	138
4.57.3.2 getNext()	138
4.57.3.3 getStock()	139
4.57.3.4 handleQuery()	139
4.57.3.5 isHealthy()	139
4.57.3.6 setNext()	140
4.58 StaffFactory Class Reference	140
4.58.1 Detailed Description	141

4.58.2 Member Function Documentation	141
4.58.2.1 createFloorStaff()	141
4.58.2.2 createSalesStaff()	142
4.59 StaffObserver Class Reference	142
4.59.1 Detailed Description	143
4.59.2 Constructor & Destructor Documentation	144
4.59.2.1 StaffObserver()	144
4.60 StockCommand Class Reference	144
4.60.1 Detailed Description	145
4.60.2 Constructor & Destructor Documentation	145
4.60.2.1 ~StockCommand()	145
4.60.3 Member Function Documentation	145
4.60.3.1 execute()	145
4.61 SunlightStrategy Class Reference	146
4.61.1 Member Function Documentation	147
4.61.1.1 applySunlight()	147
4.61.1.2 getStrategyName()	147
4.61.2 Member Data Documentation	147
4.61.2.1 currentPlant	147
4.62 Sunny Class Reference	148
4.62.1 Detailed Description	149
4.62.2 Member Function Documentation	149
4.62.2.1 Fertilizing()	149
4.62.2.2 Sunlight()	149
4.62.2.3 Watering()	149
4.63 Temperate Class Reference	150
4.63.1 Detailed Description	151
4.63.2 Member Function Documentation	151
4.63.2.1 Fertilizing()	151
4.63.2.2 Sunlight()	151
4.63.2.3 Watering()	152
4.64 Transaction Class Reference	152
4.64.1 Detailed Description	153
4.64.2 Constructor & Destructor Documentation	153
4.64.2.1 Transaction()	153
4.64.3 Member Function Documentation	153
4.64.3.1 addDecoration()	153
4.64.3.2 clone()	154
4.64.3.3 createSnapshot()	154
4.64.3.4 getAmount()	154
4.64.3.5 getDecorations()	155
4.64.3.6 getPaymentMethod()	155

4.64.3.7	getQuantity()	155
4.64.3.8	getTransactionId()	155
4.64.3.9	restoreSnapshot()	155
4.64.3.10	setPaymentStrategy()	156
4.64.3.11	setTransaction()	156
4.65	TransactionHistory Class Reference	156
4.65.1	Detailed Description	157
4.65.2	Member Function Documentation	157
4.65.2.1	addSnapshot()	157
4.65.2.2	getHistorySize()	157
4.65.2.3	getSnapshot()	157
4.66	TransactionSnapshot Class Reference	158
4.66.1	Detailed Description	158
4.66.2	Constructor & Destructor Documentation	158
4.66.2.1	TransactionSnapshot()	158
4.66.3	Member Function Documentation	159
4.66.3.1	getAmount()	159
4.66.3.2	getDecorations()	159
4.66.3.3	getOrderNum()	159
4.66.3.4	getQuantity()	160
4.67	TreeFactory Class Reference	160
4.67.1	Detailed Description	161
4.67.2	Member Function Documentation	161
4.67.2.1	createPlant()	161
4.68	Tropical Class Reference	161
4.68.1	Detailed Description	162
4.68.2	Member Function Documentation	163
4.68.2.1	Fertilizing()	163
4.68.2.2	Sunlight()	163
4.68.2.3	Watering()	163
4.69	WaterStrategy Class Reference	164
4.69.1	Detailed Description	165
4.69.2	Member Function Documentation	165
4.69.2.1	applyWater()	165
4.69.2.2	getStrategyName()	165
4.70	WitheredState Class Reference	166
4.70.1	Detailed Description	167
4.70.2	Member Function Documentation	167
4.70.2.1	applyCare()	167
4.70.2.2	clone()	167
4.70.2.3	evaluate()	167
4.70.2.4	getName()	168

5 File Documentation	169
5.1 AskQuery.cpp File Reference	169
5.1.1 Detailed Description	169
5.2 AskQuery.h File Reference	170
5.2.1 Detailed Description	171
5.3 Builder.cpp File Reference	171
5.3.1 Detailed Description	171
5.4 Builder.h File Reference	172
5.4.1 Detailed Description	173
5.5 CreditCardPaymentStrategy.h File Reference	173
5.5.1 Detailed Description	173
5.6 Customer.h File Reference	174
5.6.1 Detailed Description	174
5.7 DecorativePot.h File Reference	174
5.7.1 Detailed Description	175
5.8 DistressedState.h File Reference	175
5.8.1 Detailed Description	176
5.9 EFTPaymentStrategy.h File Reference	177
5.9.1 Detailed Description	177
5.10 EmployeeFactory.h File Reference	177
5.10.1 Detailed Description	178
5.11 EWalletPaymentStrategy.h File Reference	178
5.11.1 Detailed Description	179
5.12 FertilizerStrategy.h File Reference	179
5.12.1 Detailed Description	180
5.13 FloorEmployee.h File Reference	181
5.13.1 Detailed Description	181
5.14 FloorManager.h File Reference	181
5.14.1 Detailed Description	182
5.15 FloorStaff.h File Reference	182
5.15.1 Detailed Description	183
5.16 FlowerFactory.h File Reference	184
5.16.1 Detailed Description	185
5.17 FullSunStrategy.h File Reference	185
5.17.1 Detailed Description	186
5.18 GiftWrap.h File Reference	186
5.18.1 Detailed Description	187
5.19 HighWaterStrategy.h File Reference	187
5.19.1 Detailed Description	188
5.20 InorganicFertilizer.h File Reference	188
5.20.1 Detailed Description	189
5.21 InventoryObserver.h File Reference	189

5.21.1 Detailed Description	190
5.22 InventoryProxy.h File Reference	190
5.22.1 Detailed Description	191
5.23 ItemQueryBuilder.cpp File Reference	192
5.23.1 Detailed Description	192
5.24 ItemQueryBuilder.h File Reference	192
5.24.1 Detailed Description	194
5.25 LifeCycleObserver.h File Reference	194
5.25.1 Detailed Description	195
5.26 LowWaterStrategy.h File Reference	195
5.26.1 Detailed Description	196
5.27 ManagerFactory.h File Reference	196
5.27.1 Detailed Description	196
5.28 MatureState.h File Reference	197
5.28.1 Detailed Description	197
5.29 MediumWaterStrategy.h File Reference	198
5.29.1 Detailed Description	198
5.30 MiscQueryBuilder.cpp File Reference	199
5.30.1 Detailed Description	199
5.31 MiscQueryBuilder.h File Reference	199
5.31.1 Detailed Description	201
5.32 Oak.h File Reference	201
5.32.1 Detailed Description	202
5.33 Order.h File Reference	202
5.33.1 Detailed Description	203
5.34 OrganicFertilizer.h File Reference	203
5.34.1 Detailed Description	204
5.35 PartialSunStrategy.h File Reference	204
5.35.1 Detailed Description	205
5.36 PaymentStrategy.h File Reference	206
5.36.1 Detailed Description	206
5.37 Plant.h File Reference	207
5.37.1 Detailed Description	207
5.38 PlantCareRoutine.h File Reference	208
5.38.1 Detailed Description	208
5.39 PlantDecorator.h File Reference	209
5.39.1 Detailed Description	210
5.40 PlantFactory.h File Reference	210
5.40.1 Detailed Description	211
5.41 PlantInventory.h File Reference	211
5.41.1 Detailed Description	212
5.42 PlantState.h File Reference	213

5.42.1 Detailed Description	213
5.43 PlantStock.h File Reference	214
5.43.1 Detailed Description	215
5.44 Query.h File Reference	215
5.44.1 Detailed Description	216
5.45 RemoveStock.h File Reference	216
5.45.1 Detailed Description	218
5.46 Rose.h File Reference	218
5.46.1 Detailed Description	219
5.47 SalesEmployee.h File Reference	219
5.47.1 Detailed Description	220
5.48 SalesFloorObserver.h File Reference	220
5.48.1 Detailed Description	221
5.49 SalesManager.h File Reference	222
5.49.1 Detailed Description	222
5.50 SalesStaff.h File Reference	223
5.50.1 Detailed Description	224
5.51 SeedlingState.h File Reference	224
5.51.1 Detailed Description	224
5.52 SeedState.h File Reference	225
5.52.1 Detailed Description	225
5.53 Shade.h File Reference	226
5.53.1 Detailed Description	226
5.54 ShadeStrategy.h File Reference	227
5.54.1 Detailed Description	227
5.55 SpecialArrangement.h File Reference	228
5.55.1 Detailed Description	228
5.56 Staff.h File Reference	229
5.56.1 Detailed Description	230
5.57 StaffFactory.h File Reference	230
5.57.1 Detailed Description	231
5.58 StaffObserver.h File Reference	231
5.58.1 Detailed Description	232
5.59 StockCommand.h File Reference	232
5.59.1 Detailed Description	232
5.60 SunlightStrategy.h File Reference	233
5.60.1 Detailed Description	233
5.61 Sunny.h File Reference	234
5.61.1 Detailed Description	234
5.62 Temperate.h File Reference	235
5.62.1 Detailed Description	235
5.63 Transaction.h File Reference	236

5.63.1 Detailed Description	237
5.64 TransactionHistory.h File Reference	237
5.64.1 Detailed Description	237
5.65 TransactionSnapshot.h File Reference	238
5.65.1 Detailed Description	239
5.66 TreeFactory.h File Reference	239
5.66.1 Detailed Description	240
5.67 Tropical.h File Reference	240
5.67.1 Detailed Description	241
5.68 WitheredState.h File Reference	241
5.68.1 Detailed Description	242
Index	243

Chapter 1

Hierarchical Index

1.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

AskQuery	13
Builder	17
ItemQueryBuilder	60
MiscQueryBuilder	75
Customer	22
FertilizerStrategy	36
InorganicFertilizer	55
OrganicFertilizer	81
GreenhouseUnitTest	53
InventoryObserver	57
SalesFloorObserver	118
StaffObserver	142
InventoryProxy	58
LifeCycleObserver	62
Staff	136
FloorStaff	43
FloorEmployee	38
FloorManager	41
SalesStaff	124
SalesEmployee	115
SalesManager	121
ManagerFactory	??
Order	79
PaymentStrategy	85
CreditCardPaymentStrategy	20
EFTPaymentStrategy	29
EWalletPaymentStrategy	33
Plant	86
BasePlant	15
Oak	77
PlantDecorator	97
DecorativePot	24
GiftWrap	50

SpecialArrangement	135
Rose	114
PlantCareRoutine	96
Shade	132
Sunny	148
Temperate	150
Tropical	161
PlantFactory	99
FlowerFactory	46
TreeFactory	160
PlantInventory	100
PlantLifeCycle	104
PlantState	105
DistressedState	27
MatureState	70
SeedState	130
SeedlingState	127
WitheredState	166
PlantStock	107
Query	109
StaffFactory	140
EmployeeFactory	31
Managerfactory	68
StockCommand	144
AddStock	11
RemoveStock	112
SunlightStrategy	146
FullSunStrategy	48
PartialSunStrategy	83
ShadeStrategy	133
Transaction	152
TransactionHistory	156
TransactionSnapshot	158
WaterStrategy	164
HighWaterStrategy	53
LowWaterStrategy	64
MediumWaterStrategy	73

Chapter 2

Class Index

2.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

AddStock	Concrete command for adding plant stock to inventory	11
AskQuery	Director class that simplifies Query object creation using builders	13
BasePlant		15
Builder	Abstract builder for constructing Query objects	17
CreditCardPaymentStrategy	Payment strategy implementation using credit card information	20
Customer	Represents a store customer and their interactions	22
DecorativePot	Concrete decorator that adds a decorative pot to a Plant	24
DistressedState	Concrete state representing a plant in distress	27
EFTPaymentStrategy	Payment strategy implementation for EFT transactions	29
EmployeeFactory	Factory responsible for creating regular staff members	31
EWalletPaymentStrategy	Implements a payment method using an e-wallet ID	33
FertilizerStrategy	Abstract base for fertilizer behavior strategies	36
FloorEmployee	Regular employee handling on-floor customer queries	38
FloorManager	Manages floor staff and handles high-level queries	41
FloorStaff	Base class for all floor staff types	43
FlowerFactory	Concrete factory for creating flower-type Plants	46
FullSunStrategy	Strategy for applying full sunlight to a Plant	48
GiftWrap	Concrete decorator that adds gift wrapping to a Plant	50

GreenhouseUnitTest	Unit tests for greenhouse plant system (singleton, factories and commands design patterns)	53
HighWaterStrategy	Strategy for applying a high amount of water to a plant	53
InorganicFertilizer	Strategy for applying inorganic fertilizers to a plant	55
InventoryObserver	Abstract base class for observers of inventory changes	57
InventoryProxy	Acts as a proxy to manage plant inventory transactions	58
ItemQueryBuilder	Builder class for constructing queries about specific plants	60
LifeCycleObserver	Observer interface for plant life cycle changes	62
LowWaterStrategy	Strategy for plants requiring minimal water intake	64
ManagerFactory	Factory for creating floor and sales managers	??
Managerfactory		68
MatureState	Represents a thriving stage in the plant's life cycle	70
MediumWaterStrategy	Strategy for moderately watering plants	73
MiscQueryBuilder	Concrete builder for non-item queries	75
Oak	Represents an Oak tree plant	77
Order	Represents a customer's purchase order	79
OrganicFertilizer	Strategy for applying organic fertilizer to a plant	81
PartialSunStrategy	Strategy for applying partial sunlight to a plant	83
PaymentStrategy	Abstract strategy for processing payments	85
Plant	Abstract representation of a plant with care and lifecycle management	86
PlantCareRoutine	Abstract base class defining a plant care routine interface	96
PlantDecorator	Abstract decorator for dynamically extending Plant behavior	97
PlantFactory	Abstract base class for plant creation factories	99
PlantInventory	Singleton class managing plant inventory and observers	100
PlantLifeCycle	Manages the state and observers for a plant's life cycle	104
PlantState	Abstract base class representing a state in a plant's life cycle	105
PlantStock	Acts as the Invoker in the Command pattern	107
Query	Encapsulates a query object constructed by a Builder	109
RemoveStock	Concrete command that removes plant stock from the inventory	112
Rose	Concrete implementation of the Plant class for roses	114

SalesEmployee	Concrete class representing a regular sales employee	115
SalesFloorObserver	Observes the PlantInventory and updates the sales floor display	118
SalesManager	Represents a manager in the sales department	121
SalesStaff	Abstract base class for staff members in the sales department	124
SeedlingState	Represents the initial growth stage of a plant's life cycle. In this state, the plant is vulnerable and requires careful nurturing to develop into a healthy mature plant	127
SeedState	Represents the initial state of a plant's life cycle, where it is a seed. In this state, the seed is dormant and requires specific conditions to germinate and grow into a seedling	130
Shade		132
ShadeStrategy	SunlightStrategy implementation for shade	133
SpecialArrangement	Concrete decorator that adds a special arrangement to a plant	135
Staff	Base class representing a staff member and lifecycle observer	136
StaffFactory	Abstract base factory for creating different types of staff members	140
StaffObserver	Concrete observer that tracks inventory for staff	142
StockCommand	Abstract base class for command pattern in stock management	144
SunlightStrategy		146
Sunny	Implements the care routine for sun-loving plants	148
Temperate	Implements plant care routine for temperate climate plants	150
Transaction	Encapsulates all details of a plant purchase transaction	152
TransactionHistory	Manages the storage and retrieval of transaction snapshots	156
TransactionSnapshot	Represents a snapshot of a completed transaction	158
TreeFactory	Concrete factory for creating tree plants	160
Tropical	Represents the plant care routine for tropical plants	161
WaterStrategy	Abstract base class for water strategies. This class defines the interface for water strategies, which determine how much water to apply to a plant. Concrete strategy classes should inherit from this class and implement the <code>applyWater</code> and <code>getStrategyName</code> methods	164
WitheredState	Represents the final state of a plant's life cycle, where it has withered and died. In this state, the plant is beyond recovery and will not respond to care	166

Chapter 3

File Index

3.1 File List

Here is a list of all documented files with brief descriptions:

AddStock.h	??
AskQuery.cpp	
Implementation of AskQuery facade methods for Query creation	169
AskQuery.h	
Director for creating different types of Query objects using builders	170
BasePlant.h	??
Builder.cpp	
Implementation of common Builder functionality	171
Builder.h	
Defines the Builder interface for constructing Query objects	172
ConcretePlantCollection.h	??
ConcreteProductIterator.h	??
CreditCardPaymentStrategy.h	
Concrete payment strategy using credit card transactions	173
Customer.h	
Represents a Customer who can place orders and make staff queries	174
DecorativePot.h	
Decorator class adding a decorative pot feature to a Plant	174
DistressedState.h	
Represents the distressed state of a plant's life cycle	175
EFTPaymentStrategy.h	
Concrete payment strategy using Electronic Funds Transfer (EFT)	177
EmployeeFactory.h	
Concrete factory for creating regular employees	177
EWalletPaymentStrategy.h	
Concrete payment strategy using a digital e-wallet	178
FertilizerStrategy.h	
Defines the abstract interface for plant fertilizer strategies	179
FloorEmployee.h	
Concrete class representing a floor employee	181
FloorManager.h	
Class representing the manager of floor operations	181
FloorStaff.h	
Abstract base class for floor-level staff members	182
FlowerFactory.h	
Factory for creating flower-type Plant objects	184

FullSunStrategy.h	Concrete sunlight strategy applying full sunlight exposure to a plant	185
GiftWrap.h	Decorator class adding a gift wrapping feature to a Plant	186
GreenhouseUnitTest.h		??
HighWaterStrategy.h	Concrete watering strategy that applies high water levels to a plant	187
InorganicFertilizer.h	Concrete strategy for applying inorganic fertilizer to plants	188
InventoryObserver.h	Abstract base class for observers monitoring plant inventory changes	189
InventoryProxy.h	Proxy class controlling access to plant inventory operations	190
ItemQueryBuilder.cpp	Implementation of ItemQueryBuilder setters	192
ItemQueryBuilder.h	Concrete builder for creating item-specific Query objects	192
LifecycleObserver.h	Abstract base class for observers monitoring plant life cycle changes	194
LowWaterStrategy.h	Defines the LowWaterStrategy class used for minimal watering	195
ManagerFactory.h	Defines the factory for creating manager-level staff	196
MatureState.h	Defines the MatureState class representing a plant's healthy phase	197
MediumWaterStrategy.h	Defines the MediumWaterStrategy for moderate watering needs	198
MiscQueryBuilder.cpp	Implementation of MiscQueryBuilder methods	199
MiscQueryBuilder.h	Builder for creating general (non-item-specific) queries	199
Oak.h	Concrete Plant class representing an Oak tree	201
Order.h	Represents a customer's order containing multiple transactions	202
OrganicFertilizer.h	Concrete fertilizer strategy using organic nutrients	203
PartialSunStrategy.h	Concrete sunlight strategy providing moderate sunlight exposure	204
PaymentStrategy.h	Abstract interface for various payment strategies	206
Plant.h	Abstract base class representing a plant in the nursery system	207
PlantCareRoutine.h	Defines the abstract interface for plant care routines	208
PlantCollection.h		??
PlantDecorator.h	Abstract decorator for extending Plant behavior dynamically	209
PlantFactory.h	Abstract factory interface for creating Plant objects	210
PlantInventory.h	Manages plant inventory using Singleton and Observer patterns	211
PLantLifeCycle.h		??
PlantState.h	Defines the abstract interface for plant life cycle states	213
PlantStock.h	Invoker for executing stock-related commands	214
ProductIterator.h		??

Query.h	Represents a query within the plant management system	215
RemoveStock.h	Defines the RemoveStock command for decreasing inventory	216
Rose.h	Defines a concrete Rose plant class	218
SalesEmployee.h	Defines a regular employee in the sales department	219
SalesFloorObserver.h	Defines the SalesFloorObserver class for inventory updates	220
SalesManager.h	Defines the SalesManager class for the sales department	222
SalesStaff.h	Abstract base class for staff in the sales department	223
SeedlingState.h	Represents the seedling stage of a plant's life cycle	224
SeedState.h	Represents the seed stage of a plant's life cycle	225
Shade.h	Plant care routine implementing shade care	226
ShadeStrategy.h	Sunlight strategy for shade conditions	227
SpecialArrangement.h	Decorator for plants that adds a special arrangement feature	228
Staff.h	Abstract base class for staff members and lifecycle observers	229
StaffFactory.h	Abstract factory for creating staff members	230
StaffObserver.h	Observer class for tracking inventory relevant to staff	231
StockCommand.h	Abstract base class for stock management commands	232
SunlightStrategy.h	Abstract base class for sunlight strategies	233
Sunny.h	Plant care routine for sunny conditions	234
Temperate.h	Plant care routine for temperate conditions	235
Transaction.h	Represents a customer transaction and payment	236
TransactionHistory.h	Maintains a record of transaction snapshots for order tracking and history retrieval	237
TransactionSnapshot.h	Defines the TransactionSnapshot class for storing details of a single order transaction	238
TreeFactory.h	Declares the TreeFactory class responsible for creating tree-type plants	239
Tropical.h	Declares the Tropical care routine for tropical plant types	240
WateringStrategy.h		??
WitheredState.h	Declares the WitheredState class representing a plant's final life stage	241

Chapter 4

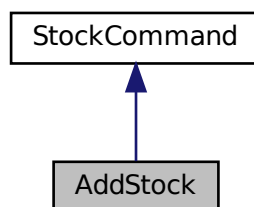
Class Documentation

4.1 AddStock Class Reference

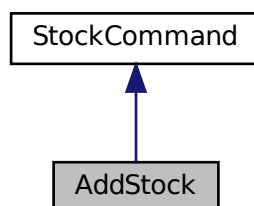
Concrete command for adding plant stock to inventory.

```
#include <AddStock.h>
```

Inheritance diagram for AddStock:



Collaboration diagram for AddStock:



Public Member Functions

- [AddStock](#) ([Plant](#) *plant, int quantityToAdd, [PlantInventory](#) *targetInventory)
Constructs an [AddStock](#) command.
- bool [execute](#) () override
Executes the add stock command.

4.1.1 Detailed Description

Concrete command for adding plant stock to inventory.

This class implements the Command pattern to add a specified quantity of plants to the target inventory.

4.1.2 Constructor & Destructor Documentation

4.1.2.1 AddStock()

```
AddStock::AddStock (
    Plant * plant,
    int quantityToAdd,
    PlantInventory * targetInventory )
```

Constructs an [AddStock](#) command.

Parameters

<i>plant</i>	Plant to add
<i>quantityToAdd</i>	Quantity to add
<i>targetInventory</i>	Inventory to add to

4.1.3 Member Function Documentation

4.1.3.1 execute()

```
bool AddStock::execute ( ) [override], [virtual]
```

Executes the add stock command.

Returns

true if successful, false otherwise

Implements [StockCommand](#).

The documentation for this class was generated from the following files:

- [AddStock.h](#)
- [AddStock.cpp](#)

4.2 AskQuery Class Reference

Director class that simplifies [Query](#) object creation using builders.

```
#include <AskQuery.h>
```

Public Member Functions

- [Query](#) * [describe](#) ([Plant](#) *item)
Creates a [Query](#) requesting description of a specific [Plant](#).
- [Query](#) * [careRoutine](#) ([Plant](#) *item)
Creates a [Query](#) requesting care routine for a specific [Plant](#).

4.2.1 Detailed Description

Director class that simplifies [Query](#) object creation using builders.

This class manages temporary [Builder](#) instances to construct [Query](#) objects. It abstracts the builder selection and configuration process, providing simple methods for common query types.

4.2.2 Member Function Documentation

4.2.2.1 [careRoutine\(\)](#)

```
Query * AskQuery::careRoutine (  
    Plant * item )
```

Creates a [Query](#) requesting care routine for a specific [Plant](#).

Creates a CARE ROUTINE [Query](#) for a specific [Plant](#).

Parameters

<i>item</i>	Pointer to the Plant to get care info for (non-owning)
-------------	--

Returns

Newly allocated [Query](#) object (caller takes ownership)

Uses [ItemQueryBuilder](#) to create a CARE ROUTINE-type [Query](#) targeting the specified [Plant](#).

Parameters

<i>item</i>	Non-owning pointer to the Plant to get care info for
-------------	--

Returns

Newly allocated [Query](#) pointer (caller must delete)

Creates and configures an [ItemQueryBuilder](#) to construct a [Query](#) that requests care routine information for the specified [Plant](#). The temporary builder is deleted after [Query](#) construction.

4.2.2.2 describe()

```
Query * AskQuery::describe (  
    Plant * item )
```

Creates a [Query](#) requesting description of a specific [Plant](#).

Creates an INFO [Query](#) for a specific [Plant](#).

Parameters

<i>item</i>	Pointer to the Plant to describe (non-owning)
-------------	---

Returns

Newly allocated [Query](#) object (caller takes ownership)

Uses [ItemQueryBuilder](#) to create an INFO-type [Query](#) targeting the specified [Plant](#).

Parameters

<i>item</i>	Non-owning pointer to the Plant to describe
-------------	---

Returns

Newly allocated [Query](#) pointer (caller must delete)

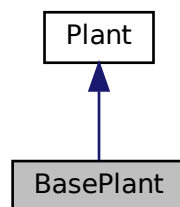
Creates and configures an [ItemQueryBuilder](#) to construct a [Query](#) that requests information about the specified [Plant](#). The temporary builder is deleted after [Query](#) construction.

The documentation for this class was generated from the following files:

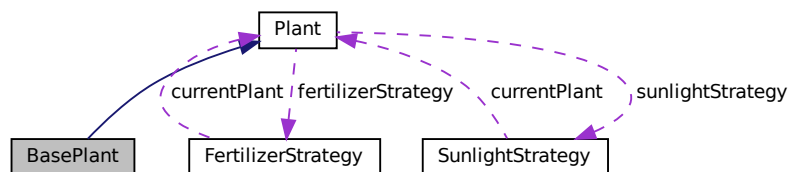
- [AskQuery.h](#)
- [AskQuery.cpp](#)

4.3 BasePlant Class Reference

Inheritance diagram for BasePlant:



Collaboration diagram for BasePlant:



Public Member Functions

- **BasePlant** (double p, std::string desc)
- std::string [getDescription](#) () const override
Gets the plant description.
- double [getPrice](#) () const override
Gets the plant price.

Additional Inherited Members

4.3.1 Member Function Documentation

4.3.1.1 getDescription()

```
std::string BasePlant::getDescription ( ) const [override], [virtual]
```

Gets the plant description.

Returns

Description string

Reimplemented from [Plant](#).

4.3.1.2 getPrice()

```
double BasePlant::getPrice ( ) const [override], [virtual]
```

Gets the plant price.

Returns

Price value

Reimplemented from [Plant](#).

The documentation for this class was generated from the following files:

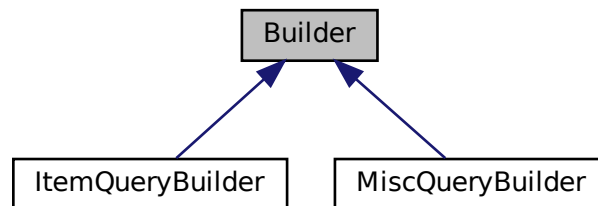
- BasePlant.h
- BasePlant.cpp

4.4 Builder Class Reference

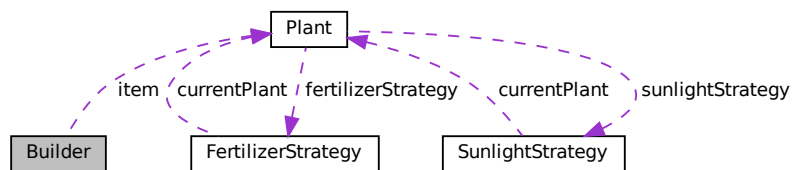
Abstract builder for constructing [Query](#) objects.

```
#include <Builder.h>
```

Inheritance diagram for Builder:



Collaboration diagram for Builder:



Public Member Functions

- virtual void **setType** (std::string **type**)=0
Set the query type/category.
- virtual void **setQuestion** (std::string **question**)=0
Set the explicit question text for the [Query](#).
- virtual void **setItem** ([Plant](#) ***item**)=0
Set the [Plant](#) item targeted by the [Query](#).
- [Query](#) * **build** ()
Construct a [Query](#) from the builder's current state.
- [Builder](#) ()
Default constructor initializes members to safe defaults.

Protected Attributes

- [Plant](#) * **item** = nullptr
- std::string **type** = ""
- std::string **question** = ""

Friends

- class **Query**

4.4.1 Detailed Description

Abstract builder for constructing **Query** objects.

Provides setters for configuring type, question, and target **Plant** items. Concrete subclasses customize the construction process.

4.4.2 Member Function Documentation

4.4.2.1 build()

```
Query * Builder::build ( )
```

Construct a **Query** from the builder's current state.

Build a **Query** using this builder's state.

Returns

Newly allocated **Query** instance (caller owns).

The **Query** constructor reads the **Builder**'s protected members to initialize itself. The returned **Query** is dynamically allocated and must be deleted by the caller.

Returns

Pointer to a newly allocated **Query**.

4.4.2.2 setItem()

```
virtual void Builder::setItem (
    Plant * item ) [pure virtual]
```

Set the **Plant** item targeted by the **Query**.

Parameters

<i>item</i>	Non-owning pointer to the Plant .
-------------	--

Implemented in [MiscQueryBuilder](#), and [ItemQueryBuilder](#).

4.4.2.3 setQuestion()

```
virtual void Builder::setQuestion (
    std::string question ) [pure virtual]
```

Set the explicit question text for the [Query](#).

Parameters

<i>question</i>	Question text to include in the Query .
-----------------	---

Implemented in [MiscQueryBuilder](#), and [ItemQueryBuilder](#).

4.4.2.4 setType()

```
virtual void Builder::setType (
    std::string type ) [pure virtual]
```

Set the query type/category.

Parameters

<i>type</i>	String describing the type (e.g. "INFO", "CARE ROUTINE", "STOCK").
-------------	--

Implemented in [MiscQueryBuilder](#), and [ItemQueryBuilder](#).

4.4.3 Member Data Documentation

4.4.3.1 item

```
Plant* Builder::item =nullptr [protected]
```

optional target item for item-specific queries

4.4.3.2 question

```
std::string Builder::question ="" [protected]
```

optional explicit question text

4.4.3.3 type

```
std::string Builder::type = "" [protected]
```

type/category of query (e.g. "INFO", "STOCK")

The documentation for this class was generated from the following files:

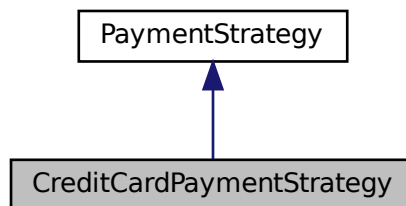
- [Builder.h](#)
- [Builder.cpp](#)

4.5 CreditCardPaymentStrategy Class Reference

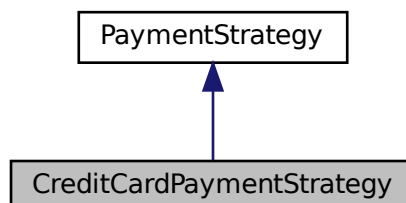
Payment strategy implementation using credit card information.

```
#include <CreditCardPaymentStrategy.h>
```

Inheritance diagram for CreditCardPaymentStrategy:



Collaboration diagram for CreditCardPaymentStrategy:



Public Member Functions

- [CreditCardPaymentStrategy](#) (const std::string &cardNumber)
Construct a [CreditCardPaymentStrategy](#).
- void [pay](#) (double amount, const std::string &orderNum) const override
Execute a credit card payment.
- std::string [getName](#) () const override
Retrieves the payment strategy name.

4.5.1 Detailed Description

Payment strategy implementation using credit card information.

Stores a card number and defines the logic to execute a payment using it.

4.5.2 Constructor & Destructor Documentation

4.5.2.1 CreditCardPaymentStrategy()

```
CreditCardPaymentStrategy::CreditCardPaymentStrategy (
    const std::string & cardNumber )
```

Construct a [CreditCardPaymentStrategy](#).

Parameters

<i>cardNumber</i>	The credit card number as a string.
-------------------	-------------------------------------

4.5.3 Member Function Documentation

4.5.3.1 getName()

```
std::string CreditCardPaymentStrategy::getName ( ) const [override], [virtual]
```

Retrieves the payment strategy name.

Returns

String representing the payment method type.

Implements [PaymentStrategy](#).

4.5.3.2 pay()

```
void CreditCardPaymentStrategy::pay (
    double amount,
    const std::string & orderNum ) const [override], [virtual]
```

Execute a credit card payment.

Parameters

<i>amount</i>	The total amount to charge.
<i>orderNum</i>	Identifier for the order being paid.

Implements [PaymentStrategy](#).

The documentation for this class was generated from the following files:

- [CreditCardPaymentStrategy.h](#)
- [CreditCardPaymentStrategy.cpp](#)

4.6 Customer Class Reference

Represents a store customer and their interactions.

```
#include <Customer.h>
```

Public Member Functions

- [Customer](#) ()
Default constructor creates an unnamed customer.
- [Customer](#) (const std::string &customerName)
Construct a [Customer](#) with a specific name.
- std::string [getName](#) () const
Retrieve the customer's name.
- void [setName](#) (const std::string &customerName)
Set or update the customer's name.
- void [placeOrder](#) (const [Order](#) &order)
Add an order to the customer's order history.
- void [viewOrders](#) () const
Print a summary of all customer orders to the console.
- void [makeQuery](#) ([Staff](#) &staff, [Query](#) *queryMessage) const
Submit a query message to a [Staff](#) member.

4.6.1 Detailed Description

Represents a store customer and their interactions.

Stores customer data, handles order placement, and supports queries directed at staff members.

4.6.2 Constructor & Destructor Documentation

4.6.2.1 Customer()

```
Customer::Customer (
    const std::string & customerName ) [explicit]
```

Construct a [Customer](#) with a specific name.

Parameters

<i>customerName</i>	The customer's name.
---------------------	----------------------

4.6.3 Member Function Documentation

4.6.3.1 getName()

```
std::string Customer::getName ( ) const
```

Retrieve the customer's name.

Returns

Name of the customer.

4.6.3.2 makeQuery()

```
void Customer::makeQuery (
    Staff & staff,
    Query * queryMessage ) const
```

Submit a query message to a [Staff](#) member.

Parameters

<i>staff</i>	Reference to the staff member receiving the query.
<i>queryMessage</i>	Pointer to the query message (caller owns).

4.6.3.3 placeOrder()

```
void Customer::placeOrder (
    const Order & order )
```

Add an order to the customer's order history.

Parameters

<i>order</i>	The order to add.
--------------	-------------------

4.6.3.4 setName()

```
void Customer::setName (
    const std::string & customerName )
```

Set or update the customer's name.

Parameters

<i>customerName</i>	The new name to assign.
---------------------	-------------------------

The documentation for this class was generated from the following files:

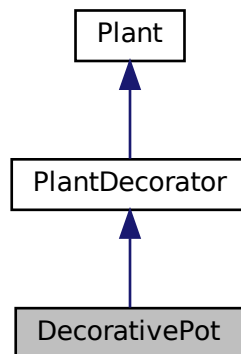
- [Customer.h](#)
- Customer.cpp

4.7 DecorativePot Class Reference

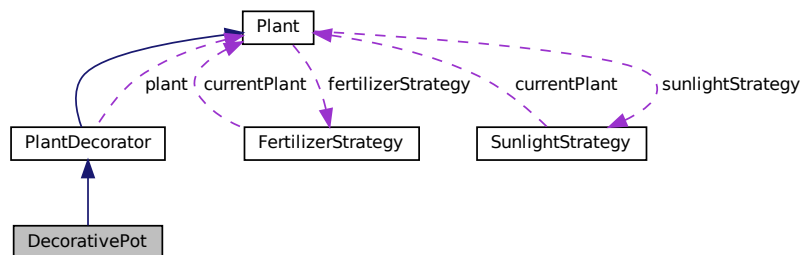
Concrete decorator that adds a decorative pot to a [Plant](#).

```
#include <DecorativePot.h>
```

Inheritance diagram for DecorativePot:



Collaboration diagram for DecorativePot:



Public Member Functions

- [DecorativePot](#) ([Plant](#) *plant)
Construct a [DecorativePot](#) decorator for the given [Plant](#).
- `std::string` [getDescription](#) () const override
Get the description including decorative pot details.
- `double` [getPrice](#) () const override
Get the total price including the decorative pot.

Additional Inherited Members

4.7.1 Detailed Description

Concrete decorator that adds a decorative pot to a [Plant](#).

Enhances the plant's visual appeal and increases its price.

4.7.2 Constructor & Destructor Documentation

4.7.2.1 DecorativePot()

```
DecorativePot::DecorativePot (
    Plant * plant )
```

Construct a [DecorativePot](#) decorator for the given [Plant](#).

Parameters

<i>plant</i>	Non-owning pointer to the Plant being decorated.
--------------	--

4.7.3 Member Function Documentation

4.7.3.1 getDescription()

```
std::string DecorativePot::getDescription ( ) const [override], [virtual]
```

Get the description including decorative pot details.

Returns

Augmented description string.

Reimplemented from [PlantDecorator](#).

4.7.3.2 getPrice()

```
double DecorativePot::getPrice ( ) const [override], [virtual]
```

Get the total price including the decorative pot.

Returns

Combined price as a double.

Reimplemented from [PlantDecorator](#).

The documentation for this class was generated from the following files:

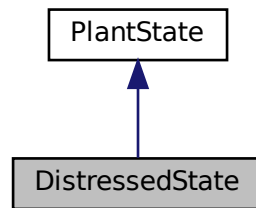
- [DecorativePot.h](#)
- [DecorativePot.cpp](#)

4.8 DistressedState Class Reference

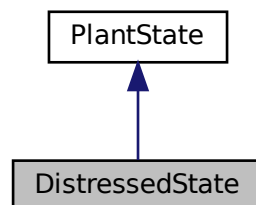
Concrete state representing a plant in distress.

```
#include <DistressedState.h>
```

Inheritance diagram for DistressedState:



Collaboration diagram for DistressedState:



Public Member Functions

- void **applyCare** (**PlantLifeCycle** *context, **Plant** *plant, **PlantCareRoutine** *routine) override
Apply care to a distressed plant.
- bool **evaluate** (**PlantLifeCycle** *context, **Plant** *plant) override
Evaluate whether the plant remains in distress or recovers.
- std::string **getName** () const override
Retrieve the name of this state.
- **PlantState** * **clone** () const override
Create a deep copy of this state.

4.8.1 Detailed Description

Concrete state representing a plant in distress.

In this state, the plant exhibits poor health. Applying the correct care routine may allow it to recover to a normal or thriving state.

4.8.2 Member Function Documentation

4.8.2.1 applyCare()

```
void DistressedState::applyCare (
    PlantLifeCycle * context,
    Plant * plant,
    PlantCareRoutine * routine ) [override], [virtual]
```

Apply care to a distressed plant.

Parameters

<i>context</i>	The lifecycle context controlling state transitions.
<i>plant</i>	Pointer to the affected plant.
<i>routine</i>	The care routine being applied.

Implements [PlantState](#).

4.8.2.2 clone()

```
PlantState * DistressedState::clone ( ) const [override], [virtual]
```

Create a deep copy of this state.

Returns

New [DistressedState](#) instance.

Implements [PlantState](#).

4.8.2.3 evaluate()

```
bool DistressedState::evaluate (
    PlantLifeCycle * context,
    Plant * plant ) [override], [virtual]
```

Evaluate whether the plant remains in distress or recovers.

Parameters

<i>context</i>	The lifecycle context.
<i>plant</i>	Pointer to the plant being evaluated.

Returns

True if state change occurs, otherwise false.

Implements [PlantState](#).

4.8.2.4 getName()

```
std::string DistressedState::getName ( ) const [override], [virtual]
```

Retrieve the name of this state.

Returns

String "Distressed".

Implements [PlantState](#).

The documentation for this class was generated from the following files:

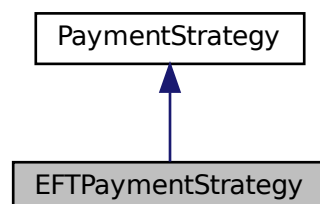
- [DistressedState.h](#)
- DistressedState.cpp

4.9 EFTPayerStrategy Class Reference

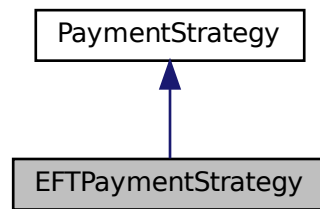
Payment strategy implementation for EFT transactions.

```
#include <EFTPayerStrategy.h>
```

Inheritance diagram for EFTPayerStrategy:



Collaboration diagram for EFTPayerStrategy:



Public Member Functions

- `EFTPayerStrategy` (const std::string &accountNumber)
Construct an `EFTPayerStrategy` with a specific account number.
- void `pay` (double amount, const std::string &orderNum) const override
Process a payment via EFT.
- std::string `getName` () const override
Retrieve the strategy's display name.

4.9.1 Detailed Description

Payment strategy implementation for EFT transactions.

Handles payments made via direct bank transfers using an account number.

4.9.2 Constructor & Destructor Documentation

4.9.2.1 EFTPayerStrategy()

```
EFTPayerStrategy::EFTPayerStrategy (  
    const std::string & accountNumber )
```

Construct an `EFTPayerStrategy` with a specific account number.

Parameters

<code>accountNumber</code>	Bank account number used for transfer.
----------------------------	--

4.9.3 Member Function Documentation

4.9.3.1 getName()

```
std::string EFTPayerStrategy::getName ( ) const [override], [virtual]
```

Retrieve the strategy's display name.

Returns

"EFT Payment".

Implements [PaymentStrategy](#).

4.9.3.2 pay()

```
void EFTPayerStrategy::pay (
    double amount,
    const std::string & orderNum ) const [override], [virtual]
```

Process a payment via EFT.

Parameters

<i>amount</i>	Amount to transfer.
<i>orderNum</i>	Associated order number.

Implements [PaymentStrategy](#).

The documentation for this class was generated from the following files:

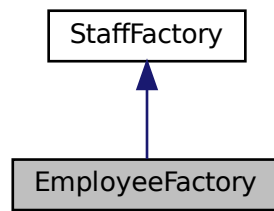
- [EFTPayerStrategy.h](#)
- [EFTPayerStrategy.cpp](#)

4.10 EmployeeFactory Class Reference

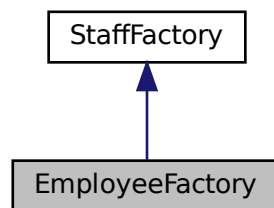
Factory responsible for creating regular staff members.

```
#include <EmployeeFactory.h>
```

Inheritance diagram for EmployeeFactory:



Collaboration diagram for EmployeeFactory:



Public Member Functions

- [EmployeeFactory](#) ()
Default constructor.
- [~EmployeeFactory](#) ()
Virtual destructor.
- [FloorStaff](#) * [createFloorStaff](#) (std::string name) override
Create a [FloorEmployee](#) instance.
- [SalesStaff](#) * [createSalesStaff](#) (std::string name) override
Create a [SalesEmployee](#) instance.

4.10.1 Detailed Description

Factory responsible for creating regular staff members.

4.10.2 Member Function Documentation

4.10.2.1 createFloorStaff()

```
FloorStaff * EmployeeFactory::createFloorStaff (
    std::string name ) [override], [virtual]
```

Create a [FloorEmployee](#) instance.

Parameters

<i>name</i>	The name of the employee to create
-------------	------------------------------------

Returns

Pointer to new [FloorEmployee](#) (caller owns memory)

Implements [StaffFactory](#).

4.10.2.2 createSalesStaff()

```
SalesStaff * EmployeeFactory::createSalesStaff (
    std::string name ) [override], [virtual]
```

Create a [SalesEmployee](#) instance.

Parameters

<i>name</i>	The name of the employee to create
-------------	------------------------------------

Returns

Pointer to new [SalesEmployee](#) (caller owns memory)

Implements [StaffFactory](#).

The documentation for this class was generated from the following files:

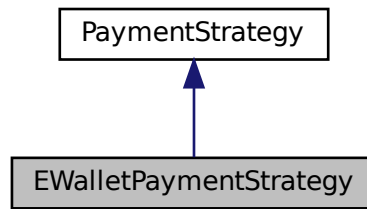
- [EmployeeFactory.h](#)
- [EmployeeFactory.cpp](#)

4.11 EWalletPaymentStrategy Class Reference

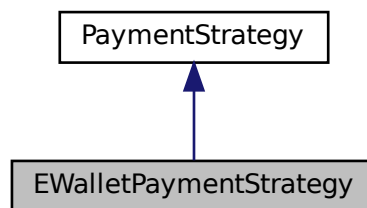
Implements a payment method using an e-wallet ID.

```
#include <EWalletPaymentStrategy.h>
```

Inheritance diagram for `EWalletPaymentStrategy`:



Collaboration diagram for `EWalletPaymentStrategy`:



Public Member Functions

- [EWalletPaymentStrategy](#) (const std::string &walletID)
Construct an [EWalletPaymentStrategy](#).
- void [pay](#) (double amount, const std::string &orderNum) const override
Execute payment using the specified e-wallet.
- std::string [getName](#) () const override
Retrieve the strategy's display name.

4.11.1 Detailed Description

Implements a payment method using an e-wallet ID.

Processes payments through digital wallet services.

4.11.2 Constructor & Destructor Documentation

4.11.2.1 EWalletPaymentStrategy()

```
EWalletPaymentStrategy::EWalletPaymentStrategy (
    const std::string & walletID )
```

Construct an [EWalletPaymentStrategy](#).

Parameters

<i>walletID</i>	Identifier of the wallet to charge.
-----------------	-------------------------------------

4.11.3 Member Function Documentation

4.11.3.1 getName()

```
std::string EWalletPaymentStrategy::getName ( ) const [override], [virtual]
```

Retrieve the strategy's display name.

Returns

"E-Wallet Payment".

Implements [PaymentStrategy](#).

4.11.3.2 pay()

```
void EWalletPaymentStrategy::pay (
    double amount,
    const std::string & orderNum ) const [override], [virtual]
```

Execute payment using the specified e-wallet.

Parameters

<i>amount</i>	Amount to deduct.
<i>orderNum</i>	The order number linked to the payment.

Implements [PaymentStrategy](#).

The documentation for this class was generated from the following files:

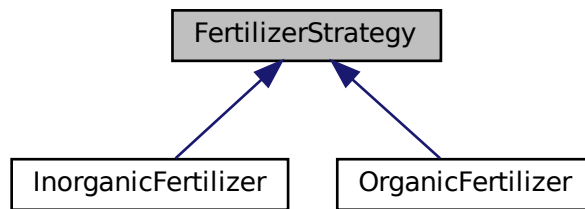
- [EWalletPaymentStrategy.h](#)
- [EWalletPaymentStrategy.cpp](#)

4.12 FertilizerStrategy Class Reference

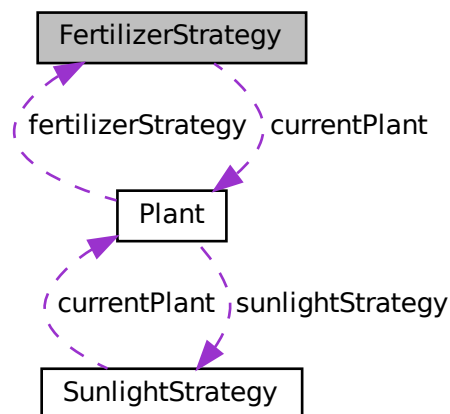
Abstract base for fertilizer behavior strategies.

```
#include <FertilizerStrategy.h>
```

Inheritance diagram for FertilizerStrategy:



Collaboration diagram for FertilizerStrategy:



Public Member Functions

- virtual void `applyFertilizer (Plant *p)=0`
Applies the fertilizer strategy to a plant. This is a pure virtual function that must be implemented by concrete sub-classes.
- virtual std::string `getStrategyName () const =0`
Gets the name of the fertilizer strategy. This is a pure virtual function that must be implemented by concrete sub-classes.

Protected Attributes

- [Plant](#) * `currentPlant`

4.12.1 Detailed Description

Abstract base for fertilizer behavior strategies.

4.12.2 Member Function Documentation

4.12.2.1 `applyFertilizer()`

```
virtual void FertilizerStrategy::applyFertilizer (  
    Plant * p ) [pure virtual]
```

Applies the fertilizer strategy to a plant. This is a pure virtual function that must be implemented by concrete subclasses.

Parameters

<i>p</i>	The plant to apply the fertilizer strategy to.
----------	--

Implemented in [OrganicFertilizer](#), and [InorganicFertilizer](#).

4.12.2.2 `getStrategyName()`

```
virtual std::string FertilizerStrategy::getStrategyName ( ) const [pure virtual]
```

Gets the name of the fertilizer strategy. This is a pure virtual function that must be implemented by concrete subclasses.

Returns

The name of the fertilizer strategy.

Implemented in [OrganicFertilizer](#), and [InorganicFertilizer](#).

4.12.3 Member Data Documentation

4.12.3.1 `currentPlant`

```
Plant* FertilizerStrategy::currentPlant [protected]
```

Pointer to the plant being fertilized.

The documentation for this class was generated from the following file:

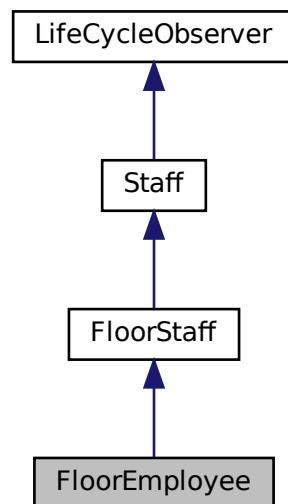
- [FertilizerStrategy.h](#)

4.13 FloorEmployee Class Reference

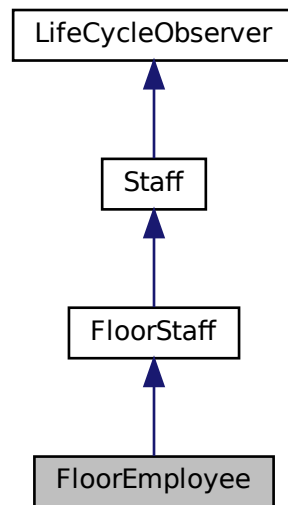
Regular employee handling on-floor customer queries.

```
#include <FloorEmployee.h>
```

Inheritance diagram for FloorEmployee:



Collaboration diagram for FloorEmployee:



Public Member Functions

- [FloorEmployee](#) (std::string name)
Construct a floor employee with given name.
- virtual [~FloorEmployee](#) ()
Virtual destructor.
- bool [staffCanHandle](#) (std::string type) override
Determine whether this staff member can handle queries of the given type.
- void [reply](#) ([Query](#) *query) override
Produce a reply for the provided [Query](#).

4.13.1 Detailed Description

Regular employee handling on-floor customer queries.

4.13.2 Constructor & Destructor Documentation

4.13.2.1 FloorEmployee()

```
FloorEmployee::FloorEmployee (  
    std::string name )
```

Construct a floor employee with given name.

Parameters

<i>name</i>	The employee's name
-------------	---------------------

4.13.3 Member Function Documentation

4.13.3.1 `reply()`

```
void FloorEmployee::reply (
    Query * query ) [override], [virtual]
```

Produce a reply for the provided [Query](#).

Called when `staffCanHandle(query->getType())` returns true. Implementations should output or otherwise process the [Query](#) and may forward it if further escalation is required.

Parameters

<i>query</i>	Pointer to the Query to reply to (non-owning).
--------------	--

Implements [FloorStaff](#).

4.13.3.2 `staffCanHandle()`

```
bool FloorEmployee::staffCanHandle (
    std::string type ) [override], [virtual]
```

Determine whether this staff member can handle queries of the given type.

Implementations should return true for supported [Query](#) types (e.g. "INFO", "STOCK", "CARE ROUTINE") and false otherwise.

Parameters

<i>type</i>	The Query type string.
-------------	--

Returns

true if this staff member can handle the type, false otherwise.

Implements [FloorStaff](#).

The documentation for this class was generated from the following files:

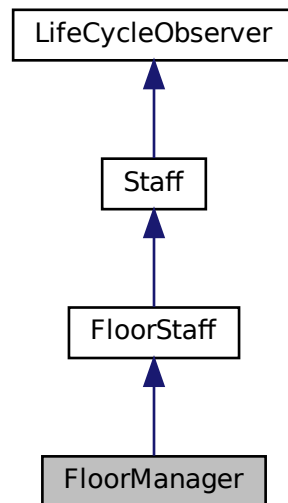
- [FloorEmployee.h](#)
- [FloorEmployee.cpp](#)

4.14 FloorManager Class Reference

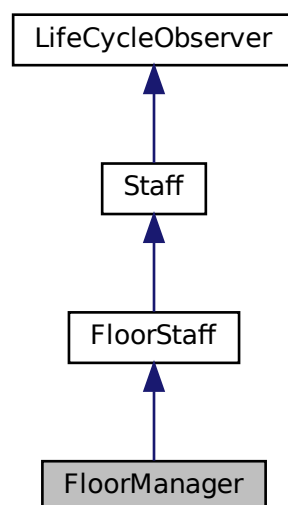
Manages floor staff and handles high-level queries.

```
#include <FloorManager.h>
```

Inheritance diagram for FloorManager:



Collaboration diagram for FloorManager:



Public Member Functions

- [FloorManager](#) (std::string name)
Construct a floor manager with given name.
- virtual [~FloorManager](#) ()
Virtual destructor.
- bool [staffCanHandle](#) (std::string type) override
Determine whether this staff member can handle queries of the given type.
- void [reply](#) ([Query](#) *query) override
Produce a reply for the provided [Query](#).

4.14.1 Detailed Description

Manages floor staff and handles high-level queries.

4.14.2 Constructor & Destructor Documentation

4.14.2.1 FloorManager()

```
FloorManager::FloorManager (
    std::string name )
```

Construct a floor manager with given name.

Parameters

<i>name</i>	The manager's name
-------------	--------------------

4.14.3 Member Function Documentation

4.14.3.1 reply()

```
void FloorManager::reply (
    Query * query ) [override], [virtual]
```

Produce a reply for the provided [Query](#).

Called when `staffCanHandle(query->getType())` returns true. Implementations should output or otherwise process the [Query](#) and may forward it if further escalation is required.

Parameters

<i>query</i>	Pointer to the Query to reply to (non-owning).
--------------	--

Implements [FloorStaff](#).

4.14.3.2 staffCanHandle()

```
bool FloorManager::staffCanHandle (
    std::string type ) [override], [virtual]
```

Determine whether this staff member can handle queries of the given type.

Implementations should return true for supported [Query](#) types (e.g. "INFO", "STOCK", "CARE ROUTINE") and false otherwise.

Parameters

<i>type</i>	The Query type string.
-------------	--

Returns

true if this staff member can handle the type, false otherwise.

Implements [FloorStaff](#).

The documentation for this class was generated from the following files:

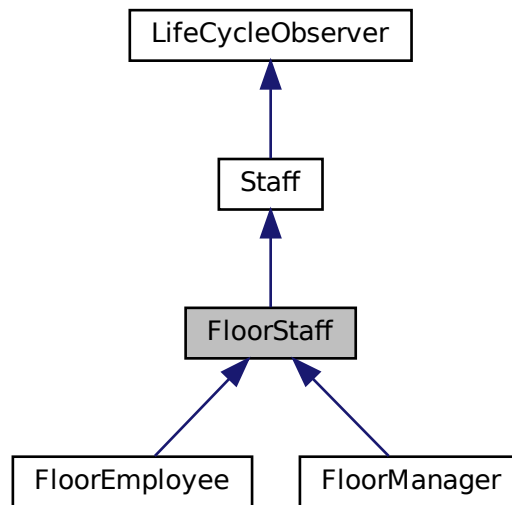
- [FloorManager.h](#)
- [FloorManager.cpp](#)

4.15 FloorStaff Class Reference

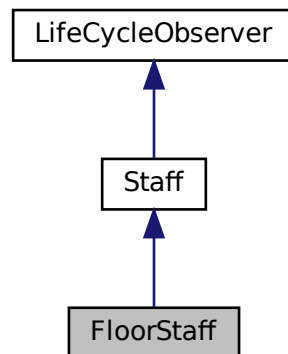
Base class for all floor staff types.

```
#include <FloorStaff.h>
```

Inheritance diagram for FloorStaff:



Collaboration diagram for FloorStaff:



Public Member Functions

- **FloorStaff** (std::string name)
*Construct a **FloorStaff** with the given name.*
- virtual ~**FloorStaff** ()
Virtual destructor.
- void **handleQuery** (**Query** *query) override

Entry point for handling a [Query](#).

- virtual bool [staffCanHandle](#) (std::string type)=0

Determine whether this staff member can handle queries of the given type.

- virtual void [reply](#) ([Query](#) *query)=0

Produce a reply for the provided [Query](#).

4.15.1 Detailed Description

Base class for all floor staff types.

4.15.2 Constructor & Destructor Documentation

4.15.2.1 FloorStaff()

```
FloorStaff::FloorStaff (
    std::string name )
```

Construct a [FloorStaff](#) with the given name.

Parameters

<i>name</i>	Staff member's name.
-------------	--------------------------------------

4.15.3 Member Function Documentation

4.15.3.1 handleQuery()

```
void FloorStaff::handleQuery (
    Query * query ) [override], [virtual]
```

Entry point for handling a [Query](#).

This override logs receipt of the [Query](#), prints its contents, asks [staffCanHandle\(\)](#) whether the current instance can service it, calls [reply\(\)](#) if it can, or forwards the [Query](#) to the next [Staff](#) in the chain when it cannot be handled.

Parameters

<i>query</i>	Pointer to the Query to handle (non-owning).
--------------	--

Implements [Staff](#).

4.15.3.2 reply()

```
virtual void FloorStaff::reply (
    Query * query ) [pure virtual]
```

Produce a reply for the provided [Query](#).

Called when `staffCanHandle(query->getType())` returns true. Implementations should output or otherwise process the [Query](#) and may forward it if further escalation is required.

Parameters

<i>query</i>	Pointer to the Query to reply to (non-owning).
--------------	--

Implemented in [FloorManager](#), and [FloorEmployee](#).

4.15.3.3 staffCanHandle()

```
virtual bool FloorStaff::staffCanHandle (
    std::string type ) [pure virtual]
```

Determine whether this staff member can handle queries of the given type.

Implementations should return true for supported [Query](#) types (e.g. "INFO", "STOCK", "CARE ROUTINE") and false otherwise.

Parameters

<i>type</i>	The Query type string.
-------------	--

Returns

true if this staff member can handle the type, false otherwise.

Implemented in [FloorManager](#), and [FloorEmployee](#).

The documentation for this class was generated from the following files:

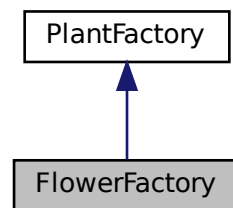
- [FloorStaff.h](#)
- [FloorStaff.cpp](#)

4.16 FlowerFactory Class Reference

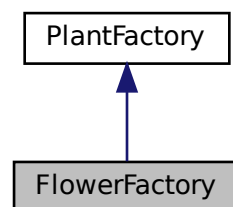
Concrete factory for creating flower-type Plants.

```
#include <FlowerFactory.h>
```

Inheritance diagram for FlowerFactory:



Collaboration diagram for FlowerFactory:



Public Member Functions

- `Plant * createPlant (std::string plantType, double price)` override
Create a flower plant of the specified type.

4.16.1 Detailed Description

Concrete factory for creating flower-type Plants.

Generates `Plant` instances based on flower types requested by clients.

4.16.2 Member Function Documentation

4.16.2.1 createPlant()

```
Plant * FlowerFactory::createPlant (
    std::string plantType = "Rose",
    double price = 100 ) [override], [virtual]
```

Create a flower plant of the specified type.

Parameters

<i>plantType</i>	The type of flower to create (e.g., "Rose").
<i>price</i>	The initial price of the flower.

Returns

Pointer to a newly created [Plant](#) instance.

Implements [PlantFactory](#).

The documentation for this class was generated from the following files:

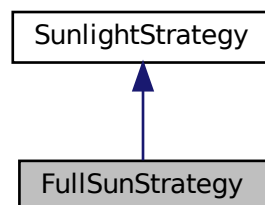
- [FlowerFactory.h](#)
- FlowerFactory.cpp

4.17 FullSunStrategy Class Reference

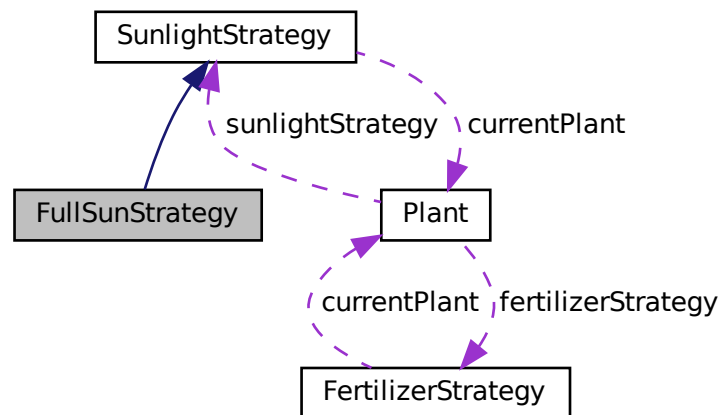
Strategy for applying full sunlight to a [Plant](#).

```
#include <FullSunStrategy.h>
```

Inheritance diagram for FullSunStrategy:



Collaboration diagram for FullSunStrategy:



Public Member Functions

- void `applySunlight` (`Plant *p`) override
Apply the full sun strategy to a `Plant`.
- std::string `getStrategyName` () const override
Get the name of this strategy.

Additional Inherited Members

4.17.1 Detailed Description

Strategy for applying full sunlight to a `Plant`.

Adjusts sunlight exposure to simulate plants requiring high sunlight levels.

4.17.2 Member Function Documentation

4.17.2.1 `applySunlight()`

```
void FullSunStrategy::applySunlight (
    Plant * p ) [override], [virtual]
```

Apply the full sun strategy to a `Plant`.

Parameters

<i>p</i>	Pointer to the Plant being exposed to sunlight.
----------	---

Note

Increases sunlight exposure by 60% (hours/day).

Warning

If exposure exceeds the max value, it is capped at the maximum.

Implements [SunlightStrategy](#).

4.17.2.2 `getStrategyName()`

```
std::string FullSunStrategy::getStrategyName ( ) const [override], [virtual]
```

Get the name of this strategy.

Returns

"Full Sun Strategy".

Implements [SunlightStrategy](#).

The documentation for this class was generated from the following files:

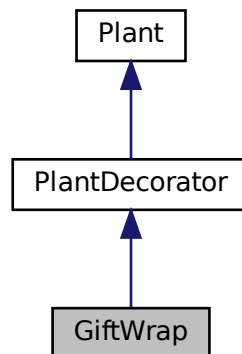
- [FullSunStrategy.h](#)
- FullSunStrategy.cpp

4.18 GiftWrap Class Reference

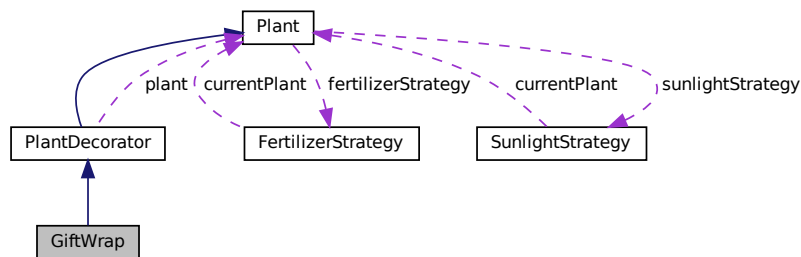
Concrete decorator that adds gift wrapping to a [Plant](#).

```
#include <GiftWrap.h>
```

Inheritance diagram for GiftWrap:



Collaboration diagram for GiftWrap:



Public Member Functions

- [GiftWrap](#) ([Plant](#) *[plant](#))
Construct a [GiftWrap](#) decorator for the given [Plant](#).
- `std::string` [getDescription](#) () const override
Get the plant's description including gift wrap details.
- `double` [getPrice](#) () const override
Get the plant's price including gift wrap cost.

Additional Inherited Members

4.18.1 Detailed Description

Concrete decorator that adds gift wrapping to a [Plant](#).

Enhances the plant's presentation, making it suitable for gifting.

4.18.2 Constructor & Destructor Documentation

4.18.2.1 GiftWrap()

```
GiftWrap::GiftWrap (
    Plant * plant )
```

Construct a [GiftWrap](#) decorator for the given [Plant](#).

Parameters

<i>plant</i>	Non-owning pointer to the Plant being decorated.
--------------	--

4.18.3 Member Function Documentation

4.18.3.1 getDescription()

```
std::string GiftWrap::getDescription ( ) const [override], [virtual]
```

Get the plant's description including gift wrap details.

Returns

Enhanced description string.

Reimplemented from [PlantDecorator](#).

4.18.3.2 getPrice()

```
double GiftWrap::getPrice ( ) const [override], [virtual]
```

Get the plant's price including gift wrap cost.

Returns

Combined price as a double.

Reimplemented from [PlantDecorator](#).

The documentation for this class was generated from the following files:

- [GiftWrap.h](#)
- [GiftWrap.cpp](#)

4.19 GreenhouseUnitTest Class Reference

Unit tests for greenhouse plant system (singleton, factories and commands design patterns)

```
#include <GreenhouseUnitTest.h>
```

Static Public Member Functions

- static void `run` ()
Runs all tests.

4.19.1 Detailed Description

Unit tests for greenhouse plant system (singleton, factories and commands design patterns)

The documentation for this class was generated from the following files:

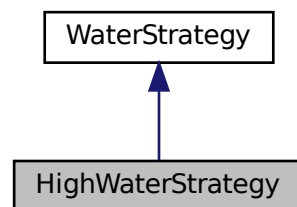
- GreenhouseUnitTest.h
- GreenhouseUnitTest.cpp

4.20 HighWaterStrategy Class Reference

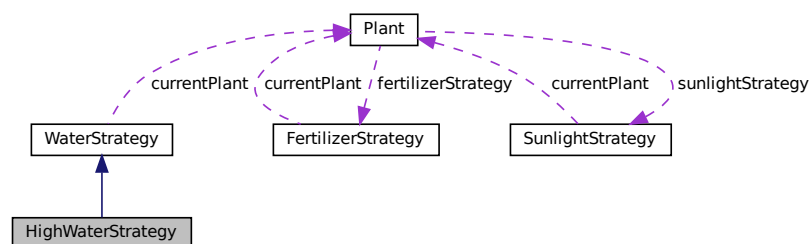
Strategy for applying a high amount of water to a plant.

```
#include <HighWaterStrategy.h>
```

Inheritance diagram for HighWaterStrategy:



Collaboration diagram for HighWaterStrategy:



Public Member Functions

- void [applyWater](#) ([Plant](#) *p) override
Applies the high water strategy to a plant.
- std::string [getStrategyName](#) () const override
Gets the name of this watering strategy.

Additional Inherited Members

4.20.1 Detailed Description

Strategy for applying a high amount of water to a plant.

4.20.2 Member Function Documentation

4.20.2.1 [applyWater\(\)](#)

```
void HighWaterStrategy::applyWater (  
    Plant * p ) [override], [virtual]
```

Applies the high water strategy to a plant.

Increases the plant's water level by 100ml, capped at the plant's maximum water capacity.

Parameters

<i>p</i>	Pointer to the plant receiving water.
----------	---------------------------------------

Implements [WaterStrategy](#).

4.20.2.2 [getStrategyName\(\)](#)

```
std::string HighWaterStrategy::getStrategyName ( ) const [override], [virtual]
```

Gets the name of this watering strategy.

Returns

String representing the strategy name.

Implements [WaterStrategy](#).

The documentation for this class was generated from the following files:

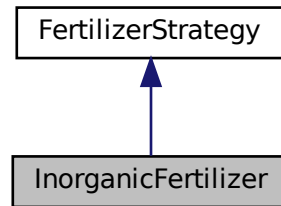
- [HighWaterStrategy.h](#)
- [HighWaterStrategy.cpp](#)

4.21 InorganicFertilizer Class Reference

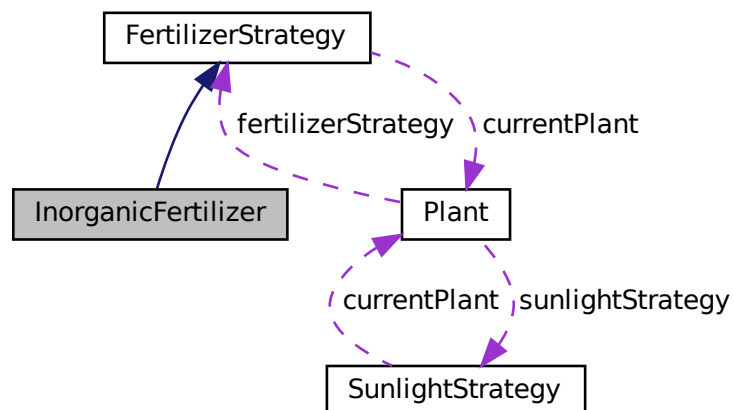
Strategy for applying inorganic fertilizers to a plant.

```
#include <InorganicFertilizer.h>
```

Inheritance diagram for InorganicFertilizer:



Collaboration diagram for InorganicFertilizer:



Public Member Functions

- void `applyFertilizer (Plant *p)` override
Applies the inorganic fertilizer strategy to a plant.
- std::string `getStrategyName ()` const override
Gets the name of this fertilizer strategy.

Additional Inherited Members

4.21.1 Detailed Description

Strategy for applying inorganic fertilizers to a plant.

4.21.2 Member Function Documentation

4.21.2.1 `applyFertilizer()`

```
void InorganicFertilizer::applyFertilizer (
    Plant * p ) [override], [virtual]
```

Applies the inorganic fertilizer strategy to a plant.

Increases the nutrient level by 40ppm, capped at the plant's maximum level.

Parameters

<i>p</i>	Pointer to the plant being fertilized.
----------	--

Implements [FertilizerStrategy](#).

4.21.2.2 `getStrategyName()`

```
std::string InorganicFertilizer::getStrategyName ( ) const [override], [virtual]
```

Gets the name of this fertilizer strategy.

Returns

String representing the strategy name.

Implements [FertilizerStrategy](#).

The documentation for this class was generated from the following files:

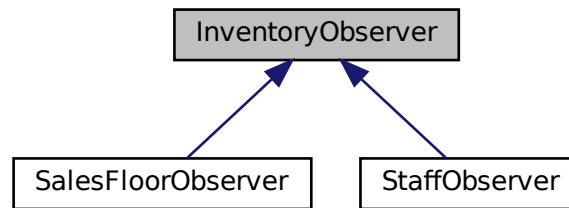
- [InorganicFertilizer.h](#)
- [InorganicFertilizer.cpp](#)

4.22 InventoryObserver Class Reference

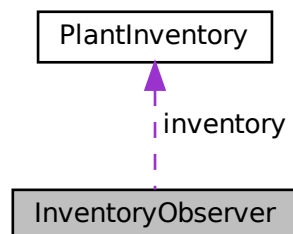
Abstract base class for observers of inventory changes.

```
#include <InventoryObserver.h>
```

Inheritance diagram for InventoryObserver:



Collaboration diagram for InventoryObserver:



Public Member Functions

- **InventoryObserver** (**PlantInventory** ***inventory**)
Constructs an observer and attaches to inventory.
- virtual void **update** ()=0
Pure virtual method called when inventory changes.

Protected Attributes

- **PlantInventory** * **inventory**
Pointer to the observed plant inventory.

4.22.1 Detailed Description

Abstract base class for observers of inventory changes.

This class implements the Observer, allowing objects to be notified when the plant inventory changes.

4.22.2 Constructor & Destructor Documentation

4.22.2.1 InventoryObserver()

```
InventoryObserver::InventoryObserver (
    PlantInventory * inventory )
```

Constructs an observer and attaches to inventory.

Parameters

<i>inventory</i>	Inventory to observe
------------------	----------------------

The documentation for this class was generated from the following files:

- [InventoryObserver.h](#)
- [InventoryObserver.cpp](#)

4.23 InventoryProxy Class Reference

Acts as a proxy to manage plant inventory transactions.

```
#include <InventoryProxy.h>
```

Public Member Functions

- [InventoryProxy](#) ([SalesFloorObserver](#) *observer)
Constructs an [InventoryProxy](#) with sales floor observer.
- bool [buyPlant](#) ([Plant](#) *plant, int quantity)
Processes plant purchase transaction.
- void [addStock](#) ([Plant](#) *plant, int quantity)
Adds stock to inventory.
- void [removeStock](#) ([Plant](#) *, int)
Removes stock from inventory.
- void [displayAvailablePlants](#) () const
Displays currently available plants. Delegates to the sales floor observer for display.

4.23.1 Detailed Description

Acts as a proxy to manage plant inventory transactions.

4.23.2 Constructor & Destructor Documentation

4.23.2.1 InventoryProxy()

```
InventoryProxy::InventoryProxy (  
    SalesFloorObserver * observer )
```

Constructs an [InventoryProxy](#) with sales floor observer.

Parameters

<i>observer</i>	Observer for sales floor updates.
-----------------	-----------------------------------

4.23.3 Member Function Documentation

4.23.3.1 addStock()

```
void InventoryProxy::addStock (  
    Plant * plant,  
    int quantity )
```

Adds stock to inventory.

Parameters

<i>plant</i>	Plant type to add.
<i>quantity</i>	Number of plants to add.

4.23.3.2 buyPlant()

```
bool InventoryProxy::buyPlant (  
    Plant * plant,  
    int quantity )
```

Processes plant purchase transaction.

Parameters

<i>plant</i>	Plant to purchase.
<i>quantity</i>	Number of plants to purchase.

Returns

True if purchase was successful, false otherwise.

4.23.3.3 removeStock()

```
void InventoryProxy::removeStock (
    Plant * ,
    int )
```

Removes stock from inventory.

Parameters

<i>plant</i>	Plant type to remove.
<i>quantity</i>	Number of plants to remove.

The documentation for this class was generated from the following files:

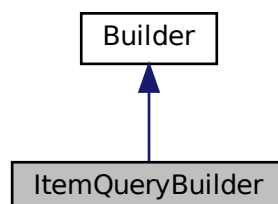
- [InventoryProxy.h](#)
- InventoryProxy.cpp

4.24 ItemQueryBuilder Class Reference

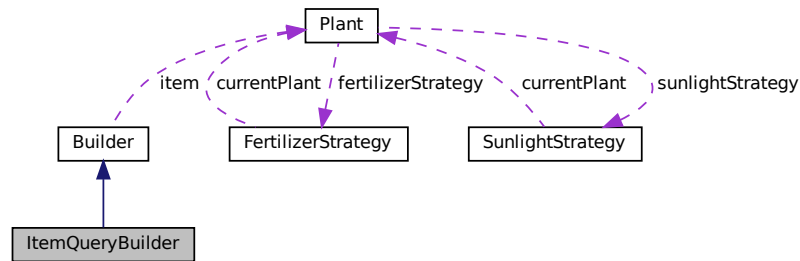
[Builder](#) class for constructing queries about specific plants.

```
#include <ItemQueryBuilder.h>
```

Inheritance diagram for ItemQueryBuilder:



Collaboration diagram for ItemQueryBuilder:



Public Member Functions

- void [setType](#) (std::string [type](#))
Set the query type (e.g. "INFO", "CARE ROUTINE").
- void [setQuestion](#) (std::string [question](#))
Set the explicit question text for the query.
- void [setItem](#) (Plant *[item](#))
Set the [Plant](#) item targeted by the query.

Additional Inherited Members

4.24.1 Detailed Description

[Builder](#) class for constructing queries about specific plants.

4.24.2 Member Function Documentation

4.24.2.1 setItem()

```
void ItemQueryBuilder::setItem (
    Plant * item ) [virtual]
```

Set the [Plant](#) item targeted by the query.

Parameters

<i>item</i>	Non-owning pointer to the Plant .
-------------	---

Implements [Builder](#).

4.24.2.2 setQuestion()

```
void ItemQueryBuilder::setQuestion (
    std::string question ) [virtual]
```

Set the explicit question text for the query.

Set the explicit question text for the item query.

Parameters

<i>question</i>	Question string.
-----------------	------------------

Implements [Builder](#).

4.24.2.3 setType()

```
void ItemQueryBuilder::setType (
    std::string type ) [virtual]
```

Set the query type (e.g. "INFO", "CARE ROUTINE").

Set the query type for the item query.

Parameters

<i>type</i>	Type string.
<i>type</i>	Type string (e.g. "INFO", "CARE ROUTINE").

Implements [Builder](#).

The documentation for this class was generated from the following files:

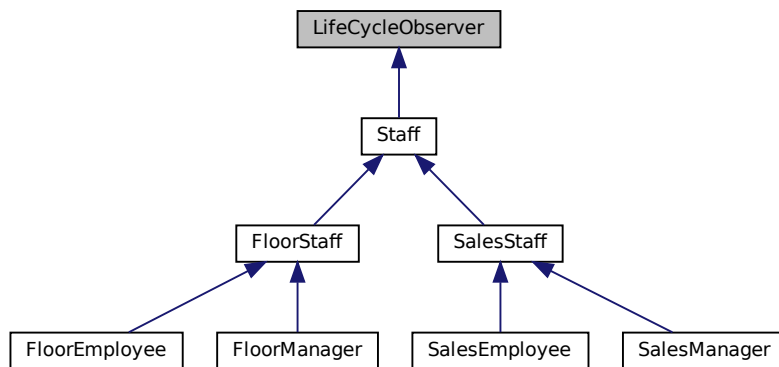
- [ItemQueryBuilder.h](#)
- [ItemQueryBuilder.cpp](#)

4.25 LifeCycleObserver Class Reference

Observer interface for plant life cycle changes.

```
#include <LifeCycleObserver.h>
```

Inheritance diagram for LifeCycleObserver:



Public Member Functions

- virtual void [update](#) ([PlantLifeCycle](#) *p)
Called when the observed plant's life cycle changes.
- virtual [~LifeCycleObserver](#) ()=default
Virtual destructor for proper cleanup.

4.25.1 Detailed Description

Observer interface for plant life cycle changes.

Implement this interface to receive notifications when a plant's life cycle state changes.

4.25.2 Member Function Documentation

4.25.2.1 [update\(\)](#)

```
void LifeCycleObserver::update (
    PlantLifeCycle * p ) [virtual]
```

Called when the observed plant's life cycle changes.

Parameters

<i>p</i>	Pointer to the PlantLifeCycle that changed.
----------	---

The documentation for this class was generated from the following files:

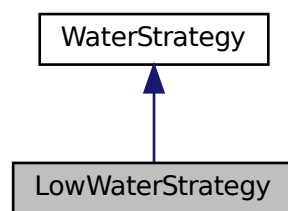
- [LifeCycleObserver.h](#)
- [LifeCycleObserver.cpp](#)

4.26 LowWaterStrategy Class Reference

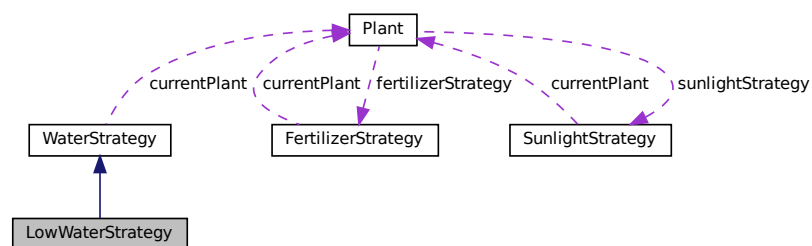
Strategy for plants requiring minimal water intake.

```
#include <LowWaterStrategy.h>
```

Inheritance diagram for LowWaterStrategy:



Collaboration diagram for LowWaterStrategy:



Public Member Functions

- void [applyWater](#) ([Plant](#) *p) override
Applies the low water strategy to a plant.
- std::string [getStrategyName](#) () const override
Retrieves the name of this watering strategy.

Additional Inherited Members

4.26.1 Detailed Description

Strategy for plants requiring minimal water intake.

Applies a 20ml water increase, capping the level at the plant's maximum water limit.

4.26.2 Member Function Documentation

4.26.2.1 applyWater()

```
void LowWaterStrategy::applyWater (
    Plant * p ) [override], [virtual]
```

Applies the low water strategy to a plant.

Parameters

<i>p</i>	The target plant.
----------	-------------------

Note

Increases water level by 20ml.

Warning

Caps the value at the plant's maximum water level.

Implements [WaterStrategy](#).

4.26.2.2 getStrategyName()

```
std::string LowWaterStrategy::getStrategyName ( ) const [override], [virtual]
```

Retrieves the name of this watering strategy.

Returns

A string representing the strategy name.

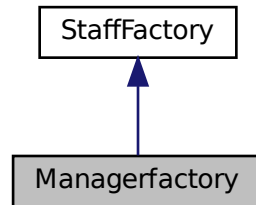
Implements [WaterStrategy](#).

The documentation for this class was generated from the following files:

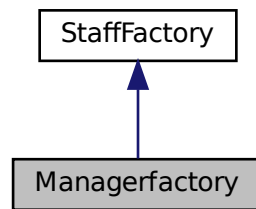
- [LowWaterStrategy.h](#)
- [LowWaterStrategy.cpp](#)

4.27 Managerfactory Class Reference

Inheritance diagram for Managerfactory:



Collaboration diagram for Managerfactory:



Public Member Functions

- [Managerfactory](#) ()
Default constructor.
- [~Managerfactory](#) ()
Virtual destructor.
- [FloorStaff](#) * [createFloorStaff](#) (std::string name) override
Create a [FloorManager](#) instance.
- [SalesStaff](#) * [createSalesStaff](#) (std::string name) override
Create a [SalesManager](#) instance.

4.27.1 Member Function Documentation

4.27.1.1 createFloorStaff()

```
FloorStaff * Managerfactory::createFloorStaff (
    std::string name ) [override], [virtual]
```

Create a [FloorManager](#) instance.

Parameters

<i>name</i>	The name of the manager to create
-------------	-----------------------------------

Returns

Pointer to new [FloorManager](#) (caller owns memory)

Implements [StaffFactory](#).

4.27.1.2 createSalesStaff()

```
SalesStaff * Managerfactory::createSalesStaff (
    std::string name ) [override], [virtual]
```

Create a [SalesManager](#) instance.

Parameters

<i>name</i>	The name of the manager to create
-------------	-----------------------------------

Returns

Pointer to new [SalesManager](#) (caller owns memory)

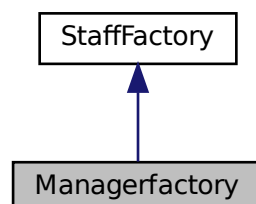
Implements [StaffFactory](#).

The documentation for this class was generated from the following files:

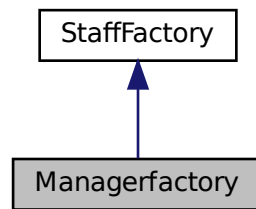
- [ManagerFactory.h](#)
- ManagerFactory.cpp

4.28 Managerfactory Class Reference

Inheritance diagram for Managerfactory:



Collaboration diagram for Managerfactory:



Public Member Functions

- [Managerfactory](#) ()
Default constructor.
- [~Managerfactory](#) ()
Virtual destructor.
- [FloorStaff](#) * [createFloorStaff](#) (std::string name) override
Create a [FloorManager](#) instance.
- [SalesStaff](#) * [createSalesStaff](#) (std::string name) override
Create a [SalesManager](#) instance.

4.28.1 Member Function Documentation

4.28.1.1 createFloorStaff()

```
FloorStaff * Managerfactory::createFloorStaff (
    std::string name ) [override], [virtual]
```

Create a [FloorManager](#) instance.

Parameters

<i>name</i>	The name of the manager to create
-------------	-----------------------------------

Returns

Pointer to new [FloorManager](#) (caller owns memory)

Implements [StaffFactory](#).

4.28.1.2 createSalesStaff()

```
SalesStaff * Managerfactory::createSalesStaff (
    std::string name ) [override], [virtual]
```

Create a [SalesManager](#) instance.

Parameters

<i>name</i>	The name of the manager to create
-------------	-----------------------------------

Returns

Pointer to new [SalesManager](#) (caller owns memory)

Implements [StaffFactory](#).

The documentation for this class was generated from the following files:

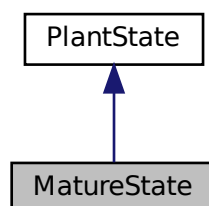
- [ManagerFactory.h](#)
- ManagerFactory.cpp

4.29 MatureState Class Reference

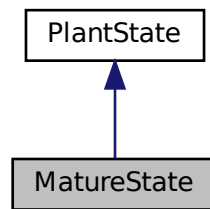
Represents a thriving stage in the plant's life cycle.

```
#include <MatureState.h>
```

Inheritance diagram for MatureState:



Collaboration diagram for MatureState:



Public Member Functions

- void `applyCare` (`PlantLifecycle` *context, `Plant` *plant, `PlantCareRoutine` *routine) override
Applies care to the plant in this state.
- bool `evaluate` (`PlantLifecycle` *context, `Plant` *plant) override
Evaluates the plant's condition and transitions to another state if necessary.
- std::string `getName` () const override
Gets the name of this state.
- `PlantState` * `clone` () const override

4.29.1 Detailed Description

Represents a thriving stage in the plant's life cycle.

4.29.2 Member Function Documentation

4.29.2.1 `applyCare()`

```

void MatureState::applyCare (
    PlantLifecycle * context,
    Plant * plant,
    PlantCareRoutine * routine ) [override], [virtual]
  
```

Applies care to the plant in this state.

Parameters

<i>context</i>	The plant's life cycle context.
<i>plant</i>	The plant object being cared for.
<i>routine</i>	The care routine to apply.

Implements [PlantState](#).

4.29.2.2 evaluate()

```
bool MatureState::evaluate (
    PlantLifeCycle * context,
    Plant * plant ) [override], [virtual]
```

Evaluates the plant's condition and transitions to another state if necessary.

Parameters

<i>context</i>	The plant's life cycle context.
<i>plant</i>	The plant object being evaluated.

Returns

True if the plant remains in this state, false otherwise.

Implements [PlantState](#).

4.29.2.3 getName()

```
std::string MatureState::getName ( ) const [override], [virtual]
```

Gets the name of this state.

Returns

The name of this state.

Implements [PlantState](#).

The documentation for this class was generated from the following files:

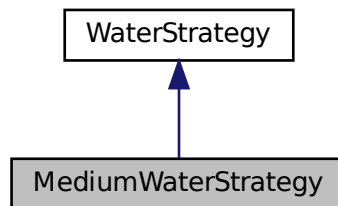
- [MatureState.h](#)
- [MatureState.cpp](#)

4.30 MediumWaterStrategy Class Reference

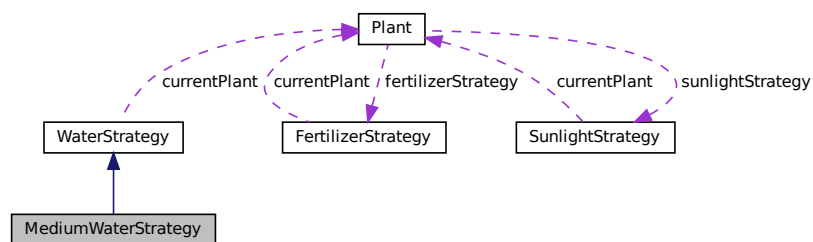
Strategy for moderately watering plants.

```
#include <MediumWaterStrategy.h>
```

Inheritance diagram for MediumWaterStrategy:



Collaboration diagram for MediumWaterStrategy:



Public Member Functions

- void `applyWater` (`Plant` *p) override
Applies the medium water strategy to the plant.
- std::string `getStrategyName` () const override
Gets the name of this watering strategy.

Additional Inherited Members

4.30.1 Detailed Description

Strategy for moderately watering plants.

Applies a 50ml water increase and prevents overflow beyond maximum capacity.

4.30.2 Member Function Documentation

4.30.2.1 applyWater()

```
void MediumWaterStrategy::applyWater (
    Plant * p ) [override], [virtual]
```

Applies the medium water strategy to the plant.

Parameters

<i>p</i>	The plant to water.
----------	---------------------

Note

Increases the plant's water level by 50ml.

Warning

Caps the water level at the maximum allowed value.

Implements [WaterStrategy](#).

4.30.2.2 getStrategyName()

```
std::string MediumWaterStrategy::getStrategyName ( ) const [override], [virtual]
```

Gets the name of this watering strategy.

Returns

A string with the strategy's name.

Implements [WaterStrategy](#).

The documentation for this class was generated from the following files:

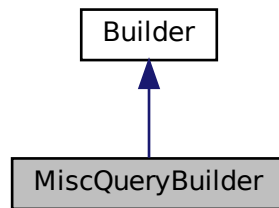
- [MediumWaterStrategy.h](#)
- [MediumWaterStrategy.cpp](#)

4.31 MiscQueryBuilder Class Reference

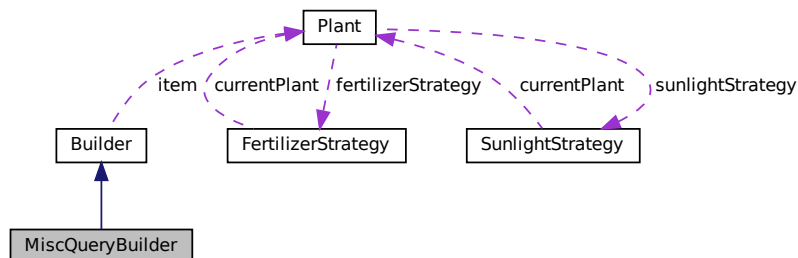
Concrete builder for non-item queries.

```
#include <MiscQueryBuilder.h>
```

Inheritance diagram for MiscQueryBuilder:



Collaboration diagram for MiscQueryBuilder:



Public Member Functions

- void `setType` (std::string `type`) override
Set the query type for the miscellaneous query.
- void `setQuestion` (std::string `question`) override
Set the explicit question text.
- void `setItem` (Plant *`item`) override
setItem has no effect for `MiscQueryBuilder`.
- `MiscQueryBuilder` ()
Constructs a new `MiscQueryBuilder`.

Additional Inherited Members

4.31.1 Detailed Description

Concrete builder for non-item queries.

See also

[Builder](#)

Implements [Builder](#) but ignores [setItem\(\)](#) since it targets no specific [Plant](#).

4.31.2 Member Function Documentation

4.31.2.1 [setItem\(\)](#)

```
void MiscQueryBuilder::setItem (
    Plant * item ) [override], [virtual]
```

[setItem](#) has no effect for [MiscQueryBuilder](#).

[setItem](#) is not supported for miscellaneous queries.

Parameters

<i>item</i>	Ignored.
<i>item</i>	Ignored.

This implementation intentionally does not store the provided item. A diagnostic message is printed to stderr to aid debugging.

Implements [Builder](#).

4.31.2.2 [setQuestion\(\)](#)

```
void MiscQueryBuilder::setQuestion (
    std::string question ) [override], [virtual]
```

Set the explicit question text.

Set the explicit question text for the misc query.

Parameters

<i>question</i>	Question string.
-----------------	------------------

Implements [Builder](#).

4.31.2.3 setType()

```
void MiscQueryBuilder::setType (
    std::string type ) [override], [virtual]
```

Set the query type for the miscellaneous query.

Set the type/category for the misc query.

Parameters

<i>type</i>	Type string (e.g. "STOCK").
-------------	-----------------------------

Implements [Builder](#).

The documentation for this class was generated from the following files:

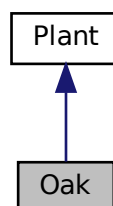
- [MiscQueryBuilder.h](#)
- [MiscQueryBuilder.cpp](#)

4.32 Oak Class Reference

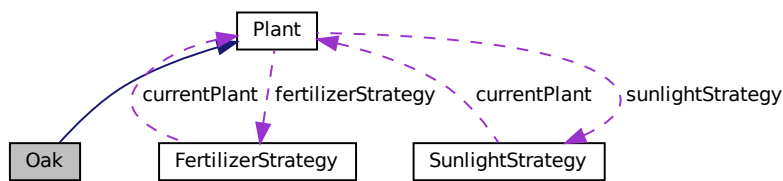
Represents an [Oak](#) tree plant.

```
#include <Oak.h>
```

Inheritance diagram for Oak:



Collaboration diagram for Oak:



Public Member Functions

- [Oak](#) (double price, std::string description)
Constructs an [Oak](#) plant object.

Additional Inherited Members

4.32.1 Detailed Description

Represents an [Oak](#) tree plant.

This class is a concrete implementation of the [Plant](#) class, representing an [Oak](#) tree in the plant inventory system.

4.32.2 Constructor & Destructor Documentation

4.32.2.1 Oak()

```
Oak::Oak (
    double price,
    std::string description ) [inline]
```

Constructs an [Oak](#) plant object.

Parameters

<i>price</i>	The price of the oak tree
<i>description</i>	A description of the oak tree

The documentation for this class was generated from the following file:

- [Oak.h](#)

4.33 Order Class Reference

Represents a customer's purchase order.

```
#include <Order.h>
```

Public Member Functions

- [Order](#) ([Customer](#) *customer, const std::string &orderId)
Constructs a new [Order](#) instance.
- [~Order](#) ()
Destructor to clean up order resources.
- void [addTransaction](#) ([Transaction](#) *transaction)
Adds a transaction to this order.
- void [processOrder](#) ()
Processes all transactions associated with this order.
- void [displayOrderDetails](#) () const
Displays detailed information about this order.
- std::string [getOrderId](#) () const
Retrieves the order ID.
- std::string [getStatus](#) () const
Retrieves the current order status.
- [Customer](#) * [getCustomer](#) () const
Gets the customer associated with this order.

4.33.1 Detailed Description

Represents a customer's purchase order.

An [Order](#) aggregates multiple transactions and manages their processing, status updates, and display details.

4.33.2 Constructor & Destructor Documentation

4.33.2.1 Order()

```
Order::Order (  
    Customer * customer,  
    const std::string & orderId )
```

Constructs a new [Order](#) instance.

Parameters

<i>customer</i>	Pointer to the customer placing the order.
<i>orderId</i>	Unique order identifier.

4.33.3 Member Function Documentation

4.33.3.1 addTransaction()

```
void Order::addTransaction (
    Transaction * transaction )
```

Adds a transaction to this order.

Parameters

<i>transaction</i>	Pointer to the transaction to add.
--------------------	------------------------------------

4.33.3.2 getCustomer()

```
Customer * Order::getCustomer ( ) const
```

Gets the customer associated with this order.

Returns

Pointer to the [Customer](#).

4.33.3.3 getOrderId()

```
std::string Order::getOrderId ( ) const
```

Retrieves the order ID.

Returns

String representing the order ID.

4.33.3.4 getStatus()

```
std::string Order::getStatus ( ) const
```

Retrieves the current order status.

Returns

String representing the order status.

The documentation for this class was generated from the following files:

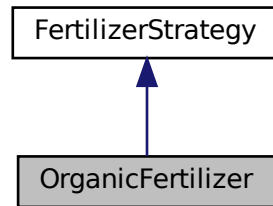
- [Order.h](#)
- [Order.cpp](#)

4.34 OrganicFertilizer Class Reference

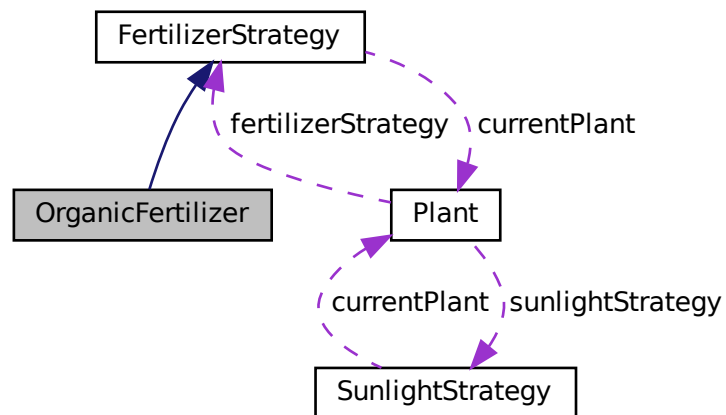
Strategy for applying organic fertilizer to a plant.

```
#include <OrganicFertilizer.h>
```

Inheritance diagram for OrganicFertilizer:



Collaboration diagram for OrganicFertilizer:



Public Member Functions

- void [applyFertilizer](#) ([Plant](#) *p) override
Applies the organic fertilizer strategy to a plant.
- std::string [getStrategyName](#) () const override
Get the name of this strategy.

Additional Inherited Members

4.34.1 Detailed Description

Strategy for applying organic fertilizer to a plant.

4.34.2 Member Function Documentation

4.34.2.1 `applyFertilizer()`

```
void OrganicFertilizer::applyFertilizer (
    Plant * p ) [override], [virtual]
```

Applies the organic fertilizer strategy to a plant.

Increases nutrient levels by 80 ppm, capped at the plant's maximum limit. This promotes growth using natural compounds.

Parameters

<i>p</i>	Pointer to the plant receiving the fertilizer.
----------	--

Implements [FertilizerStrategy](#).

4.34.2.2 `getStrategyName()`

```
std::string OrganicFertilizer::getStrategyName ( ) const [override], [virtual]
```

Get the name of this strategy.

Returns

String identifying the strategy type.

Implements [FertilizerStrategy](#).

The documentation for this class was generated from the following files:

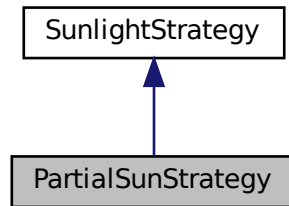
- [OrganicFertilizer.h](#)
- [OrganicFertilizer.cpp](#)

4.35 PartialSunStrategy Class Reference

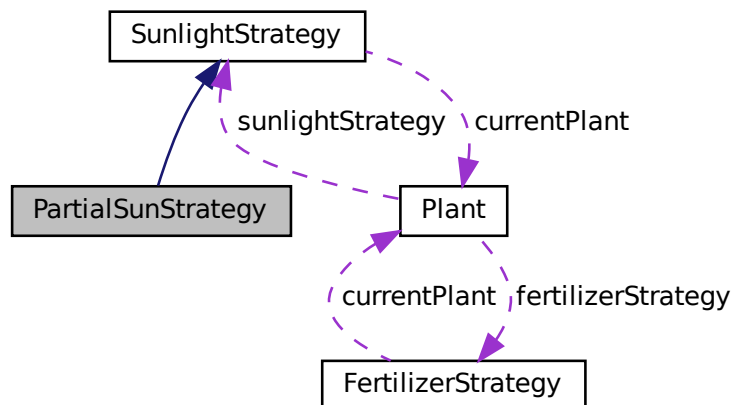
Strategy for applying partial sunlight to a plant.

```
#include <PartialSunStrategy.h>
```

Inheritance diagram for PartialSunStrategy:



Collaboration diagram for PartialSunStrategy:



Public Member Functions

- void `applySunlight (Plant *p)` override
Applies the partial sun strategy to a plant.
- std::string `getStrategyName ()` const override
Retrieves the strategy name.

Additional Inherited Members

4.35.1 Detailed Description

Strategy for applying partial sunlight to a plant.

4.35.2 Member Function Documentation

4.35.2.1 `applySunlight()`

```
void PartialSunStrategy::applySunlight (
    Plant * p ) [override], [virtual]
```

Applies the partial sun strategy to a plant.

Increases sunlight by 40% of the plant's maximum capacity. Caps at the maximum sunlight threshold.

Parameters

<i>p</i>	Pointer to the plant receiving sunlight.
----------	--

Implements [SunlightStrategy](#).

4.35.2.2 `getStrategyName()`

```
std::string PartialSunStrategy::getStrategyName ( ) const [override], [virtual]
```

Retrieves the strategy name.

Returns

String representing the strategy type.

Implements [SunlightStrategy](#).

The documentation for this class was generated from the following files:

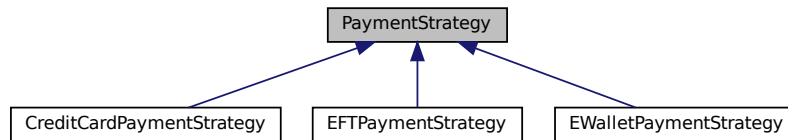
- [PartialSunStrategy.h](#)
- [PartialSunStrategy.cpp](#)

4.36 PaymentStrategy Class Reference

Abstract strategy for processing payments.

```
#include <PaymentStrategy.h>
```

Inheritance diagram for PaymentStrategy:



Public Member Functions

- virtual [~PaymentStrategy](#) ()
Virtual destructor.
- virtual void [pay](#) (double amount, const std::string &orderNum) const =0
Executes a payment operation.
- virtual std::string [getName](#) () const =0
Retrieves the payment strategy name.

4.36.1 Detailed Description

Abstract strategy for processing payments.

Provides a common interface for different payment types (EFT, E-Wallet, etc.).

4.36.2 Member Function Documentation

4.36.2.1 getName()

```
virtual std::string PaymentStrategy::getName ( ) const [pure virtual]
```

Retrieves the payment strategy name.

Returns

String representing the payment method type.

Implemented in [EWalletPaymentStrategy](#), [EFTPaymentStrategy](#), and [CreditCardPaymentStrategy](#).

4.36.2.2 pay()

```
virtual void PaymentStrategy::pay (
    double amount,
    const std::string & orderNum ) const [pure virtual]
```

Executes a payment operation.

Parameters

<i>amount</i>	The payment amount.
<i>orderNum</i>	The associated order identifier.

Implemented in [EWalletPaymentStrategy](#), [EFTPaymentStrategy](#), and [CreditCardPaymentStrategy](#).

The documentation for this class was generated from the following file:

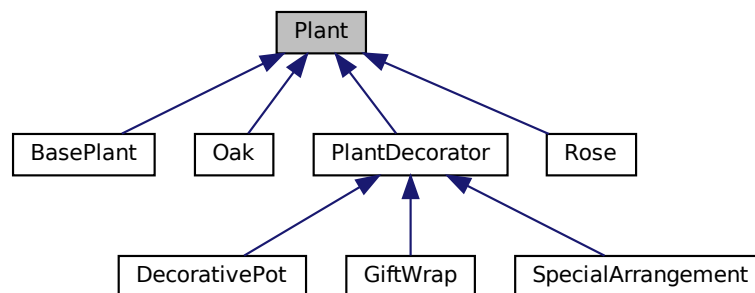
- [PaymentStrategy.h](#)

4.37 Plant Class Reference

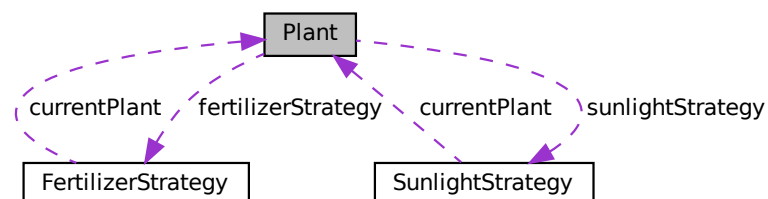
Abstract representation of a plant with care and lifecycle management.

```
#include <Plant.h>
```

Inheritance diagram for Plant:



Collaboration diagram for Plant:



Public Member Functions

- [Plant](#) ()
Default constructor.
- [Plant](#) (double price, std::string description)
Parameterized constructor.
- void [applyCare](#) ()
Applies care routines to the plant.
- void [setState](#) (std::unique_ptr< [PlantState](#) > s)
Sets the state of the plant.
- [PlantState](#) * [getState](#) ()
Gets the current state of the plant.
- void [attach](#) ([LifecycleObserver](#) *o)
Attaches a lifecycle observer to the plant.
- void [detach](#) ([LifecycleObserver](#) *o)
Detaches a lifecycle observer from the plant.
- void [notify](#) ()
Notifies all attached observers of changes.
- virtual std::string [getDescription](#) () const
Gets the plant description.
- virtual double [getPrice](#) () const
Gets the plant price.
- bool [setColor](#) (const std::string &color)
Sets the color of the plant.
- std::string [getColor](#) () const
Gets the plant color.
- std::string [getName](#) ()
Gets the plant name.
- int [getMaxWater](#) ()
Gets the maximum water capacity.
- void [setMaxWater](#) (int mW)
Sets the maximum water capacity.
- int [getCurrentWater](#) ()
Gets the current water level.
- void [setCurrentWater](#) (int cW)
Sets the current water level.
- int [getCurrentNutrients](#) ()
Gets the current nutrient level.
- void [setCurrentNutrients](#) (int cN)
Sets the current nutrient level.
- int [getMaxNutrients](#) ()
Gets the maximum nutrient capacity.
- void [setMaxNutrients](#) (int mN)
Sets the maximum nutrient capacity.
- int [getCurrentSunlight](#) ()
Gets the current sunlight level.
- void [setCurrentSunlight](#) (int mL)
Sets the current sunlight level.
- int [getMaxSunlight](#) ()
Gets the maximum sunlight capacity.
- void [setMaxSunlight](#) (int mL)

- Sets the maximum sunlight capacity.*
 - int `getMinWater` () const
- Gets the minimum water requirement.*
 - int `getMinSunlight` () const
- Gets the minimum sunlight requirement.*
 - int `getMinNutrients` () const
- Gets the minimum nutrient requirement.*
 - std::string `getCategory` ()
- Gets the plant category.*
 - void `setCategory` (const std::string &c)
- Sets the plant category.*
 - int `getGrowthProgress` () const
- Gets the current growth progress.*
 - void `setGrowthProgress` (int gp)
- Sets the growth progress.*
 - void `increaseGrowthProgress` ()
- Increments the growth progress by 1.*

Protected Attributes

- WateringStrategy * **waterStrategy**
- SunlightStrategy * **sunlightStrategy**
- FertilizerStrategy * **fertilizerStrategy**
- std::vector< LifecycleObserver * > **observerList**
- std::string **category**
- std::string **description**
- double **price**
- std::string **color**

4.37.1 Detailed Description

Abstract representation of a plant with care and lifecycle management.

4.37.2 Constructor & Destructor Documentation

4.37.2.1 Plant()

```
Plant::Plant (
    double price,
    std::string description )
```

Parameterized constructor.

Parameters

<i>price</i>	The price of the plant
<i>description</i>	Description of the plant

4.37.3 Member Function Documentation

4.37.3.1 attach()

```
void Plant::attach (
    LifecycleObserver * o )
```

Attaches a lifecycle observer to the plant.

Parameters

<i>o</i>	Observer to attach
----------	--------------------

4.37.3.2 detach()

```
void Plant::detach (
    LifecycleObserver * o )
```

Detaches a lifecycle observer from the plant.

Parameters

<i>o</i>	Observer to detach
----------	--------------------

4.37.3.3 getCategory()

```
std::string Plant::getCategory ( )
```

Gets the plant category.

Returns

Category string

4.37.3.4 getColor()

```
std::string Plant::getColor ( ) const
```

Gets the plant color.

Returns

Color string

4.37.3.5 `getCurrentNutrients()`

```
int Plant::getCurrentNutrients ( )
```

Gets the current nutrient level.

Returns

Current nutrient value

4.37.3.6 `getCurrentSunlight()`

```
int Plant::getCurrentSunlight ( )
```

Gets the current sunlight level.

Returns

Current sunlight value

4.37.3.7 `getCurrentWater()`

```
int Plant::getCurrentWater ( )
```

Gets the current water level.

Returns

Current water value

4.37.3.8 `getDescription()`

```
std::string Plant::getDescription ( ) const [virtual]
```

Gets the plant description.

Returns

Description string

Reimplemented in [SpecialArrangement](#), [PlantDecorator](#), [GiftWrap](#), [DecorativePot](#), and [BasePlant](#).

4.37.3.9 getGrowthProgress()

```
int Plant::getGrowthProgress ( ) const
```

Gets the current growth progress.

Returns

Growth progress value

4.37.3.10 getMaxNutrients()

```
int Plant::getMaxNutrients ( )
```

Gets the maximum nutrient capacity.

Returns

Maximum nutrient value

4.37.3.11 getMaxSunlight()

```
int Plant::getMaxSunlight ( )
```

Gets the maximum sunlight capacity.

Returns

Maximum sunlight value

4.37.3.12 getMaxWater()

```
int Plant::getMaxWater ( )
```

Gets the maximum water capacity.

Returns

Maximum water value

4.37.3.13 getMinNutrients()

```
int Plant::getMinNutrients ( ) const
```

Gets the minimum nutrient requirement.

Returns

Minimum nutrient value

4.37.3.14 getMinSunlight()

```
int Plant::getMinSunlight ( ) const
```

Gets the minimum sunlight requirement.

Returns

Minimum sunlight value

4.37.3.15 getMinWater()

```
int Plant::getMinWater ( ) const
```

Gets the minimum water requirement.

Returns

Minimum water value

4.37.3.16 getName()

```
std::string Plant::getName ( )
```

Gets the plant name.

Returns

Name string

4.37.3.17 getPrice()

```
double Plant::getPrice ( ) const [virtual]
```

Gets the plant price.

Returns

Price value

Reimplemented in [SpecialArrangement](#), [PlantDecorator](#), [GiftWrap](#), [DecorativePot](#), and [BasePlant](#).

4.37.3.18 getState()

```
PlantState* Plant::getState ( ) [inline]
```

Gets the current state of the plant.

Returns

Pointer to the current [PlantState](#)

4.37.3.19 setCategory()

```
void Plant::setCategory (
    const std::string & c )
```

Sets the plant category.

Parameters

<i>c</i>	Category string
----------	-----------------

4.37.3.20 setColor()

```
bool Plant::setColor (
    const std::string & color )
```

Sets the color of the plant.

Parameters

<i>color</i>	Color to set
--------------	--------------

Returns

true if successful

4.37.3.21 setCurrentNutrients()

```
void Plant::setCurrentNutrients (
    int cN )
```

Sets the current nutrient level.

Parameters

<i>cN</i>	Current nutrient value (capped at max)
-----------	--

4.37.3.22 setCurrentSunlight()

```
void Plant::setCurrentSunlight (
    int mL )
```

Sets the current sunlight level.

Parameters

<i>mL</i>	Current sunlight value (capped at max)
-----------	--

4.37.3.23 setCurrentWater()

```
void Plant::setCurrentWater (
    int cW )
```

Sets the current water level.

Parameters

<i>cW</i>	Current water value (capped at max)
-----------	-------------------------------------

4.37.3.24 setGrowthProgress()

```
void Plant::setGrowthProgress (
```

```
int gp )
```

Sets the growth progress.

Parameters

<i>gp</i>	Growth progress value
-----------	-----------------------

4.37.3.25 setMaxNutrients()

```
void Plant::setMaxNutrients (
    int mN )
```

Sets the maximum nutrient capacity.

Parameters

<i>mN</i>	Maximum nutrient value
-----------	------------------------

4.37.3.26 setMaxSunlight()

```
void Plant::setMaxSunlight (
    int mL )
```

Sets the maximum sunlight capacity.

Parameters

<i>mL</i>	Maximum sunlight value
-----------	------------------------

4.37.3.27 setMaxWater()

```
void Plant::setMaxWater (
    int mW )
```

Sets the maximum water capacity.

Parameters

<i>mW</i>	Maximum water value
-----------	---------------------

4.37.3.28 setState()

```
void Plant::setState (
    std::unique_ptr< PlantState > s ) [inline]
```

Sets the state of the plant.

Parameters

s	New state to set
---	------------------

The documentation for this class was generated from the following files:

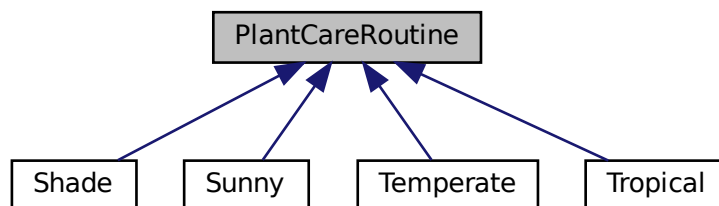
- [Plant.h](#)
- [Plant.cpp](#)

4.38 PlantCareRoutine Class Reference

Abstract base class defining a plant care routine interface.

```
#include <PlantCareRoutine.h>
```

Inheritance diagram for PlantCareRoutine:



Public Member Functions

- virtual void [Fertilizing](#) (Plant *p)=0
Apply fertilization routine to a plant.
- virtual void [Watering](#) (Plant *p)=0
Apply watering routine to a plant.
- virtual void [Sunlight](#) (Plant *p)=0
Apply sunlight exposure routine to a plant.
- virtual void [printCareRoutine](#) ()=0
Print the details of the care routine.

Static Public Member Functions

- static `std::unique_ptr< PlantCareRoutine > PlantCare (Plant *p)`
Factory method for creating care routines.

4.38.1 Detailed Description

Abstract base class defining a plant care routine interface.

4.38.2 Member Function Documentation

4.38.2.1 PlantCare()

```
std::unique_ptr< PlantCareRoutine > PlantCareRoutine::PlantCare (  
    Plant * p ) [static]
```

Factory method for creating care routines.

Parameters

<i>p</i>	Pointer to the plant needing care.
----------	------------------------------------

Returns

Unique pointer to a new [PlantCareRoutine](#) instance.

The documentation for this class was generated from the following files:

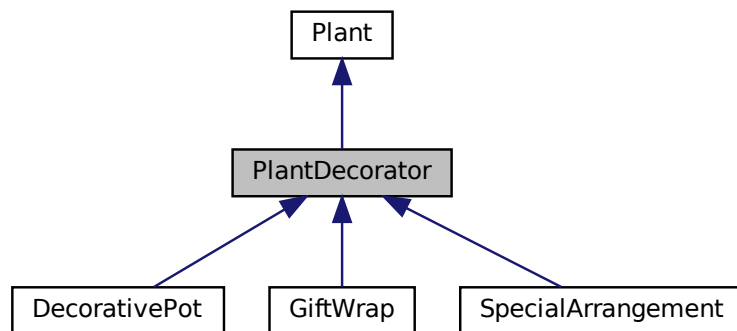
- [PlantCareRoutine.h](#)
- [PlantCareRoutine.cpp](#)

4.39 PlantDecorator Class Reference

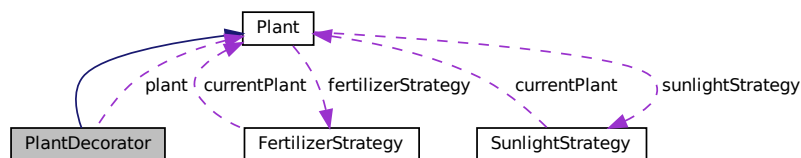
Abstract decorator for dynamically extending [Plant](#) behavior.

```
#include <PlantDecorator.h>
```

Inheritance diagram for PlantDecorator:



Collaboration diagram for PlantDecorator:



Public Member Functions

- [PlantDecorator \(Plant *plant\)](#)
Constructs a decorator wrapping the given plant.
- virtual std::string [getDescription \(\)](#) const override
Get the decorated plant's description.
- virtual double [getPrice \(\)](#) const override
Get the decorated plant's price.

Protected Attributes

- [Plant * plant](#)
Pointer to the wrapped [Plant](#) object.

4.39.1 Detailed Description

Abstract decorator for dynamically extending [Plant](#) behavior.

The documentation for this class was generated from the following files:

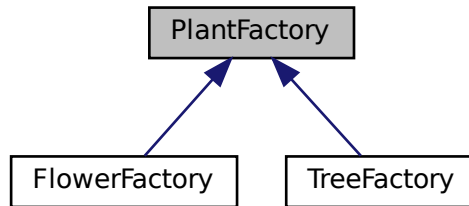
- [PlantDecorator.h](#)
- [PlantDecorator.cpp](#)

4.40 PlantFactory Class Reference

Abstract base class for plant creation factories.

```
#include <PlantFactory.h>
```

Inheritance diagram for PlantFactory:



Public Member Functions

- virtual [Plant](#) * [createPlant](#) (std::string plantType, double price)=0
Pure virtual method to create a plant.

4.40.1 Detailed Description

Abstract base class for plant creation factories.

4.40.2 Member Function Documentation

4.40.2.1 createPlant()

```
virtual Plant* PlantFactory::createPlant (  
    std::string plantType,  
    double price ) [pure virtual]
```

Pure virtual method to create a plant.

Parameters

<i>plantType</i>	The type of plant to create
<i>price</i>	The price of the plant

Returns

Pointer to the newly created [Plant](#) object

Implemented in [TreeFactory](#), and [FlowerFactory](#).

The documentation for this class was generated from the following file:

- [PlantFactory.h](#)

4.41 PlantInventory Class Reference

Singleton class managing plant inventory and observers.

```
#include <PlantInventory.h>
```

Public Member Functions

- bool [attach](#) ([InventoryObserver](#) *observer)
Attaches an observer to the inventory.
- bool [detach](#) ([InventoryObserver](#) *observer)
Detaches an observer from the inventory.
- void [notify](#) ()
Notifies all observers if changes are made.
- void [updateStock](#) ()
Updates stock and notifies observers.
- bool [addStock](#) ([Plant](#) *plant, int quantity)
Adds stock to inventory (creates and executes [AddStock](#) command)
- bool [removeStock](#) ([Plant](#) *plant, int quantity)
Removes stock from inventory (creates and executes [RemoveStock](#) command)
- bool [executeCommand](#) ([StockCommand](#) *command)
Executes a stock command and removes it from memory.
- std::vector< std::pair< [Plant](#) *, int > > & [getInventoryReference](#) ()
Gets a reference to inventory items for command operations.
- const std::vector< std::pair< [Plant](#) *, int > > [getInventoryView](#) () const
Gets a copy (not reference) of inventory.
- int [findPlantIndex](#) ([Plant](#) *plant) const
Finds inventory index for a given [Plant](#).
- int [getQuantity](#) ([Plant](#) *plant) const
Gets the quantity of a specific plant in inventory.

Static Public Member Functions

- static [PlantInventory](#) * [getInstance](#) ()
Gets the singleton instance of [PlantInventory](#).

4.41.1 Detailed Description

Singleton class managing plant inventory and observers.

4.41.2 Member Function Documentation

4.41.2.1 addStock()

```
bool PlantInventory::addStock (
    Plant * plant,
    int quantity )
```

Adds stock to inventory (creates and executes [AddStock](#) command)

Parameters

<i>plant</i>	Plant to add
<i>quantity</i>	Quantity to add

Returns

true if successful

4.41.2.2 attach()

```
bool PlantInventory::attach (
    InventoryObserver * observer )
```

Attaches an observer to the inventory.

Parameters

<i>observer</i>	Observer to attach
-----------------	--------------------

Returns

true if successful

4.41.2.3 detach()

```
bool PlantInventory::detach (
    InventoryObserver * observer )
```

Detaches an observer from the inventory.

Parameters

<i>observer</i>	Observer to detach
-----------------	--------------------

Returns

true if successful

4.41.2.4 executeCommand()

```
bool PlantInventory::executeCommand (
    StockCommand * command )
```

Executes a stock command and removes it from memory.

Parameters

<i>command</i>	Command to execute
----------------	--------------------

Returns

true if execution successful

4.41.2.5 findPlantIndex()

```
int PlantInventory::findPlantIndex (
    Plant * plant ) const
```

Finds inventory index for a given [Plant](#).

Parameters

<i>plant</i>	Plant to find
--------------	-------------------------------

Returns

Index of plant in inventory, or -1 if not found

4.41.2.6 getInstance()

```
PlantInventory * PlantInventory::getInstance ( ) [static]
```

Gets the singleton instance of [PlantInventory](#).

Returns

Pointer to the singleton instance

4.41.2.7 getInventoryReference()

```
std::vector< std::pair< Plant *, int > > & PlantInventory::getInventoryReference ( )
```

Gets a reference to inventory items for command operations.

Returns

Reference to inventory vector

4.41.2.8 getInventoryView()

```
const std::vector< std::pair< Plant *, int > > PlantInventory::getInventoryView ( ) const
```

Gets a copy (not reference) of inventory.

Returns

Copy of inventory vector

4.41.2.9 getQuantity()

```
int PlantInventory::getQuantity (
    Plant * plant ) const
```

Gets the quantity of a specific plant in inventory.

Parameters

<i>plant</i>	Plant to query
--------------	--------------------------------

Returns

Quantity of the plant, or 0 if not found

4.41.2.10 removeStock()

```
bool PlantInventory::removeStock (
    Plant * plant,
    int quantity )
```

Removes stock from inventory (creates and executes [RemoveStock](#) command)

Parameters

<i>plant</i>	Plant to remove
<i>quantity</i>	Quantity to remove

Returns

true if successful

The documentation for this class was generated from the following files:

- [PlantInventory.h](#)
- [PlantInventory.cpp](#)

4.42 PlantLifeCycle Class Reference

Manages the state and observers for a plant's life cycle.

```
#include <PLantLifeCycle.h>
```

Public Member Functions

- **PlantLifeCycle** ([Plant](#) *plant, std::unique_ptr< [PlantState](#) > initialState, std::string name)
- std::string **getState** ()
- [PlantState](#) * **getStateObj** () const
- void **setState** (std::unique_ptr< [PlantState](#) > state)
- std::unique_ptr< [PlantState](#) > **releaseState** ()
- void **attach** ([LifeCycleObserver](#) *ol)
- void **detach** ([LifeCycleObserver](#) *ol)
- void **notify** ()
- void **simulateTimePassing** ()
- bool **updatePlant** ()
- [Plant](#) * **getPlant** ()
- std::string **getName** ()
- std::vector< [LifeCycleObserver](#) * >::iterator **observersBegin** ()
- std::vector< [LifeCycleObserver](#) * >::iterator **observersEnd** ()

4.42.1 Detailed Description

Manages the state and observers for a plant's life cycle.

The documentation for this class was generated from the following files:

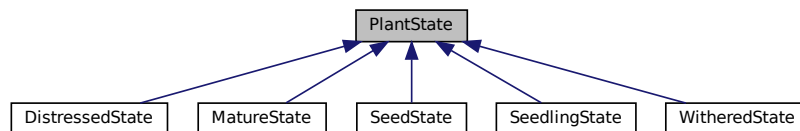
- PlantLifeCycle.h
- PlantLifeCycle.cpp

4.43 PlantState Class Reference

Abstract base class representing a state in a plant's life cycle.

```
#include <PlantState.h>
```

Inheritance diagram for PlantState:



Public Member Functions

- virtual void `applyCare` (`PlantLifeCycle` *context, `Plant` *plant, `PlantCareRoutine` *routine)=0
Applies care to the plant in this state.
- virtual bool `evaluate` (`PlantLifeCycle` *context, `Plant` *plant)=0
Evaluates the plant's condition and transitions to another state if necessary.
- virtual std::string `getName` () const =0
Gets the name of this state.
- virtual `PlantState` * `clone` () const =0

4.43.1 Detailed Description

Abstract base class representing a state in a plant's life cycle.

4.43.2 Member Function Documentation

4.43.2.1 `applyCare()`

```
virtual void PlantState::applyCare (
    PlantLifeCycle * context,
    Plant * plant,
    PlantCareRoutine * routine ) [pure virtual]
```

Applies care to the plant in this state.

Parameters

<i>context</i>	The plant's life cycle context.
<i>plant</i>	The plant object being cared for.
<i>routine</i>	The care routine to apply.

Implemented in [WitheredState](#), [SeedState](#), [SeedlingState](#), [MatureState](#), and [DistressedState](#).

4.43.2.2 evaluate()

```
virtual bool PlantState::evaluate (
    PlantLifeCycle * context,
    Plant * plant ) [pure virtual]
```

Evaluates the plant's condition and transitions to another state if necessary.

Parameters

<i>context</i>	The plant's life cycle context.
<i>plant</i>	The plant object being evaluated.

Returns

True if the plant remains in this state, false otherwise.

Implemented in [WitheredState](#), [SeedState](#), [SeedlingState](#), [MatureState](#), and [DistressedState](#).

4.43.2.3 getName()

```
virtual std::string PlantState::getName ( ) const [pure virtual]
```

Gets the name of this state.

Returns

The name of this state.

Implemented in [WitheredState](#), [SeedState](#), [SeedlingState](#), [MatureState](#), and [DistressedState](#).

The documentation for this class was generated from the following file:

- [PlantState.h](#)

4.44 PlantStock Class Reference

Acts as the Invoker in the Command pattern.

```
#include <PlantStock.h>
```

Public Member Functions

- [PlantStock](#) ([PlantInventory](#) *inventory)
Constructs a [PlantStock](#) invoker.
- bool [addStock](#) ([Plant](#) *plant, int quantity)
Adds stock to inventory via command.
- bool [removeStock](#) ([Plant](#) *plant, int quantity)
Removes stock from inventory via command.

4.44.1 Detailed Description

Acts as the Invoker in the Command pattern.

This class creates and executes stock commands for adding and removing plants from inventory.

4.44.2 Constructor & Destructor Documentation

4.44.2.1 PlantStock()

```
PlantStock::PlantStock (  
    PlantInventory * inventory )
```

Constructs a [PlantStock](#) invoker.

Parameters

<i>inventory</i>	Pointer to the inventory to manage
------------------	------------------------------------

4.44.3 Member Function Documentation

4.44.3.1 addStock()

```
bool PlantStock::addStock (  
    Plant * plant,  
    int quantity )
```

Adds stock to inventory via command.

Parameters

<i>plant</i>	Plant to add
<i>quantity</i>	Quantity to add

Returns

true if successful

4.44.3.2 removeStock()

```
bool PlantStock::removeStock (
    Plant * plant,
    int quantity )
```

Removes stock from inventory via command.

Parameters

<i>plant</i>	Plant to remove
<i>quantity</i>	Quantity to remove

Returns

true if successful

The documentation for this class was generated from the following files:

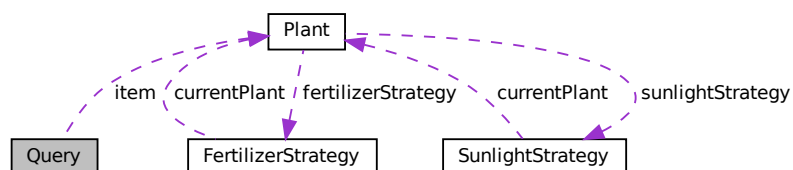
- [PlantStock.h](#)
- PlantStock.cpp

4.45 Query Class Reference

Encapsulates a query object constructed by a [Builder](#).

```
#include <Query.h>
```

Collaboration diagram for Query:



Public Member Functions

- [Query](#) ([Builder](#) *builder)
Constructs a [Query](#) object from a [Builder](#).
- std::string [getType](#) ()
Gets the type of query.
- std::string [getQuestion](#) ()
Gets the question text.
- [Plant](#) * [getItem](#) ()
Gets the plant item associated with the query.
- void [printQuery](#) ()
Prints the query information to console.

Protected Attributes

- [Plant](#) * [item](#)
The plant associated with this query (optional)
- std::string [type](#)
Type of query (e.g., "STOCK", "CARE")
- std::string [question](#)
The query text or question detail.

4.45.1 Detailed Description

Encapsulates a query object constructed by a [Builder](#).

4.45.2 Constructor & Destructor Documentation

4.45.2.1 Query()

```
Query::Query (
    Builder * builder )
```

Constructs a [Query](#) object from a [Builder](#).

Parameters

<i>builder</i>	Pointer to a Builder used to initialize the Query .
----------------	---

4.45.3 Member Function Documentation

4.45.3.1 getItem()

```
Plant * Query::getItem ( )
```

Gets the plant item associated with the query.

Returns

Pointer to the plant item.

4.45.3.2 getQuestion()

```
std::string Query::getQuestion ( )
```

Gets the question text.

Returns

The query question.

4.45.3.3 getType()

```
std::string Query::getType ( )
```

Gets the type of query.

Returns

The query type as a string.

The documentation for this class was generated from the following files:

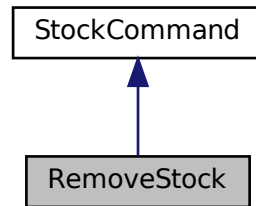
- [Query.h](#)
- [Query.cpp](#)

4.46 RemoveStock Class Reference

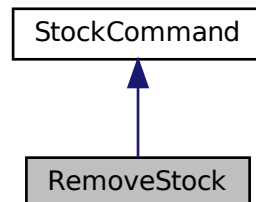
Concrete command that removes plant stock from the inventory.

```
#include <RemoveStock.h>
```

Inheritance diagram for RemoveStock:



Collaboration diagram for RemoveStock:



Public Member Functions

- `RemoveStock` (`Plant` *plant, int quantityToRemove, `PlantInventory` *targetInventory)
Constructs a `RemoveStock` command.
- `bool execute ()` override
Executes the remove stock command.

4.46.1 Detailed Description

Concrete command that removes plant stock from the inventory.

4.46.2 Constructor & Destructor Documentation

4.46.2.1 RemoveStock()

```
RemoveStock::RemoveStock (
    Plant * plant,
    int quantityToRemove,
    PlantInventory * targetInventory )
```

Constructs a [RemoveStock](#) command.

Parameters

<i>plant</i>	Plant to remove
<i>quantityToRemove</i>	Quantity to remove
<i>targetInventory</i>	Inventory to remove from

4.46.3 Member Function Documentation

4.46.3.1 execute()

```
bool RemoveStock::execute ( ) [override], [virtual]
```

Executes the remove stock command.

Returns

true if successful, false otherwise

Implements [StockCommand](#).

The documentation for this class was generated from the following files:

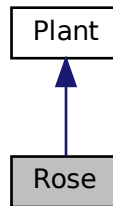
- [RemoveStock.h](#)
- [RemoveStock.cpp](#)

4.47 Rose Class Reference

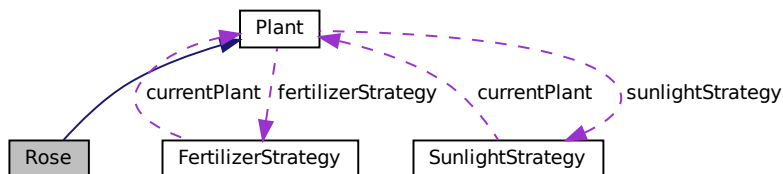
Concrete implementation of the [Plant](#) class for roses.

```
#include <Rose.h>
```

Inheritance diagram for Rose:



Collaboration diagram for Rose:



Public Member Functions

- [Rose](#) (double [price](#), std::string [description](#))
Constructs a [Rose](#) plant.

Protected Attributes

- double [price](#)
Price of the rose.
- std::string [description](#)
Description of the rose.

4.47.1 Detailed Description

Concrete implementation of the [Plant](#) class for roses.

4.47.2 Constructor & Destructor Documentation

4.47.2.1 Rose()

```
Rose::Rose (
    double price,
    std::string description ) [inline]
```

Constructs a [Rose](#) plant.

Parameters

<i>price</i>	The price of the rose
<i>description</i>	A description of the rose

The documentation for this class was generated from the following file:

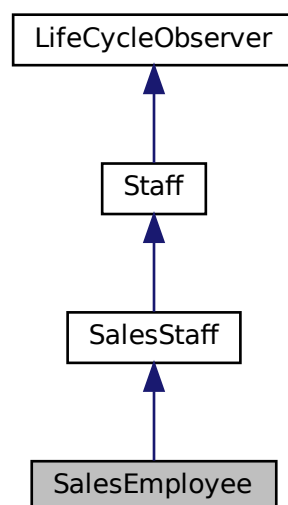
- [Rose.h](#)

4.48 SalesEmployee Class Reference

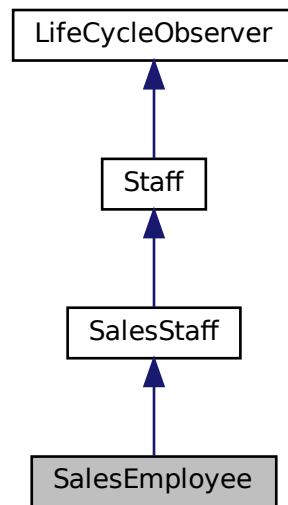
Concrete class representing a regular sales employee.

```
#include <SalesEmployee.h>
```

Inheritance diagram for SalesEmployee:



Collaboration diagram for SalesEmployee:



Public Member Functions

- [SalesEmployee](#) (std::string name)
Construct a sales employee with given name.
- virtual [~SalesEmployee](#) ()
Virtual destructor.
- bool [staffCanHandle](#) (std::string type) override
Check if this staff member can handle a query type.
- void [reply](#) ([Query](#) *query) override
Generate a reply for a query.

4.48.1 Detailed Description

Concrete class representing a regular sales employee.

4.48.2 Constructor & Destructor Documentation

4.48.2.1 SalesEmployee()

```
SalesEmployee::SalesEmployee (  
    std::string name )
```

Construct a sales employee with given name.

Parameters

<i>name</i>	The employee's name
-------------	---------------------

4.48.3 Member Function Documentation

4.48.3.1 reply()

```
void SalesEmployee::reply (  
    Query * query ) [override], [virtual]
```

Generate a reply for a query.

Parameters

<i>query</i>	The query to reply to (non-owning pointer)
--------------	--

Implements [SalesStaff](#).

4.48.3.2 staffCanHandle()

```
bool SalesEmployee::staffCanHandle (  
    std::string type ) [override], [virtual]
```

Check if this staff member can handle a query type.

Parameters

<i>type</i>	The query type to check
-------------	-------------------------

Returns

true if the staff member can handle it, false otherwise

Implements [SalesStaff](#).

The documentation for this class was generated from the following files:

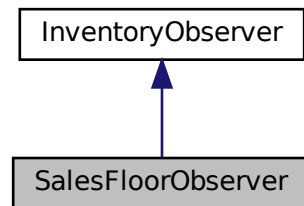
- [SalesEmployee.h](#)
- [SalesEmployee.cpp](#)

4.49 SalesFloorObserver Class Reference

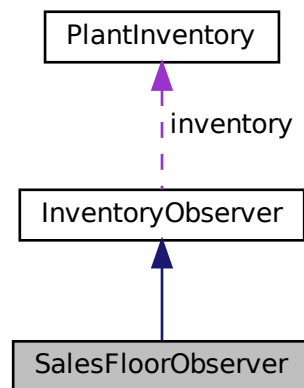
Observes the [PlantInventory](#) and updates the sales floor display.

```
#include <SalesFloorObserver.h>
```

Inheritance diagram for SalesFloorObserver:



Collaboration diagram for SalesFloorObserver:



Public Member Functions

- [SalesFloorObserver](#) ([PlantInventory](#) *inventory)
Constructs a sales floor observer.
- void [update](#) () override
Updates sales floor when inventory changes.
- void [displayAvailablePlants](#) () const
Displays all available plants on sales floor.
- bool [isPlantAvailable](#) ([Plant](#) *plant) const

- Checks if a plant is available.*
- `const std::vector< std::pair< Plant *, int > > & getAvailableItems ()` const
Gets available items on sales floor.
- `int getPlantQuantity (Plant *plant)` const
Gets quantity of a specific plant.
- `void attachStaff (Staff *staff)`
Attaches a staff member for notifications.
- `void detachStaff (Staff *staff)`
Detaches a staff member from notifications.
- `void notifyStaff (const std::string &message)`
Notifies all staff members with a message.
- `const std::vector< std::string > & getNotificationHistory ()` const
Gets all past notifications.

Additional Inherited Members

4.49.1 Detailed Description

Observes the [PlantInventory](#) and updates the sales floor display.

4.49.2 Constructor & Destructor Documentation

4.49.2.1 SalesFloorObserver()

```
SalesFloorObserver::SalesFloorObserver (
    PlantInventory * inventory )
```

Constructs a sales floor observer.

Parameters

<i>inventory</i>	Inventory to observe
------------------	----------------------

4.49.3 Member Function Documentation

4.49.3.1 attachStaff()

```
void SalesFloorObserver::attachStaff (
    Staff * staff )
```

Attaches a staff member for notifications.

Parameters

<i>staff</i>	Staff member to attach
--------------	--

4.49.3.2 detachStaff()

```
void SalesFloorObserver::detachStaff (
    Staff * staff )
```

Detaches a staff member from notifications.

Parameters

<i>staff</i>	Staff member to detach
--------------	--

4.49.3.3 getAvailableItems()

```
const std::vector<std::pair<Plant*, int> >& SalesFloorObserver::getAvailableItems ( ) const
[inline]
```

Gets available items on sales floor.

Returns

Const reference to available items

4.49.3.4 getNotificationHistory()

```
const std::vector<std::string>& SalesFloorObserver::getNotificationHistory ( ) const [inline]
```

Gets all past notifications.

Returns

Const reference to notification history

4.49.3.5 getPlantQuantity()

```
int SalesFloorObserver::getPlantQuantity (
    Plant * plant ) const
```

Gets quantity of a specific plant.

Parameters

<i>plant</i>	Plant to query
--------------	--------------------------------

Returns

Quantity available

4.49.3.6 isPlantAvailable()

```
bool SalesFloorObserver::isPlantAvailable (
    Plant * plant ) const
```

Checks if a plant is available.

Parameters

<i>plant</i>	Plant to check
--------------	--------------------------------

Returns

true if plant is available (quantity > 0)

4.49.3.7 notifyStaff()

```
void SalesFloorObserver::notifyStaff (
    const std::string & message )
```

Notifies all staff members with a message.

Parameters

<i>message</i>	Message to send to staff
----------------	--------------------------

The documentation for this class was generated from the following files:

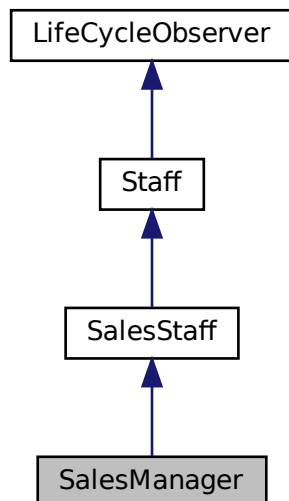
- [SalesFloorObserver.h](#)
- SalesFloorObserver.cpp

4.50 SalesManager Class Reference

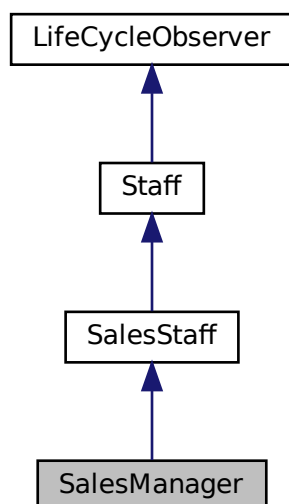
Represents a manager in the sales department.

```
#include <SalesManager.h>
```

Inheritance diagram for SalesManager:



Collaboration diagram for SalesManager:



Public Member Functions

- [SalesManager](#) (std::string name)

- Construct a sales manager with given name.*
 - virtual [~SalesManager](#) ()
Virtual destructor.
 - bool [staffCanHandle](#) (std::string type) override
Check if this staff member can handle a query type.
 - void [reply](#) ([Query](#) *query) override
Generate a reply for a query.

4.50.1 Detailed Description

Represents a manager in the sales department.

Handles escalated sales queries and has additional authority for special deals and events.

See also

[SalesStaff](#)

4.50.2 Constructor & Destructor Documentation

4.50.2.1 SalesManager()

```
SalesManager::SalesManager (  
    std::string name )
```

Construct a sales manager with given name.

Parameters

<i>name</i>	The manager's name
-------------	--------------------

4.50.3 Member Function Documentation

4.50.3.1 reply()

```
void SalesManager::reply (  
    Query * query ) [override], [virtual]
```

Generate a reply for a query.

Parameters

<i>query</i>	The query to reply to (non-owning pointer)
--------------	--

Implements [SalesStaff](#).

4.50.3.2 staffCanHandle()

```
bool SalesManager::staffCanHandle (
    std::string type ) [override], [virtual]
```

Check if this staff member can handle a query type.

Parameters

<i>type</i>	The query type to check
-------------	-------------------------

Returns

true if the staff member can handle it, false otherwise

Implements [SalesStaff](#).

The documentation for this class was generated from the following files:

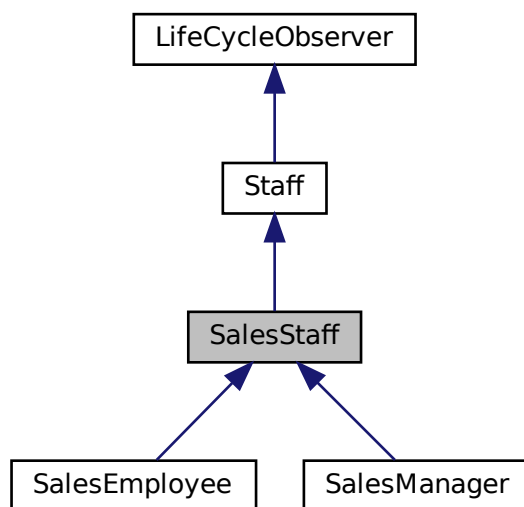
- [SalesManager.h](#)
- SalesManager.cpp

4.51 SalesStaff Class Reference

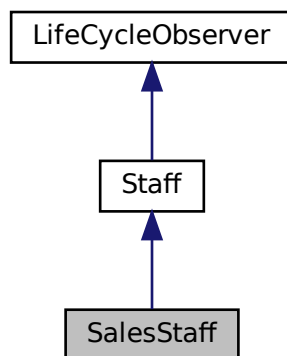
Abstract base class for staff members in the sales department.

```
#include <SalesStaff.h>
```

Inheritance diagram for SalesStaff:



Collaboration diagram for SalesStaff:



Public Member Functions

- `SalesStaff` (std::string name)
Construct a sales staff member with given name.
- virtual `~SalesStaff` ()
Virtual destructor.
- void `handleQuery` (Query *query) override

Handle an incoming query using chain of responsibility.

- virtual bool [staffCanHandle](#) (std::string type)=0

Check if this staff member can handle a query type.

- virtual void [reply](#) ([Query](#) *query)=0

Generate a reply for a query.

4.51.1 Detailed Description

Abstract base class for staff members in the sales department.

Provides common functionality for sales staff including query handling and response generation. Concrete classes must implement [staffCanHandle\(\)](#) to specify their responsibilities and [reply\(\)](#) to handle queries.

See also

[Staff](#)

4.51.2 Constructor & Destructor Documentation

4.51.2.1 SalesStaff()

```
SalesStaff::SalesStaff (
    std::string name )
```

Construct a sales staff member with given name.

Parameters

<i>name</i>	The staff member's name
-------------	-------------------------

4.51.3 Member Function Documentation

4.51.3.1 handleQuery()

```
void SalesStaff::handleQuery (
    Query * query ) [override], [virtual]
```

Handle an incoming query using chain of responsibility.

Parameters

<i>query</i>	The query to handle (non-owning pointer)
--------------	--

Implements [Staff](#).

4.51.3.2 reply()

```
virtual void SalesStaff::reply (
    Query * query ) [pure virtual]
```

Generate a reply for a query.

Parameters

<i>query</i>	The query to reply to (non-owning pointer)
--------------	--

Implemented in [SalesManager](#), and [SalesEmployee](#).

4.51.3.3 staffCanHandle()

```
virtual bool SalesStaff::staffCanHandle (
    std::string type ) [pure virtual]
```

Check if this staff member can handle a query type.

Parameters

<i>type</i>	The query type to check
-------------	-------------------------

Returns

true if the staff member can handle it, false otherwise

Implemented in [SalesManager](#), and [SalesEmployee](#).

The documentation for this class was generated from the following files:

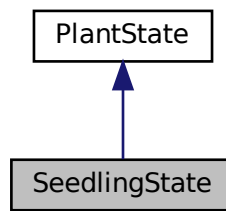
- [SalesStaff.h](#)
- SalesStaff.cpp

4.52 SeedlingState Class Reference

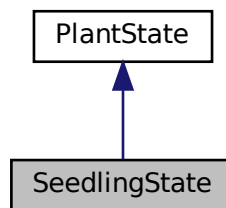
Represents the initial growth stage of a plant's life cycle. In this state, the plant is vulnerable and requires careful nurturing to develop into a healthy mature plant.

```
#include <SeedlingState.h>
```

Inheritance diagram for SeedlingState:



Collaboration diagram for SeedlingState:



Public Member Functions

- void `applyCare` (`PlantLifeCycle` *context, `Plant` *plant, `PlantCareRoutine` *routine) override
Applies care to the plant in this state.
- bool `evaluate` (`PlantLifeCycle` *context, `Plant` *plant) override
Evaluates the plant's condition and transitions to another state if necessary.
- std::string `getName` () const override
Gets the name of this state.
- `PlantState` * `clone` () const override

4.52.1 Detailed Description

Represents the initial growth stage of a plant's life cycle. In this state, the plant is vulnerable and requires careful nurturing to develop into a healthy mature plant.

4.52.2 Member Function Documentation

4.52.2.1 applyCare()

```
void SeedlingState::applyCare (
    PlantLifeCycle * context,
    Plant * plant,
    PlantCareRoutine * routine ) [override], [virtual]
```

Applies care to the plant in this state.

Parameters

<i>context</i>	The plant's life cycle context.
<i>plant</i>	The plant object being cared for.
<i>routine</i>	The care routine to apply.

Implements [PlantState](#).

4.52.2.2 evaluate()

```
bool SeedlingState::evaluate (
    PlantLifeCycle * context,
    Plant * plant ) [override], [virtual]
```

Evaluates the plant's condition and transitions to another state if necessary.

Parameters

<i>context</i>	The plant's life cycle context.
<i>plant</i>	The plant object being evaluated.

Returns

True if the plant remains in this state, false otherwise.

Implements [PlantState](#).

4.52.2.3 getName()

```
std::string SeedlingState::getName ( ) const [override], [virtual]
```

Gets the name of this state.

Returns

The name of this state.

Implements [PlantState](#).

The documentation for this class was generated from the following files:

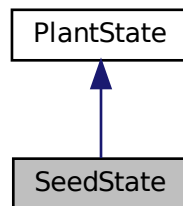
- [SeedlingState.h](#)
- [SeedlingState.cpp](#)

4.53 SeedState Class Reference

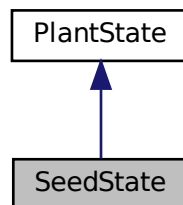
Represents the initial state of a plant's life cycle, where it is a seed. In this state, the seed is dormant and requires specific conditions to germinate and grow into a seedling.

```
#include <SeedState.h>
```

Inheritance diagram for SeedState:



Collaboration diagram for SeedState:



Public Member Functions

- void `applyCare` (`PlantLifeCycle` *context, `Plant` *plant, `PlantCareRoutine` *routine) override
Applies care to the plant in this state.
- bool `evaluate` (`PlantLifeCycle` *context, `Plant` *plant) override
Evaluates the plant's condition and transitions to another state if necessary.
- std::string `getName` () const override
Gets the name of this state.
- `PlantState` * `clone` () const override

4.53.1 Detailed Description

Represents the initial state of a plant's life cycle, where it is a seed. In this state, the seed is dormant and requires specific conditions to germinate and grow into a seedling.

4.53.2 Member Function Documentation

4.53.2.1 applyCare()

```
void SeedState::applyCare (
    PlantLifeCycle * context,
    Plant * plant,
    PlantCareRoutine * routine ) [override], [virtual]
```

Applies care to the plant in this state.

Parameters

<i>context</i>	The plant's life cycle context.
<i>plant</i>	The plant object being cared for.
<i>routine</i>	The care routine to apply.

Implements [PlantState](#).

4.53.2.2 evaluate()

```
bool SeedState::evaluate (
    PlantLifeCycle * context,
    Plant * plant ) [override], [virtual]
```

Evaluates the plant's condition and transitions to another state if necessary.

Parameters

<i>context</i>	The plant's life cycle context.
<i>plant</i>	The plant object being evaluated.

Returns

True if the plant remains in this state, false otherwise.

Implements [PlantState](#).

4.53.2.3 getName()

```
std::string SeedState::getName ( ) const [override], [virtual]
```

Gets the name of this state.

Returns

The name of this state.

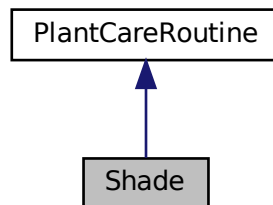
Implements [PlantState](#).

The documentation for this class was generated from the following files:

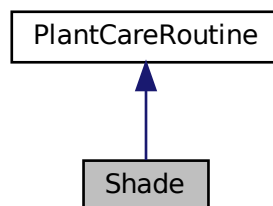
- [SeedState.h](#)
- [SeedState.cpp](#)

4.54 Shade Class Reference

Inheritance diagram for Shade:



Collaboration diagram for Shade:



Public Member Functions

- void [Fertilizing](#) ([Plant](#) *p) override
Apply fertilization routine to a plant.
- void [Watering](#) ([Plant](#) *p) override
Apply watering routine to a plant.
- void [Sunlight](#) ([Plant](#) *p) override
Apply sunlight exposure routine to a plant.
- void [printCareRoutine](#) () override
Print the details of the care routine.

Additional Inherited Members

The documentation for this class was generated from the following files:

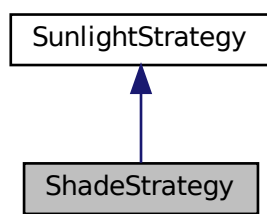
- [Shade.h](#)
- [Shade.cpp](#)

4.55 ShadeStrategy Class Reference

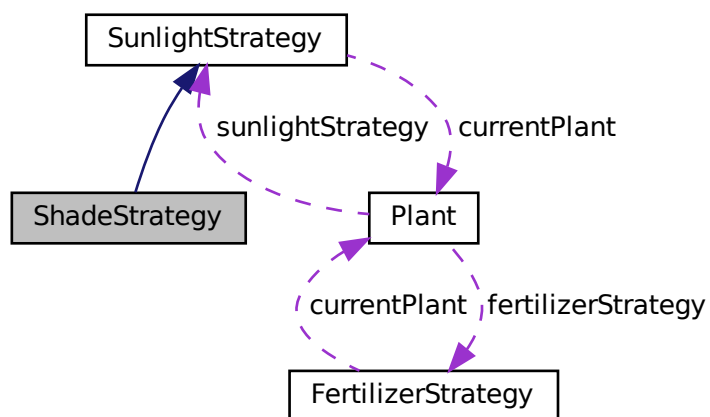
[SunlightStrategy](#) implementation for shade.

```
#include <ShadeStrategy.h>
```

Inheritance diagram for ShadeStrategy:



Collaboration diagram for ShadeStrategy:



Public Member Functions

- void [applySunlight](#) ([Plant](#) *p) override

Applies the shade sunlight strategy to a plant. This method applies the shade sunlight strategy to the given plant by reducing its sunlight exposure.

- std::string [getStrategyName](#) () const override

Gets the name of the sunlight strategy. This method must be implemented by concrete strategy classes.

Additional Inherited Members

4.55.1 Detailed Description

[SunlightStrategy](#) implementation for shade.

4.55.2 Member Function Documentation

4.55.2.1 [applySunlight\(\)](#)

```
void ShadeStrategy::applySunlight (  
    Plant * p ) [override], [virtual]
```

Applies the shade sunlight strategy to a plant. This method applies the shade sunlight strategy to the given plant by reducing its sunlight exposure.

Parameters

<i>p</i>	The plant to apply the shade sunlight strategy to.
----------	--

Note

The current implementation does not modify the plant's sunlight level, as the setCurrentSunlight call adds 0 to the current value.

Implements [SunlightStrategy](#).

4.55.2.2 [getStrategyName\(\)](#)

```
std::string ShadeStrategy::getStrategyName ( ) const [override], [virtual]
```

Gets the name of the sunlight strategy. This method must be implemented by concrete strategy classes.

Returns

The name of the sun strategy.

Implements [SunlightStrategy](#).

The documentation for this class was generated from the following files:

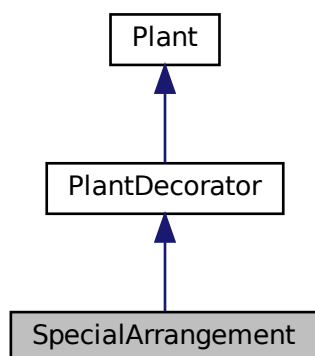
- [ShadeStrategy.h](#)
- [ShadeStrategy.cpp](#)

4.56 SpecialArrangement Class Reference

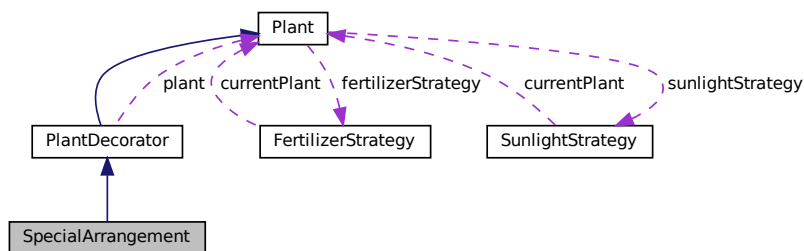
Concrete decorator that adds a special arrangement to a plant.

```
#include <SpecialArrangement.h>
```

Inheritance diagram for SpecialArrangement:



Collaboration diagram for SpecialArrangement:



Public Member Functions

- **SpecialArrangement** ([Plant *plant](#))
- `std::string` [getDescription](#) () const override
Get the decorated plant's description.
- `double` [getPrice](#) () const override
Get the decorated plant's price.

Additional Inherited Members

4.56.1 Detailed Description

Concrete decorator that adds a special arrangement to a plant.

The documentation for this class was generated from the following files:

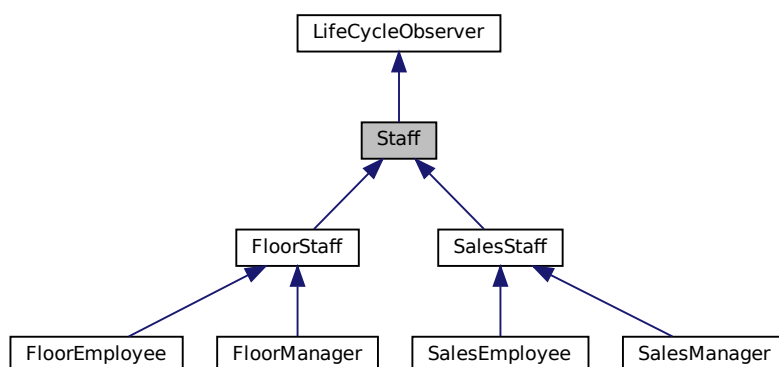
- [SpecialArrangement.h](#)
- [SpecialArrangement.cpp](#)

4.57 Staff Class Reference

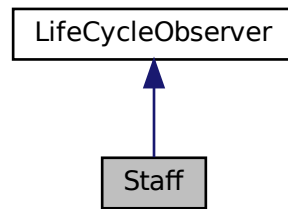
Base class representing a staff member and lifecycle observer.

```
#include <Staff.h>
```

Inheritance diagram for Staff:



Collaboration diagram for Staff:



Public Member Functions

- void `getStock` (`Plant` *plant)
Request stock information or trigger stock-related behaviour.
- virtual void `handleQuery` (`Query` *query)=0
Handle an incoming `Query`.
- `Staff` (std::string name)
Construct a `Staff` instance with the given name.
- virtual `~Staff` ()
Virtual destructor.
- std::string `getName` ()
Get this staff member's name.
- void `setNext` (`Staff` *staff)
Set the next staff member in the chain-of-responsibility.
- `Staff` * `getNext` ()
Get the next staff member in the chain.
- void `isHealthy` (`PlantLifeCycle` *lifeCycle)
Notification callback from `PlantLifeCycle` to indicate health state.

4.57.1 Detailed Description

Base class representing a staff member and lifecycle observer.

Inheritance:

- `Staff` : `LifecycleObserver`

Data members:

- next: pointer to the next `Staff` in the chain (non-owning).
- name: human-readable staff member name.
- careRoutine: optional `PlantCareRoutine` associated with this staff member.
- salesFloorObserver: optional observer used for sales-floor notifications.

4.57.2 Constructor & Destructor Documentation

4.57.2.1 Staff()

```
Staff::Staff (
    std::string name )
```

Construct a [Staff](#) instance with the given name.

Parameters

<i>name</i>	Human-readable name of the staff member.
-------------	--

4.57.2.2 ~Staff()

```
Staff::~~Staff ( ) [virtual]
```

Virtual destructor.

Ensures derived destructors are invoked correctly.

4.57.3 Member Function Documentation

4.57.3.1 getName()

```
std::string Staff::getName ( )
```

Get this staff member's name.

Returns

Name string.

4.57.3.2 getNext()

```
Staff * Staff::getNext ( )
```

Get the next staff member in the chain.

Returns

Non-owning pointer to the next [Staff](#), or nullptr if none.

4.57.3.3 getStock()

```
void Staff::getStock (
    Plant * plant )
```

Request stock information or trigger stock-related behaviour.

Parameters

<i>plant</i>	Plant for which to get stock information.
--------------	---

Concrete implementations or callers may use this hook to fetch or report stock data. Default behaviour is implementation-specific.

4.57.3.4 handleQuery()

```
virtual void Staff::handleQuery (
    Query * query ) [pure virtual]
```

Handle an incoming Query.

This pure virtual function must be implemented by concrete staff types to process or forward Query objects. Implementations are responsible for deciding whether to reply or pass the query on.

Parameters

<i>query</i>	Non-owning pointer to the Query to handle.
--------------	--

Implemented in SalesStaff, and FloorStaff.

4.57.3.5 isHealthy()

```
void Staff::isHealthy (
    PlantLifeCycle * p )
```

Notification callback from PlantLifeCycle to indicate health state.

As part of the LifeCycleObserver interface, this method is invoked when observed PlantLifeCycle instances produce lifecycle events. Implementations can use careRoutine to adjust or log plant care state.

Parameters

<i>lifeCycle</i>	Pointer to the PlantLifeCycle that triggered the update.
------------------	--

```
void Staff::update(PlantLifeCycle* p){ Plant* plant = p->getPlant(); std::string state = p->getState();
std::cout<< "[Staff] " << name << " noticed " << p->getName() << " is now in state: " << state << std::endl;
if(!p->isHealthy()){ careRoutine = careRoutine->PlantCare(plant); std::cout << "[Staff] " << name; if(state ==
```

```
"Withered" && p->getCurrentWater() < 10 ){ careRoutine->Watering(plant); } else if(state == "Withered" && p->getCurrentSunlight() < 10){ careRoutine->Sunlight(plant); } else if(state == "Withered" && p->getCurrentNutrients() < 10){ careRoutine->Fertilizing(plant); }
```

```
if(state == "Distressed" && p->getCurrentWater() < 20 ){ careRoutine->Watering(plant); } else if(state == "Distressed" && p->getCurrentSunlight() < 20){ careRoutine->Sunlight(plant); } else if(state == "Distressed" && p->getCurrentNutrients() < 20){ careRoutine->Fertilizing(plant); } } else{ std::cout << "[Staff] " << name << " continues regular maintenance of " << p->getName() << std::endl; } }
```

```
void Staff::update(PlantLifeCycle* p) { Plant* plant = p->getPlant(); std::string stateName = p->getState();
```

```
std::cout << "[Staff] " << name << " noticed " << p->getName() << " is now in state: " << stateName << std::endl;
```

```
careRoutine = careRoutine->PlantCare(plant);
```

```
if (!p->isHealthy()) { std::cout << "[Staff] " << name << " is applying care..." << std::endl; p->getStateObj()->applyCare(p, plant, careRoutine); //delete careRoutine; } else { std::cout << "[Staff] " << name << " continues regular maintenance of " << p->getName() << std::endl; } Plant* plant = p->getPlant(); std::string stateName = p->getState();
```

```
std::cout << "[Staff] " << name << " noticed " << p->getName() << " is now in state: " << stateName << std::endl;
```

```
Create care routine PlantCareRoutine* routine = PlantCareRoutine::PlantCare(plant);
```

```
Apply care through current state PlantState* currentState = p->getStateObj(); if (currentState) { currentState->applyCare(p, plant, routine); }
```

FIX: Delete the routine immediately after use delete routine;

```
std::cout << "[Staff] " << name << " continues regular maintenance of " << p->getName() << std::endl; }
```

4.57.3.6 setNext()

```
void Staff::setNext (
    Staff * staff )
```

Set the next staff member in the chain-of-responsibility.

The [Staff](#) class does not take ownership of the provided pointer.

Parameters

<i>staff</i>	Non-owning pointer to the next Staff .
--------------	--

The documentation for this class was generated from the following files:

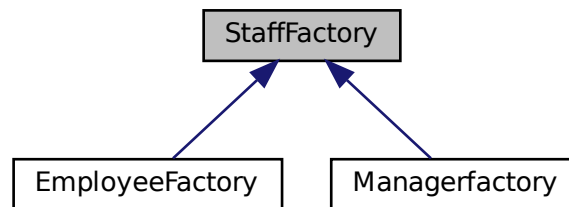
- [Staff.h](#)
- [Staff.cpp](#)

4.58 StaffFactory Class Reference

Abstract base factory for creating different types of staff members.

```
#include <StaffFactory.h>
```

Inheritance diagram for StaffFactory:



Public Member Functions

- [StaffFactory](#) ()
Default constructor.
- virtual [~StaffFactory](#) ()
Virtual destructor.
- virtual [FloorStaff](#) * [createFloorStaff](#) (std::string name)=0
Create a floor staff member.
- virtual [SalesStaff](#) * [createSalesStaff](#) (std::string name)=0
Create a sales staff member.

4.58.1 Detailed Description

Abstract base factory for creating different types of staff members.

This abstract factory defines the interface for creating floor and sales staff members. Concrete factories implement these methods to create specific staff types (employees or managers).

4.58.2 Member Function Documentation

4.58.2.1 createFloorStaff()

```
virtual FloorStaff* StaffFactory::createFloorStaff (  
    std::string name ) [pure virtual]
```

Create a floor staff member.

Parameters

<i>name</i>	The name of the staff member to create
-------------	--

Returns

Pointer to the created [FloorStaff](#) (caller owns memory)

Implemented in [Managerfactory](#), and [EmployeeFactory](#).

4.58.2.2 createSalesStaff()

```
virtual SalesStaff* StaffFactory::createSalesStaff (
    std::string name ) [pure virtual]
```

Create a sales staff member.

Parameters

<i>name</i>	The name of the staff member to create
-------------	--

Returns

Pointer to the created [SalesStaff](#) (caller owns memory)

Implemented in [Managerfactory](#), and [EmployeeFactory](#).

The documentation for this class was generated from the following files:

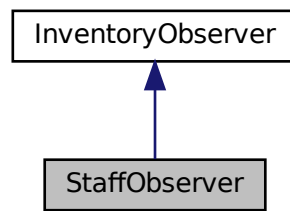
- [StaffFactory.h](#)
- [StaffFactory.cpp](#)

4.59 StaffObserver Class Reference

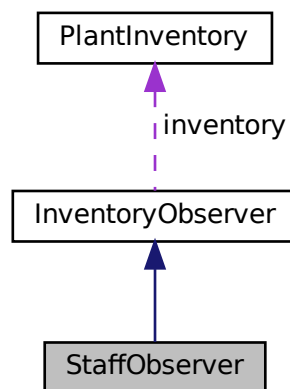
Concrete observer that tracks inventory for staff.

```
#include <StaffObserver.h>
```

Inheritance diagram for StaffObserver:



Collaboration diagram for StaffObserver:



Public Member Functions

- [StaffObserver](#) ([PlantInventory](#) *inventory)
Constructs a staff observer.
- void [update](#) () override
Updates when inventory changes.

Additional Inherited Members

4.59.1 Detailed Description

Concrete observer that tracks inventory for staff.

This observer monitors inventory changes and maintains internal tracking for staff members.

4.59.2 Constructor & Destructor Documentation

4.59.2.1 StaffObserver()

```
StaffObserver::StaffObserver (
    PlantInventory * inventory )
```

Constructs a staff observer.

Parameters

<i>inventory</i>	Inventory to observe
------------------	----------------------

The documentation for this class was generated from the following files:

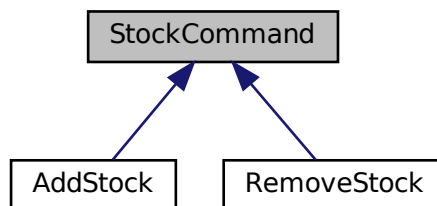
- [StaffObserver.h](#)
- [StaffObserver.cpp](#)

4.60 StockCommand Class Reference

Abstract base class for command pattern in stock management.

```
#include <StockCommand.h>
```

Inheritance diagram for StockCommand:



Public Member Functions

- virtual bool [execute](#) ()=0
Executes the stock command.
- virtual [~StockCommand](#) ()
Virtual destructor.

4.60.1 Detailed Description

Abstract base class for command pattern in stock management.

This interface defines the execute method that all concrete stock commands must implement.

4.60.2 Constructor & Destructor Documentation

4.60.2.1 ~StockCommand()

```
virtual StockCommand::~~StockCommand ( ) [inline], [virtual]
```

Virtual destructor.

Ensures proper cleanup in derived command classes.

4.60.3 Member Function Documentation

4.60.3.1 execute()

```
virtual bool StockCommand::execute ( ) [pure virtual]
```

Executes the stock command.

Returns

true on success, false on failure

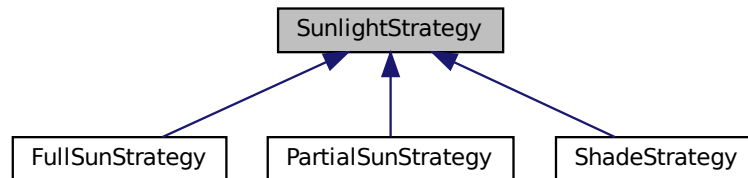
Implemented in [RemoveStock](#), and [AddStock](#).

The documentation for this class was generated from the following file:

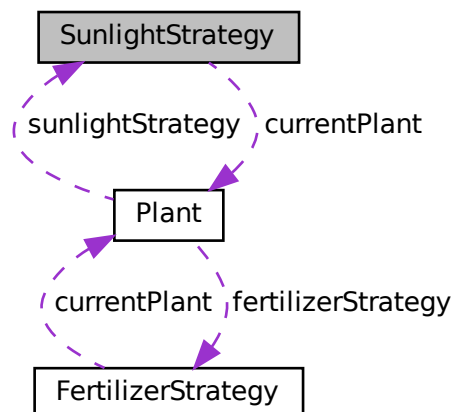
- [StockCommand.h](#)

4.61 SunlightStrategy Class Reference

Inheritance diagram for SunlightStrategy:



Collaboration diagram for SunlightStrategy:



Public Member Functions

- virtual void `applySunlight (Plant *p)=0`
Applies sun to a plant according to the strategy. This method must be implemented by concrete strategy classes.
- virtual std::string `getStrategyName () const =0`
Gets the name of the sunlight strategy. This method must be implemented by concrete strategy classes.

Protected Attributes

- `Plant * currentPlant`

4.61.1 Member Function Documentation

4.61.1.1 applySunlight()

```
virtual void SunlightStrategy::applySunlight (
    Plant * p ) [pure virtual]
```

Applies sun to a plant according to the strategy. This method must be implemented by concrete strategy classes.

Parameters

<i>p</i>	The plant to be put in the sun.
----------	---------------------------------

Implemented in [ShadeStrategy](#), [PartialSunStrategy](#), and [FullSunStrategy](#).

4.61.1.2 getStrategyName()

```
virtual std::string SunlightStrategy::getStrategyName ( ) const [pure virtual]
```

Gets the name of the sunlight strategy. This method must be implemented by concrete strategy classes.

Returns

The name of the sun strategy.

Implemented in [ShadeStrategy](#), [PartialSunStrategy](#), and [FullSunStrategy](#).

4.61.2 Member Data Documentation

4.61.2.1 currentPlant

```
Plant* SunlightStrategy::currentPlant [protected]
```

Pointer to the plant currently exposed to sunlight.

The documentation for this class was generated from the following file:

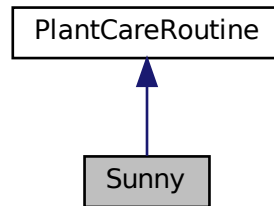
- [SunlightStrategy.h](#)

4.62 Sunny Class Reference

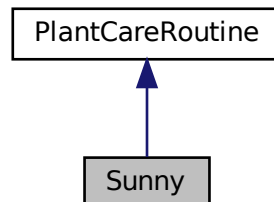
Implements the care routine for sun-loving plants.

```
#include <Sunny.h>
```

Inheritance diagram for Sunny:



Collaboration diagram for Sunny:



Public Member Functions

- `Sunny ()`
Constructs a `Sunny` care routine with appropriate strategies.
- `void Fertilizing (Plant *p) override`
Applies fertilizing strategy for sunny plants.
- `void Watering (Plant *p) override`
Applies watering strategy for sunny plants.
- `void Sunlight (Plant *p) override`
Applies sunlight strategy for sunny plants.
- `void printCareRoutine () override`
Prints the care routine details for debugging or display.

Additional Inherited Members

4.62.1 Detailed Description

Implements the care routine for sun-loving plants.

Provides high sunlight exposure, moderate watering, and rich fertilizing patterns.

4.62.2 Member Function Documentation

4.62.2.1 Fertilizing()

```
void Sunny::Fertilizing (  
    Plant * p ) [override], [virtual]
```

Applies fertilizing strategy for sunny plants.

Parameters

<i>p</i>	Pointer to the plant to fertilize.
----------	------------------------------------

Implements [PlantCareRoutine](#).

4.62.2.2 Sunlight()

```
void Sunny::Sunlight (  
    Plant * p ) [override], [virtual]
```

Applies sunlight strategy for sunny plants.

Parameters

<i>p</i>	Pointer to the plant to expose to sunlight.
----------	---

Implements [PlantCareRoutine](#).

4.62.2.3 Watering()

```
void Sunny::Watering (  
    Plant * p ) [override], [virtual]
```

Applies watering strategy for sunny plants.

Parameters

p	Pointer to the plant to water.
-----	--------------------------------

Implements [PlantCareRoutine](#).

The documentation for this class was generated from the following files:

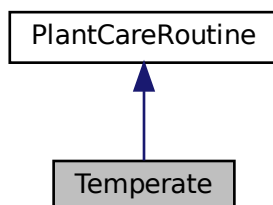
- [Sunny.h](#)
- [Sunny.cpp](#)

4.63 Temperate Class Reference

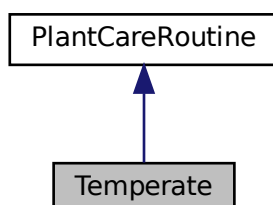
Implements plant care routine for temperate climate plants.

```
#include <Temperate.h>
```

Inheritance diagram for Temperate:



Collaboration diagram for Temperate:



Public Member Functions

- [Temperate](#) ()
Constructs a [Temperate](#) care routine with balanced strategies.
- void [Fertilizing](#) ([Plant](#) *p) override
Applies fertilizer suitable for temperate plants.
- void [Watering](#) ([Plant](#) *p) override
Applies watering strategy suitable for temperate plants.
- void [Sunlight](#) ([Plant](#) *p) override
Applies sunlight exposure for temperate conditions.
- void [printCareRoutine](#) () override
Displays the care routine summary for temperate plants.

Additional Inherited Members

4.63.1 Detailed Description

Implements plant care routine for temperate climate plants.

4.63.2 Member Function Documentation

4.63.2.1 Fertilizing()

```
void Temperate::Fertilizing (  
    Plant * p ) [override], [virtual]
```

Applies fertilizer suitable for temperate plants.

Parameters

<i>p</i>	Pointer to the plant to fertilize.
----------	------------------------------------

Implements [PlantCareRoutine](#).

4.63.2.2 Sunlight()

```
void Temperate::Sunlight (  
    Plant * p ) [override], [virtual]
```

Applies sunlight exposure for temperate conditions.

Parameters

<i>p</i>	Pointer to the plant to expose to sunlight.
----------	---

Implements [PlantCareRoutine](#).

4.63.2.3 Watering()

```
void Temperate::Watering (  
    Plant * p ) [override], [virtual]
```

Applies watering strategy suitable for temperate plants.

Parameters

<i>p</i>	Pointer to the plant to water.
----------	--------------------------------

Implements [PlantCareRoutine](#).

The documentation for this class was generated from the following files:

- [Temperate.h](#)
- [Temperate.cpp](#)

4.64 Transaction Class Reference

Encapsulates all details of a plant purchase transaction.

```
#include <Transaction.h>
```

Public Member Functions

- [Transaction](#) (const std::string &orderNum, double amount, int quantity)
Constructs a transaction.
- void [setTransaction](#) (const std::string &orderNum, double amount, int quantity)
Updates transaction details.
- void [setPaymentStrategy](#) ([PaymentStrategy](#) *method)
Sets the payment strategy.
- void [processPayment](#) () const
Processes payment using the selected strategy.
- void [getDetails](#) () const
Displays the transaction details.
- std::string [getTransactionId](#) () const
Retrieves the transaction ID.
- double [getAmount](#) () const

- Retrieves the transaction amount.*
- `int` [getQuantity](#) () const
- Retrieves quantity purchased.*
- `std::string` [getPaymentMethod](#) () const
- Retrieves payment method name.*
- `void` [addDecoration](#) (const `std::string` &decor)
- Adds a decorative feature to the transaction.*
- `const std::vector< std::string > &` [getDecorations](#) () const
- Returns all applied decorations.*
- `TransactionSnapshot` [createSnapshot](#) () const
- Creates a snapshot of the current transaction state.*
- `void` [restoreSnapshot](#) (const `TransactionSnapshot` &snapshot)
- Restores the transaction from a snapshot.*
- `Transaction *` [clone](#) () const
- Clones this transaction.*

4.64.1 Detailed Description

Encapsulates all details of a plant purchase transaction.

4.64.2 Constructor & Destructor Documentation

4.64.2.1 Transaction()

```
Transaction::Transaction (
    const std::string & orderNum,
    double amount,
    int quantity )
```

Constructs a transaction.

Parameters

<i>orderNum</i>	Unique identifier for the order.
<i>amount</i>	Total purchase amount.
<i>quantity</i>	Number of items purchased.

4.64.3 Member Function Documentation

4.64.3.1 addDecoration()

```
void Transaction::addDecoration (
    const std::string & decor )
```

Adds a decorative feature to the transaction.

Parameters

<i>decor</i>	Name of the decoration.
--------------	-------------------------

4.64.3.2 clone()

```
Transaction * Transaction::clone ( ) const
```

Clones this transaction.

Returns

Pointer to a new [Transaction](#) object with the same state.

4.64.3.3 createSnapshot()

```
TransactionSnapshot Transaction::createSnapshot ( ) const
```

Creates a snapshot of the current transaction state.

Returns

A [TransactionSnapshot](#) instance.

4.64.3.4 getAmount()

```
double Transaction::getAmount ( ) const
```

Retrieves the transaction amount.

Returns

Total amount.

4.64.3.5 getDecorations()

```
const std::vector< std::string > & Transaction::getDecorations ( ) const
```

Returns all applied decorations.

Returns

Vector of decoration names.

4.64.3.6 getPaymentMethod()

```
std::string Transaction::getPaymentMethod ( ) const
```

Retrieves payment method name.

Returns

Payment strategy name.

4.64.3.7 getQuantity()

```
int Transaction::getQuantity ( ) const
```

Retrieves quantity purchased.

Returns

Quantity of items.

4.64.3.8 getTransactionId()

```
std::string Transaction::getTransactionId ( ) const
```

Retrieves the transaction ID.

Returns

[Order](#) number as a string.

4.64.3.9 restoreSnapshot()

```
void Transaction::restoreSnapshot (
    const TransactionSnapshot & snapshot )
```

Restores the transaction from a snapshot.

Parameters

<i>snapshot</i>	The snapshot to restore from.
-----------------	-------------------------------

4.64.3.10 setPaymentStrategy()

```
void Transaction::setPaymentStrategy (
    PaymentStrategy * method )
```

Sets the payment strategy.

Parameters

<i>method</i>	Pointer to the chosen PaymentStrategy .
---------------	---

4.64.3.11 setTransaction()

```
void Transaction::setTransaction (
    const std::string & orderNum,
    double amount,
    int quantity )
```

Updates transaction details.

Parameters

<i>orderNum</i>	Order identifier.
<i>amount</i>	Updated amount.
<i>quantity</i>	Updated quantity.

The documentation for this class was generated from the following files:

- [Transaction.h](#)
- Transaction.cpp

4.65 TransactionHistory Class Reference

Manages the storage and retrieval of transaction snapshots.

```
#include <TransactionHistory.h>
```

Public Member Functions

- void [addSnapshot](#) (const [TransactionSnapshot](#) &snapshot)
Adds a new snapshot to the transaction history.
- [TransactionSnapshot](#) [getSnapshot](#) (int index) const
Retrieves a specific snapshot by index.
- int [getHistorySize](#) () const
Gets the total number of snapshots in the history.

4.65.1 Detailed Description

Manages the storage and retrieval of transaction snapshots.

Provides functionality to add new snapshots, access previous ones, and get the total number of transactions recorded.

4.65.2 Member Function Documentation

4.65.2.1 addSnapshot()

```
void TransactionHistory::addSnapshot (  
    const TransactionSnapshot & snapshot )
```

Adds a new snapshot to the transaction history.

Parameters

<i>snapshot</i>	The TransactionSnapshot object representing a completed order.
-----------------	--

4.65.2.2 getHistorySize()

```
int TransactionHistory::getHistorySize ( ) const
```

Gets the total number of snapshots in the history.

Returns

The number of stored [TransactionSnapshot](#) objects.

4.65.2.3 getSnapshot()

```
TransactionSnapshot TransactionHistory::getSnapshot (  
    int index ) const
```

Retrieves a specific snapshot by index.

Parameters

<i>index</i>	The index of the snapshot to retrieve.
--------------	--

Returns

The [TransactionSnapshot](#) at the specified index.

Warning

Calling with an invalid index may cause undefined behavior.

The documentation for this class was generated from the following files:

- [TransactionHistory.h](#)
- TransactionHistory.cpp

4.66 TransactionSnapshot Class Reference

Represents a snapshot of a completed transaction.

```
#include <TransactionSnapshot.h>
```

Public Member Functions

- [TransactionSnapshot](#) (const std::string &orderNum, double amount, int quantity, const std::vector< std::string > &decorations={})
Constructs a new [TransactionSnapshot](#) object.
- std::string [getOrderNum](#) () const
Retrieves the order number.
- double [getAmount](#) () const
Retrieves the total transaction amount.
- int [getQuantity](#) () const
Retrieves the quantity of items purchased.
- const std::vector< std::string > & [getDecorations](#) () const
Retrieves the decorations associated with the transaction.

4.66.1 Detailed Description

Represents a snapshot of a completed transaction.

Encapsulates details such as order number, transaction amount, quantity purchased, and applied decorations.

4.66.2 Constructor & Destructor Documentation

4.66.2.1 TransactionSnapshot()

```
TransactionSnapshot::TransactionSnapshot (
    const std::string & orderNum,
    double amount,
    int quantity,
    const std::vector< std::string > & decorations = {} )
```

Constructs a new [TransactionSnapshot](#) object.

Parameters

<i>orderNum</i>	The order number associated with the transaction.
<i>amount</i>	The total amount paid.
<i>quantity</i>	The quantity of items purchased.
<i>decorations</i>	A list of decorations applied (optional).

4.66.3 Member Function Documentation

4.66.3.1 getAmount()

```
double TransactionSnapshot::getAmount ( ) const
```

Retrieves the total transaction amount.

Returns

The amount paid for the order.

4.66.3.2 getDecorations()

```
const std::vector< std::string > & TransactionSnapshot::getDecorations ( ) const
```

Retrieves the decorations associated with the transaction.

Returns

A constant reference to the vector of decoration names.

4.66.3.3 getOrderNum()

```
std::string TransactionSnapshot::getOrderNum ( ) const
```

Retrieves the order number.

Returns

The order number as a string.

4.66.3.4 getQuantity()

```
int TransactionSnapshot::getQuantity ( ) const
```

Retrieves the quantity of items purchased.

Returns

The number of items in the order.

The documentation for this class was generated from the following files:

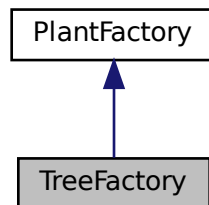
- [TransactionSnapshot.h](#)
- TransactionSnapshot.cpp

4.67 TreeFactory Class Reference

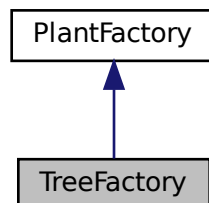
Concrete factory for creating tree plants.

```
#include <TreeFactory.h>
```

Inheritance diagram for TreeFactory:



Collaboration diagram for TreeFactory:



Public Member Functions

- [Plant](#) * [createPlant](#) (std::string plantType, double price) override
Creates a tree plant of the specified type.

4.67.1 Detailed Description

Concrete factory for creating tree plants.

This class implements the [PlantFactory](#) interface to create specific tree-type plants like [Oak](#) trees.

4.67.2 Member Function Documentation

4.67.2.1 createPlant()

```
Plant * TreeFactory::createPlant (
    std::string plantType = "Oak",
    double price = 100 ) [override], [virtual]
```

Creates a tree plant of the specified type.

Parameters

<i>plantType</i>	The type of tree to create (e.g., "Oak")
<i>price</i>	The price of the tree plant

Returns

Pointer to the newly created [Plant](#) object

Implements [PlantFactory](#).

The documentation for this class was generated from the following files:

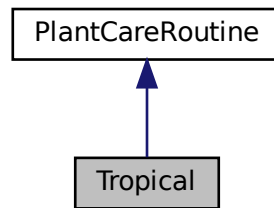
- [TreeFactory.h](#)
- [TreeFactory.cpp](#)

4.68 Tropical Class Reference

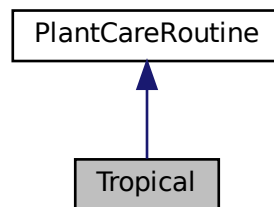
Represents the plant care routine for tropical plants.

```
#include <Tropical.h>
```

Inheritance diagram for Tropical:



Collaboration diagram for Tropical:



Public Member Functions

- `Tropical ()`
Constructs a `Tropical` care routine with appropriate strategies.
- `void Fertilizing (Plant *p) override`
Applies the fertilizing strategy to the plant.
- `void Watering (Plant *p) override`
Applies the watering strategy to the plant.
- `void Sunlight (Plant *p) override`
Applies the sunlight strategy to the plant.
- `void printCareRoutine () override`
Prints the current tropical care routine details.

Additional Inherited Members

4.68.1 Detailed Description

Represents the plant care routine for tropical plants.

This class provides specific care methods tailored for tropical plants, including fertilizing, watering, and sunlight exposure.

4.68.2 Member Function Documentation

4.68.2.1 Fertilizing()

```
void Tropical::Fertilizing (  
    Plant * p ) [override], [virtual]
```

Applies the fertilizing strategy to the plant.

Parameters

<i>p</i>	Pointer to the plant receiving fertilization.
----------	---

Implements [PlantCareRoutine](#).

4.68.2.2 Sunlight()

```
void Tropical::Sunlight (  
    Plant * p ) [override], [virtual]
```

Applies the sunlight strategy to the plant.

Parameters

<i>p</i>	Pointer to the plant receiving sunlight.
----------	--

Implements [PlantCareRoutine](#).

4.68.2.3 Watering()

```
void Tropical::Watering (  
    Plant * p ) [override], [virtual]
```

Applies the watering strategy to the plant.

Parameters

<i>p</i>	Pointer to the plant being watered.
----------	-------------------------------------

Implements [PlantCareRoutine](#).

The documentation for this class was generated from the following files:

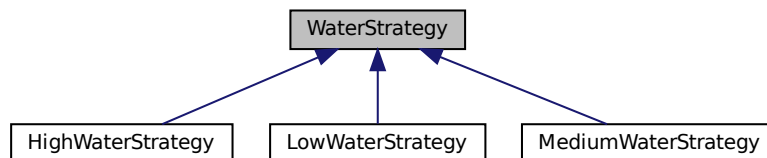
- [Tropical.h](#)
- [Tropical.cpp](#)

4.69 WaterStrategy Class Reference

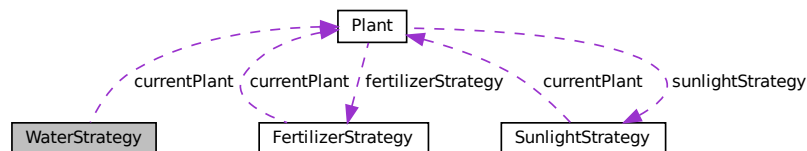
Abstract base class for water strategies. This class defines the interface for water strategies, which determine how much water to apply to a plant. Concrete strategy classes should inherit from this class and implement the `applyWater` and `getStrategyName` methods.

```
#include <WateringStrategy.h>
```

Inheritance diagram for WaterStrategy:



Collaboration diagram for WaterStrategy:



Public Member Functions

- virtual void `applyWater` (`Plant` *p)=0
Applies water to a plant according to the strategy. This method must be implemented by concrete strategy classes.
- virtual std::string `getStrategyName` () const =0
Gets the name of the water strategy. This method must be implemented by concrete strategy classes.

Protected Attributes

- `Plant` * `currentPlant`
The current plant being watered.

4.69.1 Detailed Description

Abstract base class for water strategies. This class defines the interface for water strategies, which determine how much water to apply to a plant. Concrete strategy classes should inherit from this class and implement the `applyWater` and `getStrategyName` methods.

Abstract base class for defining water strategies.

This class defines the interface for all watering strategies, determining how much and how often a plant should be watered.

4.69.2 Member Function Documentation

4.69.2.1 `applyWater()`

```
virtual void WaterStrategy::applyWater (
    Plant * p ) [pure virtual]
```

Applies water to a plant according to the strategy. This method must be implemented by concrete strategy classes.

Parameters

<i>p</i>	The plant to water.
----------	---------------------

Implemented in [MediumWaterStrategy](#), [LowWaterStrategy](#), and [HighWaterStrategy](#).

4.69.2.2 `getStrategyName()`

```
virtual std::string WaterStrategy::getStrategyName ( ) const [pure virtual]
```

Gets the name of the water strategy. This method must be implemented by concrete strategy classes.

Returns

The name of the water strategy.

Implemented in [MediumWaterStrategy](#), [LowWaterStrategy](#), and [HighWaterStrategy](#).

The documentation for this class was generated from the following file:

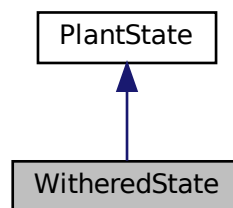
- `WateringStrategy.h`

4.70 WitheredState Class Reference

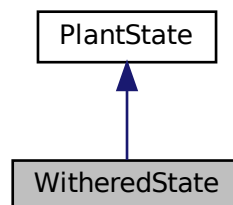
Represents the final state of a plant's life cycle, where it has withered and died. In this state, the plant is beyond recovery and will not respond to care.

```
#include <WitheredState.h>
```

Inheritance diagram for WitheredState:



Collaboration diagram for WitheredState:



Public Member Functions

- void `applyCare` (`PlantLifeCycle` *context, `Plant` *plant, `PlantCareRoutine` *routine) override
Attempts to apply care to a withered plant.
- bool `evaluate` (`PlantLifeCycle` *context, `Plant` *plant) override
Evaluates state transitions for a withered plant.
- std::string `getName` () const override
Retrieves the name of the current state.
- `PlantState` * `clone` () const override
Creates a clone of this state.

4.70.1 Detailed Description

Represents the final state of a plant's life cycle, where it has withered and died. In this state, the plant is beyond recovery and will not respond to care.

4.70.2 Member Function Documentation

4.70.2.1 `applyCare()`

```
void WitheredState::applyCare (
    PlantLifeCycle * context,
    Plant * plant,
    PlantCareRoutine * routine ) [override], [virtual]
```

Attempts to apply care to a withered plant.

Parameters

<i>context</i>	The plant life cycle context.
<i>plant</i>	The plant that is in the withered state.
<i>routine</i>	The care routine being attempted (has no effect).

Implements [PlantState](#).

4.70.2.2 `clone()`

```
PlantState * WitheredState::clone ( ) const [override], [virtual]
```

Creates a clone of this state.

Returns

A pointer to a new [WitheredState](#) object.

Implements [PlantState](#).

4.70.2.3 `evaluate()`

```
bool WitheredState::evaluate (
    PlantLifeCycle * context,
    Plant * plant ) [override], [virtual]
```

Evaluates state transitions for a withered plant.

Parameters

<i>context</i>	The life cycle managing the plant's state.
<i>plant</i>	The plant being evaluated.

Returns

Always returns false, as withered plants cannot recover.

Implements [PlantState](#).

4.70.2.4 getName()

```
std::string WitheredState::getName ( ) const [override], [virtual]
```

Retrieves the name of the current state.

Returns

A string representing "Withered State".

Implements [PlantState](#).

The documentation for this class was generated from the following files:

- [WitheredState.h](#)
- [WitheredState.cpp](#)

Chapter 5

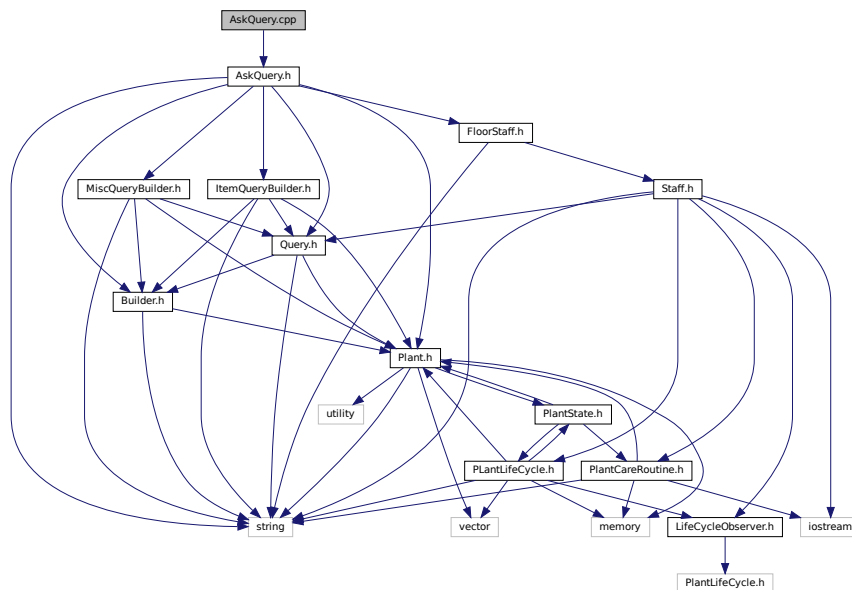
File Documentation

5.1 AskQuery.cpp File Reference

Implementation of [AskQuery](#) facade methods for [Query](#) creation.

```
#include "AskQuery.h"
```

Include dependency graph for AskQuery.cpp:



5.1.1 Detailed Description

Implementation of [AskQuery](#) facade methods for [Query](#) creation.

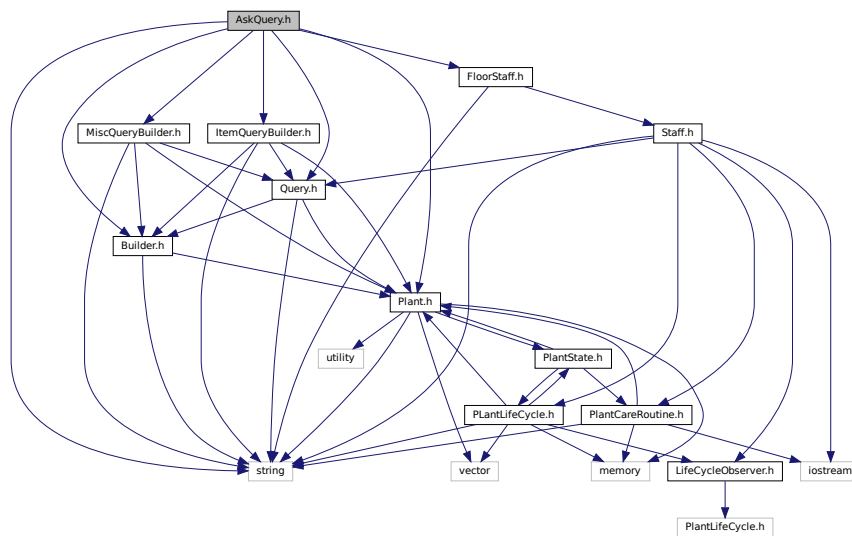
Contains implementations of describe() and careRoutine() methods that use [ItemQueryBuilder](#) to construct specialized [Query](#) objects.

5.2 AskQuery.h File Reference

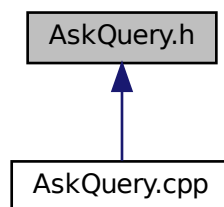
Director for creating different types of [Query](#) objects using builders.

```
#include "Builder.h"
#include "FloorStaff.h"
#include "ItemQueryBuilder.h"
#include "MiscQueryBuilder.h"
#include "Plant.h"
#include "Query.h"
#include <string>
```

Include dependency graph for AskQuery.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [AskQuery](#)

Director class that simplifies [Query](#) object creation using builders.

5.2.1 Detailed Description

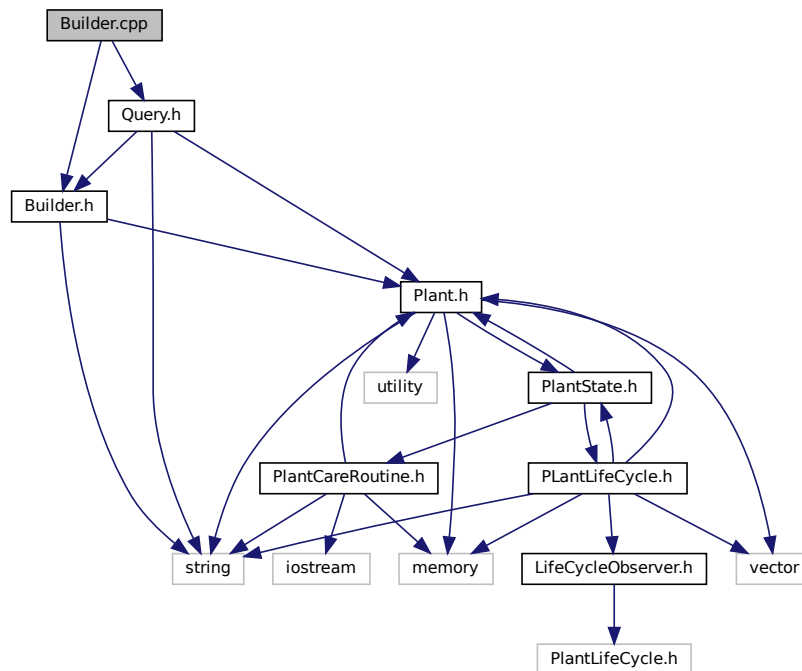
Director for creating different types of [Query](#) objects using builders.

This class provides a simplified interface for creating [Query](#) objects using the [Builder](#) pattern. It handles the details of choosing and configuring the appropriate builder ([ItemQueryBuilder](#) or [MiscQueryBuilder](#)) based on the type of query needed.

5.3 Builder.cpp File Reference

Implementation of common [Builder](#) functionality.

```
#include "Builder.h"
#include "Query.h"
Include dependency graph for Builder.cpp:
```



5.3.1 Detailed Description

Implementation of common [Builder](#) functionality.

Provides the default constructor and a generic build() that delegates [Query](#) construction to [Query](#)'s constructor taking a Builder*.

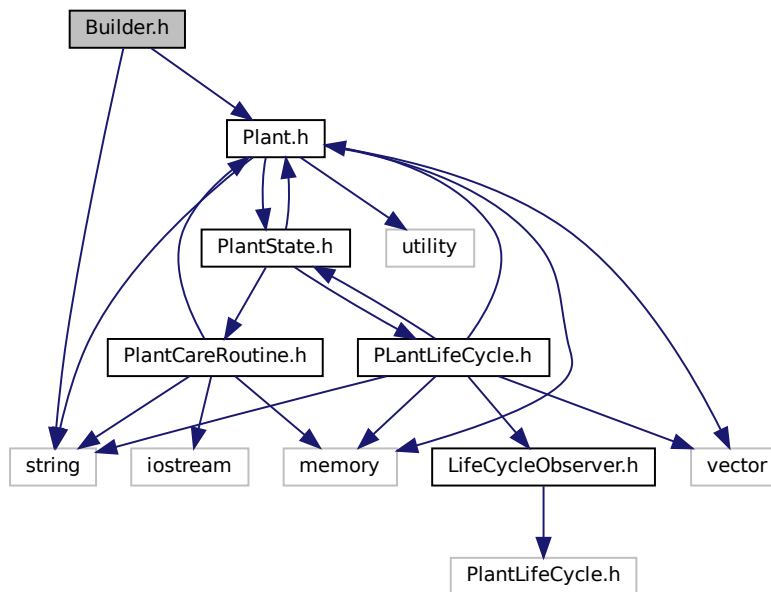
5.4 Builder.h File Reference

Defines the [Builder](#) interface for constructing [Query](#) objects.

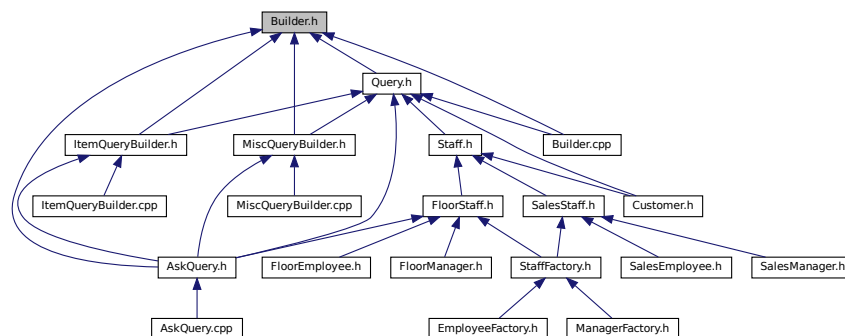
```
#include "Plant.h"
```

```
#include <string>
```

Include dependency graph for Builder.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [Builder](#)

Abstract builder for constructing [Query](#) objects.

5.4.1 Detailed Description

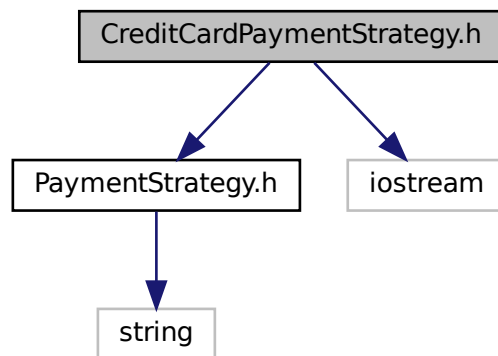
Defines the [Builder](#) interface for constructing [Query](#) objects.

The [Builder](#) class serves as the base for specific query builders such as [ItemQueryBuilder](#) and [MiscQueryBuilder](#). It provides the shared structure and interface used to assemble [Query](#) objects.

5.5 CreditCardPaymentStrategy.h File Reference

Concrete payment strategy using credit card transactions.

```
#include "PaymentStrategy.h"
#include <iostream>
Include dependency graph for CreditCardPaymentStrategy.h:
```



Classes

- class [CreditCardPaymentStrategy](#)
Payment strategy implementation using credit card information.

5.5.1 Detailed Description

Concrete payment strategy using credit card transactions.

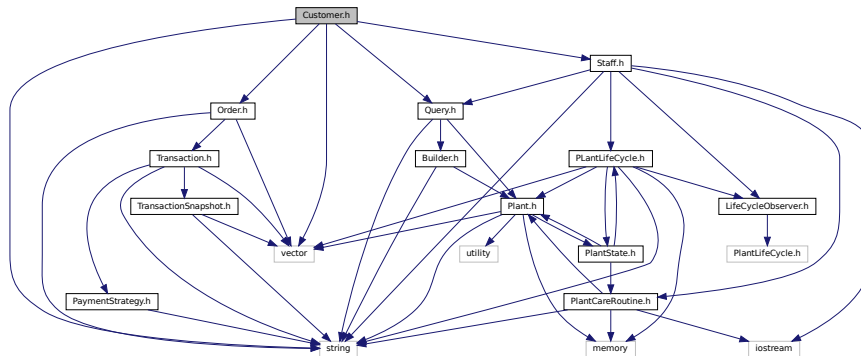
Implements the [PaymentStrategy](#) interface to process payments through a stored credit card number. Outputs confirmation or transaction details to the console.

5.6 Customer.h File Reference

Represents a [Customer](#) who can place orders and make staff queries.

```
#include <string>
#include <vector>
#include "Order.h"
#include "Staff.h"
#include "Query.h"
```

Include dependency graph for Customer.h:



Classes

- class [Customer](#)

Represents a store customer and their interactions.

5.6.1 Detailed Description

Represents a [Customer](#) who can place orders and make staff queries.

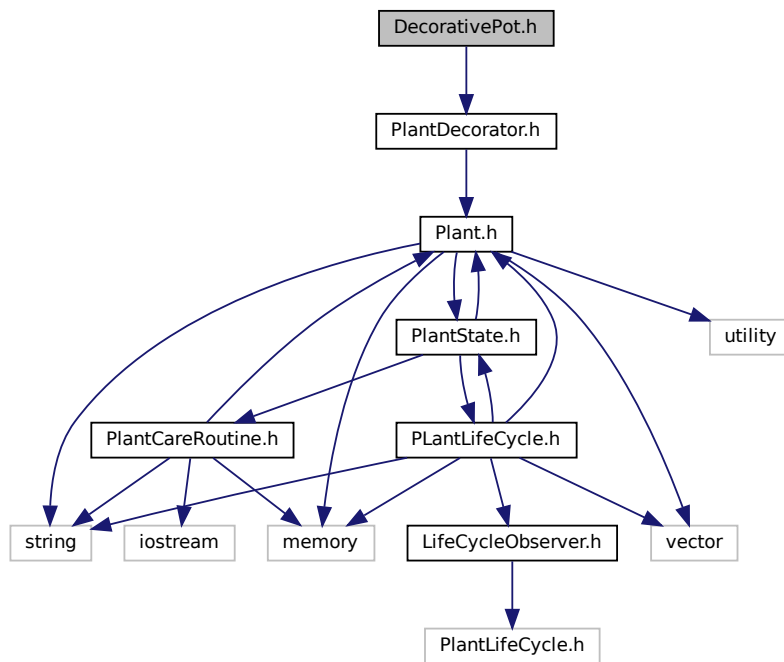
The [Customer](#) class models a retail customer with a name and a list of previous orders. Customers can interact with staff by submitting queries and can review their order history.

5.7 DecorativePot.h File Reference

Decorator class adding a decorative pot feature to a [Plant](#).

```
#include "PlantDecorator.h"
```

Include dependency graph for DecorativePot.h:



Classes

- class [DecorativePot](#)
Concrete decorator that adds a decorative pot to a [Plant](#).

5.7.1 Detailed Description

Decorator class adding a decorative pot feature to a [Plant](#).

[DecorativePot](#) is a concrete decorator extending [PlantDecorator](#). It enhances the description and price of a plant by adding a decorative pot.

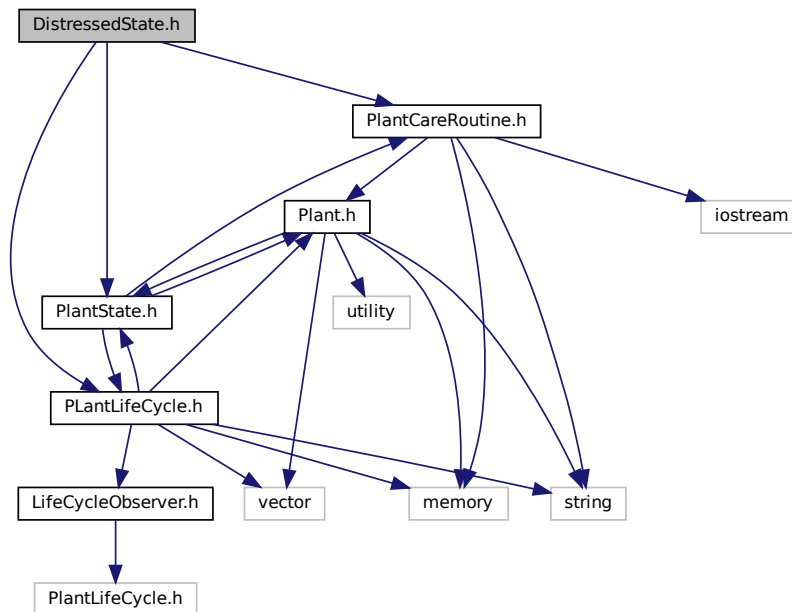
5.8 DistressedState.h File Reference

Represents the distressed state of a plant's life cycle.

```
#include "PlantCareRoutine.h"
#include "PLantLifeCycle.h"
```

```
#include "PlantState.h"
```

Include dependency graph for DistressedState.h:



Classes

- class [DistressedState](#)
Concrete state representing a plant in distress.

5.8.1 Detailed Description

Represents the distressed state of a plant's life cycle.

The [DistressedState](#) class models a condition where a plant is struggling due to insufficient care or environmental factors. With proper care, it may recover and transition to a healthier state.

See also

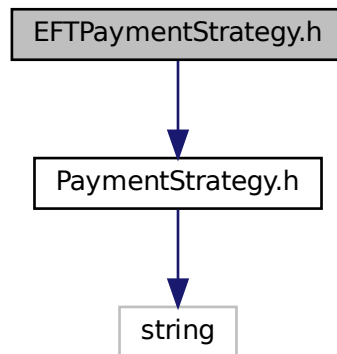
[PlantState](#)
[PlantLifeCycle](#)
[PlantCareRoutine](#)

5.9 EFTPaymentStrategy.h File Reference

Concrete payment strategy using Electronic Funds Transfer (EFT).

```
#include "PaymentStrategy.h"
```

Include dependency graph for EFTPaymentStrategy.h:



Classes

- class [EFTPaymentStrategy](#)
Payment strategy implementation for EFT transactions.

5.9.1 Detailed Description

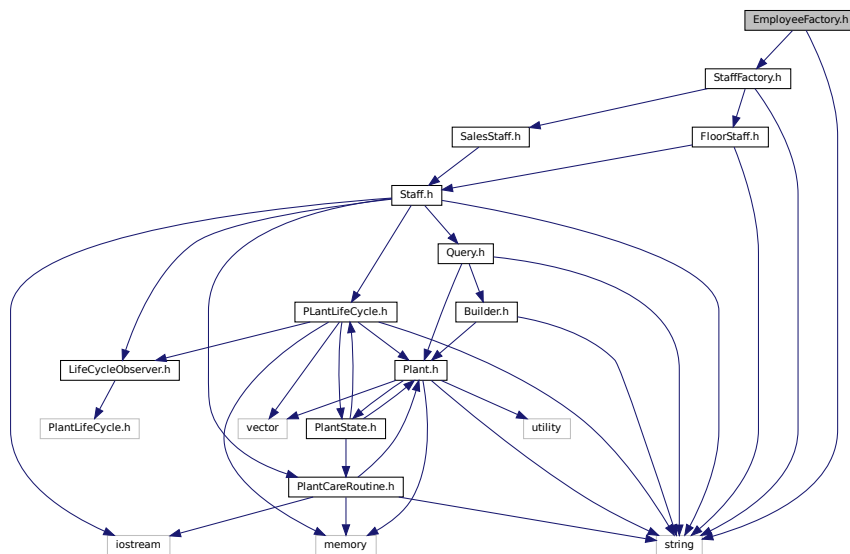
Concrete payment strategy using Electronic Funds Transfer (EFT).

Implements the [PaymentStrategy](#) interface for EFT-based payments. Stores an account number and handles electronic transfers.

5.10 EmployeeFactory.h File Reference

Concrete factory for creating regular employees.

```
#include "StaffFactory.h"
#include <string>
Include dependency graph for EmployeeFactory.h:
```



Classes

- class [EmployeeFactory](#)
Factory responsible for creating regular staff members.

5.10.1 Detailed Description

Concrete factory for creating regular employees.

Implements the [StaffFactory](#) interface to create instances of [FloorEmployee](#) and [SalesEmployee](#).

Author

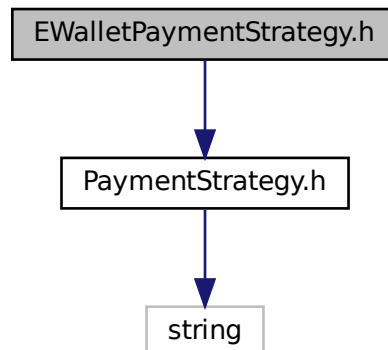
Okaile Gaesale/Cobus Botha/Jay Lopes

5.11 EWalletPaymentStrategy.h File Reference

Concrete payment strategy using a digital e-wallet.

```
#include "PaymentStrategy.h"
```

Include dependency graph for EWalletPaymentStrategy.h:



Classes

- class [EWalletPaymentStrategy](#)
Implements a payment method using an e-wallet ID.

5.11.1 Detailed Description

Concrete payment strategy using a digital e-wallet.

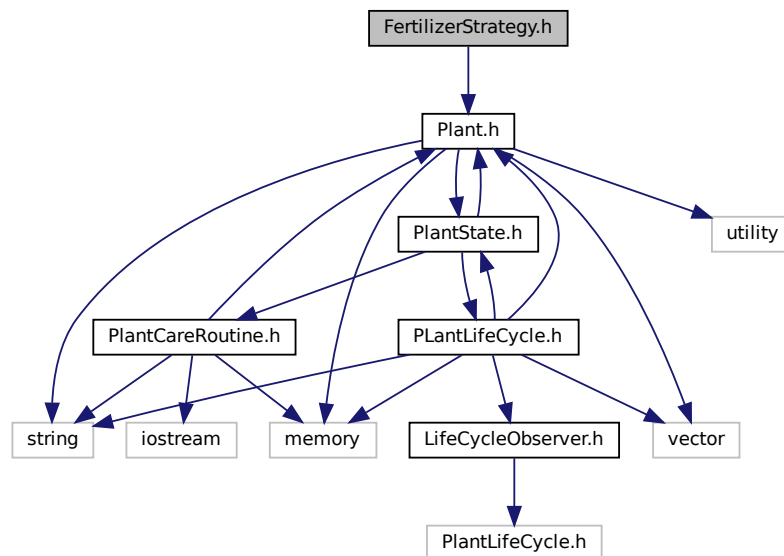
Implements the [PaymentStrategy](#) interface for processing e-wallet payments through a stored wallet ID (e.g., PayPal, SnapScan, etc.).

5.12 FertilizerStrategy.h File Reference

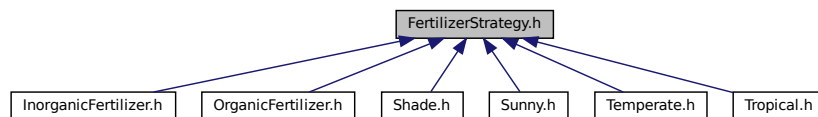
Defines the abstract interface for plant fertilizer strategies.

```
#include "Plant.h"
```

Include dependency graph for FertilizerStrategy.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [FertilizerStrategy](#)
Abstract base for fertilizer behavior strategies.

5.12.1 Detailed Description

Defines the abstract interface for plant fertilizer strategies.

This interface allows for different fertilizer strategies to be applied polymorphically to various plant types.

5.13 FloorEmployee.h File Reference

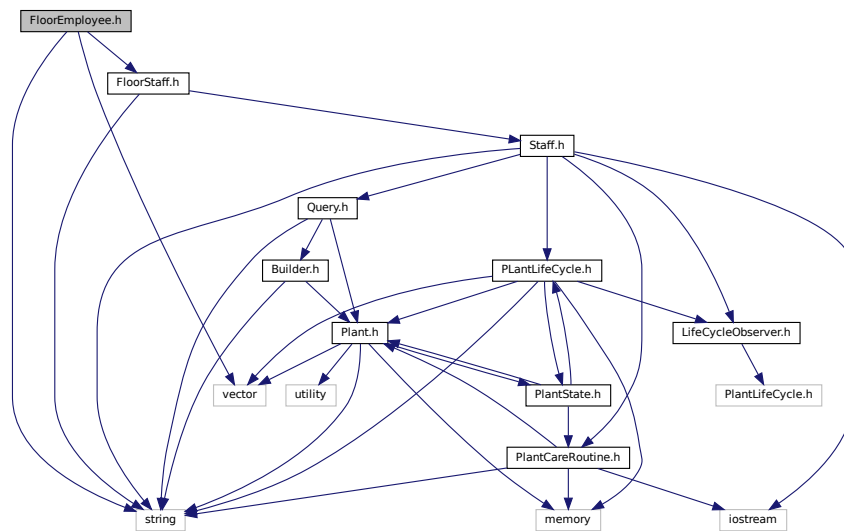
Concrete class representing a floor employee.

```
#include "FloorStaff.h"
```

```
#include <vector>
```

```
#include <string>
```

Include dependency graph for FloorEmployee.h:



Classes

- class [FloorEmployee](#)
Regular employee handling on-floor customer queries.

5.13.1 Detailed Description

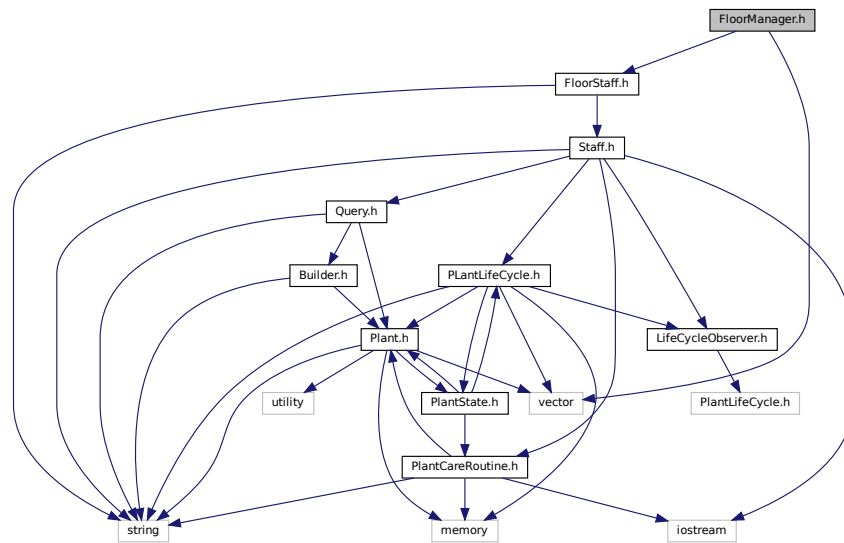
Concrete class representing a floor employee.

Handles basic customer interactions such as providing plant care information and general inquiries.

5.14 FloorManager.h File Reference

Class representing the manager of floor operations.

```
#include "FloorStaff.h"
#include <vector>
Include dependency graph for FloorManager.h:
```



Classes

- class [FloorManager](#)
Manages floor staff and handles high-level queries.

5.14.1 Detailed Description

Class representing the manager of floor operations.

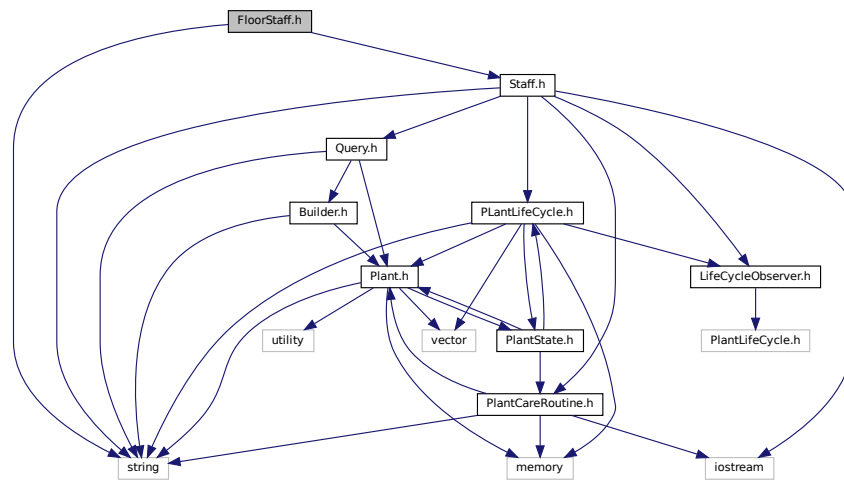
Handles advanced floor-related queries requiring managerial authority, such as stock and inventory control.

5.15 FloorStaff.h File Reference

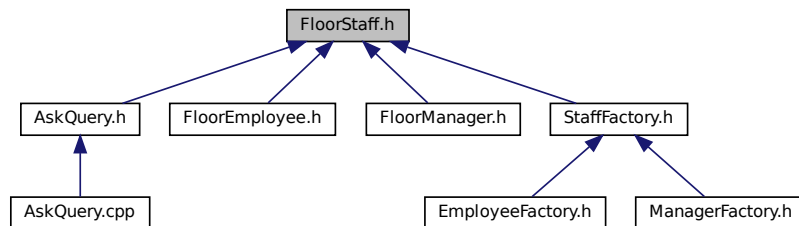
Abstract base class for floor-level staff members.

```
#include "Staff.h"
#include <string>
```

Include dependency graph for FloorStaff.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [FloorStaff](#)

Base class for all floor staff types.

5.15.1 Detailed Description

Abstract base class for floor-level staff members.

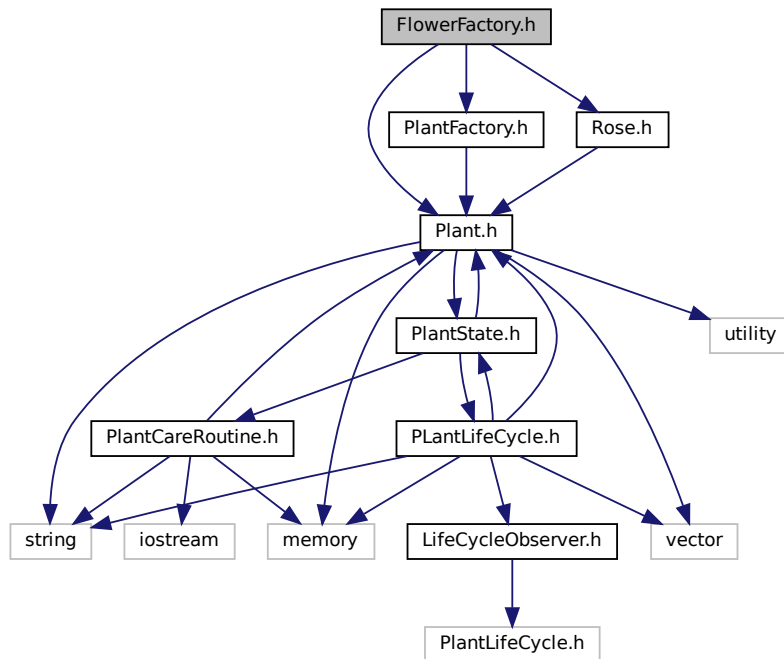
Serves as the base for [FloorEmployee](#) and [FloorManager](#) classes, implementing shared behavior and chain-of-responsibility forwarding.

5.16 FlowerFactory.h File Reference

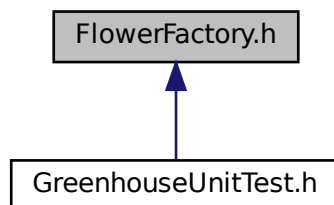
Factory for creating flower-type [Plant](#) objects.

```
#include "PlantFactory.h"
#include "Plant.h"
#include "Rose.h"
```

Include dependency graph for FlowerFactory.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [FlowerFactory](#)
Concrete factory for creating flower-type *Plants*.

5.16.1 Detailed Description

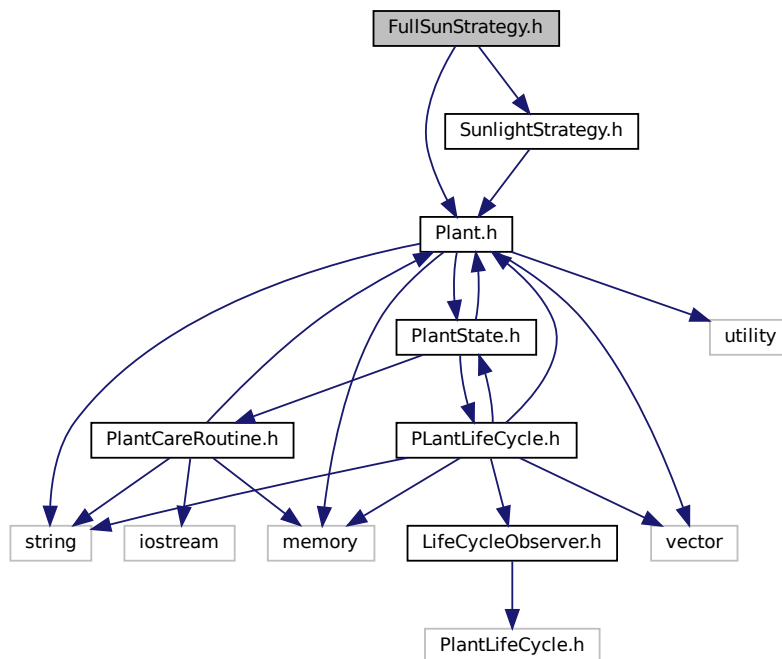
Factory for creating flower-type [Plant](#) objects.

The [FlowerFactory](#) is a concrete implementation of [PlantFactory](#) that constructs various flower-based [Plant](#) objects (e.g., [Roses](#)).

5.17 FullSunStrategy.h File Reference

Concrete sunlight strategy applying full sunlight exposure to a plant.

```
#include "Plant.h"
#include "SunlightStrategy.h"
Include dependency graph for FullSunStrategy.h:
```



Classes

- class [FullSunStrategy](#)
Strategy for applying full sunlight to a [Plant](#).

5.17.1 Detailed Description

Concrete sunlight strategy applying full sunlight exposure to a plant.

Implements the [SunlightStrategy](#) interface to provide maximum sunlight exposure to a plant. Increases sunlight level by 60% per day, capped at the plant's maximum allowed sunlight.

Note

Helps simulate sun-loving plants that thrive in full light.

Warning

Exceeding the maximum sunlight value resets it to the limit.

See also

[SunlightStrategy](#)

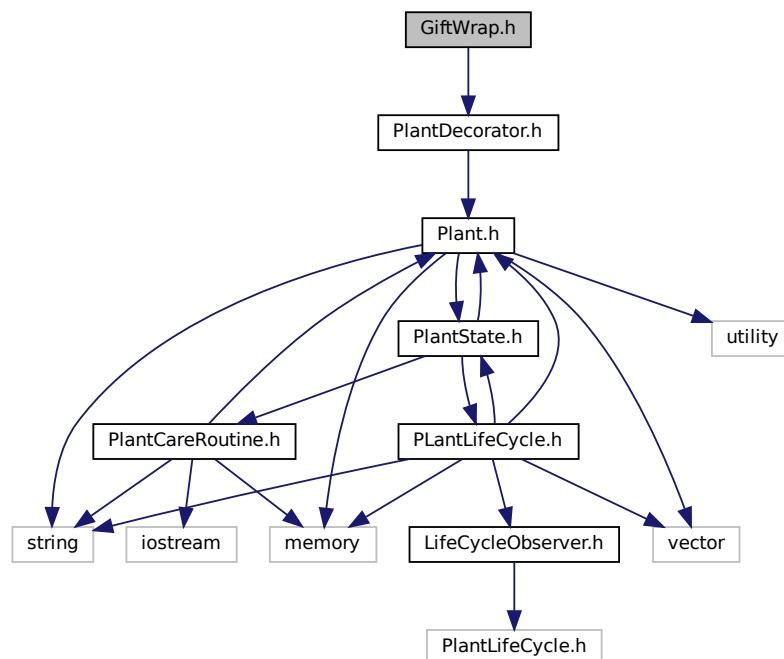
[Plant](#)

5.18 GiftWrap.h File Reference

Decorator class adding a gift wrapping feature to a [Plant](#).

```
#include "PlantDecorator.h"
```

Include dependency graph for GiftWrap.h:



Classes

- class [GiftWrap](#)

Concrete decorator that adds gift wrapping to a [Plant](#).

5.18.1 Detailed Description

Decorator class adding a gift wrapping feature to a [Plant](#).

[GiftWrap](#) is a concrete decorator extending [PlantDecorator](#). It modifies the description and price of a plant by adding a gift wrap option.

See also

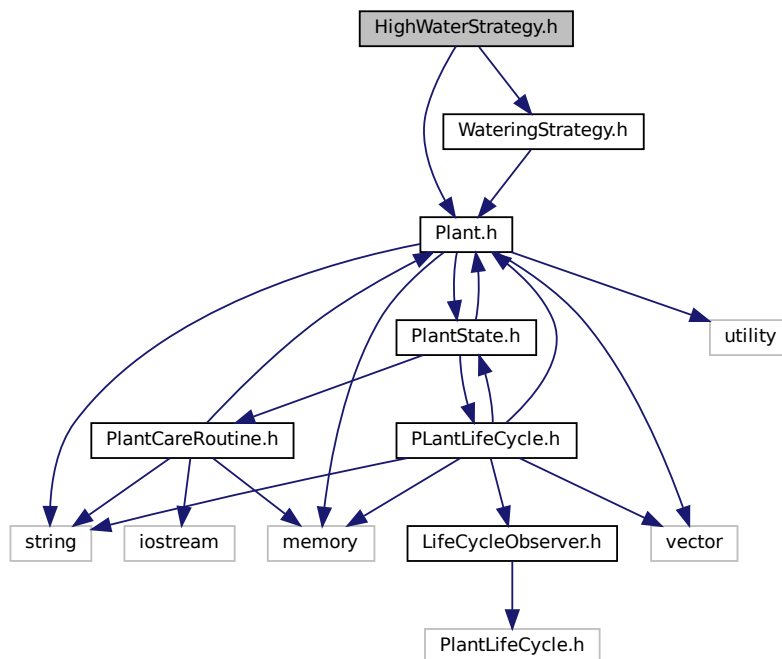
[PlantDecorator](#)

[Plant](#)

5.19 HighWaterStrategy.h File Reference

Concrete watering strategy that applies high water levels to a plant.

```
#include "Plant.h"
#include "WateringStrategy.h"
Include dependency graph for HighWaterStrategy.h:
```



Classes

- class [HighWaterStrategy](#)
Strategy for applying a high amount of water to a plant.

5.19.1 Detailed Description

Concrete watering strategy that applies high water levels to a plant.

[HighWaterStrategy](#) implements the [WaterStrategy](#) interface and provides plants with a large amount of water during watering operations.

See also

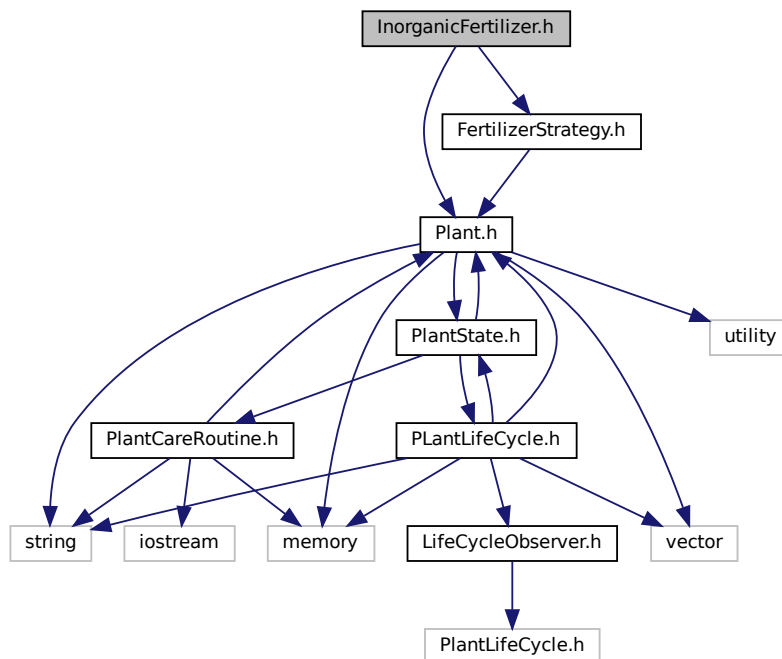
[WaterStrategy](#)

[Plant](#)

5.20 InorganicFertilizer.h File Reference

Concrete strategy for applying inorganic fertilizer to plants.

```
#include "Plant.h"
#include "FertilizerStrategy.h"
Include dependency graph for InorganicFertilizer.h:
```



Classes

- class [InorganicFertilizer](#)
Strategy for applying inorganic fertilizers to a plant.

5.20.1 Detailed Description

Concrete strategy for applying inorganic fertilizer to plants.

[InorganicFertilizer](#) implements the [FertilizerStrategy](#) interface to provide a fertilization strategy using synthetic, chemical-based nutrients. This strategy provides quick nutrient release for rapid plant growth.

See also

[FertilizerStrategy](#)

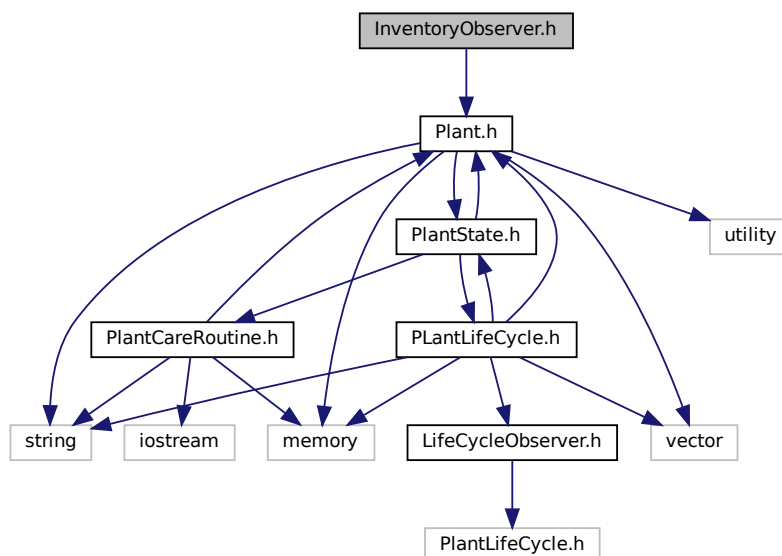
[Plant](#)

5.21 InventoryObserver.h File Reference

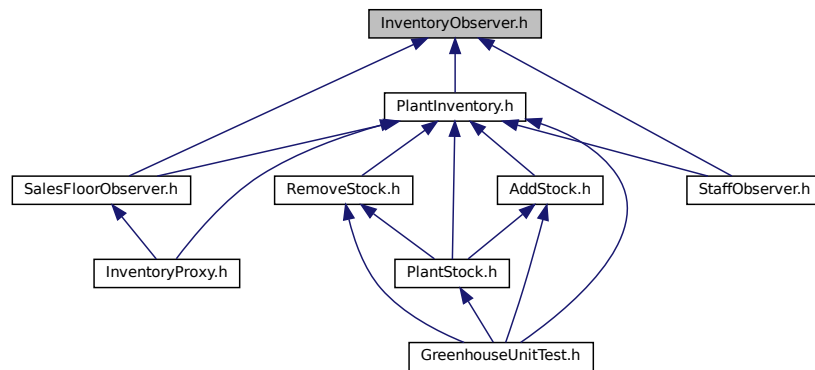
Abstract base class for observers monitoring plant inventory changes.

```
#include "Plant.h"
```

Include dependency graph for InventoryObserver.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [InventoryObserver](#)
Abstract base class for observers of inventory changes.

5.21.1 Detailed Description

Abstract base class for observers monitoring plant inventory changes.

[InventoryObserver](#) defines the interface for objects that need to be notified when changes occur in the plant inventory. Concrete observers must implement the update method to respond to inventory changes.

See also

[PlantInventory](#)

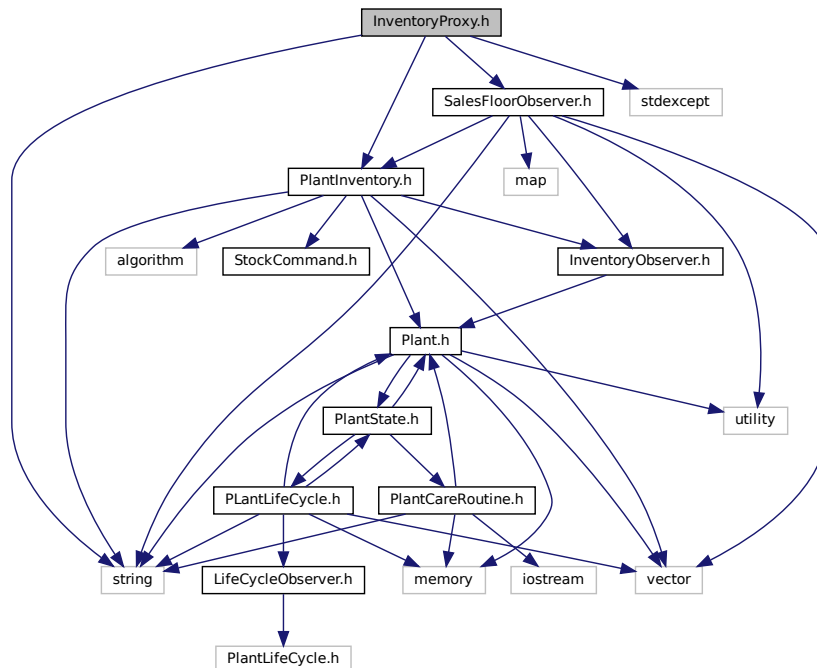
5.22 InventoryProxy.h File Reference

Proxy class controlling access to plant inventory operations.

```
#include "PlantInventory.h"
#include "SalesFloorObserver.h"
#include <stdexcept>
```

```
#include <string>
```

Include dependency graph for InventoryProxy.h:



Classes

- class [InventoryProxy](#)
Acts as a proxy to manage plant inventory transactions.

5.22.1 Detailed Description

Proxy class controlling access to plant inventory operations.

[InventoryProxy](#) provides controlled access to [PlantInventory](#) functionality, acting as a protection proxy that can enforce business rules and access control. It coordinates with [SalesFloorObserver](#) to maintain consistency between inventory and sales floor displays.

See also

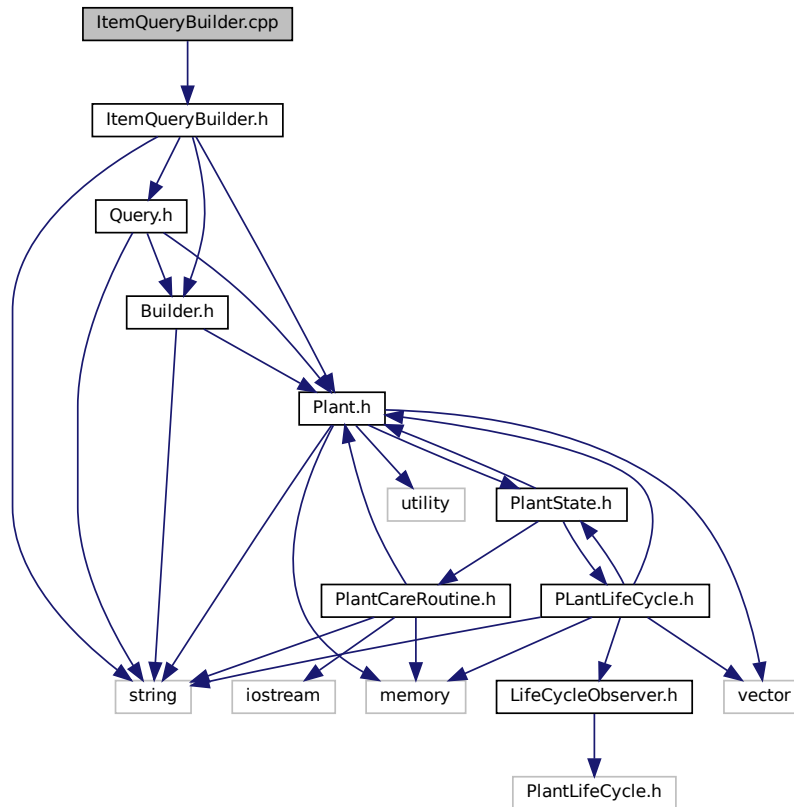
[PlantInventory](#)
[SalesFloorObserver](#)

5.23 ItemQueryBuilder.cpp File Reference

Implementation of [ItemQueryBuilder](#) setters.

```
#include "ItemQueryBuilder.h"
```

Include dependency graph for ItemQueryBuilder.cpp:



5.23.1 Detailed Description

Implementation of [ItemQueryBuilder](#) setters.

These methods configure the builder state (item, type, question) which is later consumed by [Builder::build\(\)](#)/Query constructor.

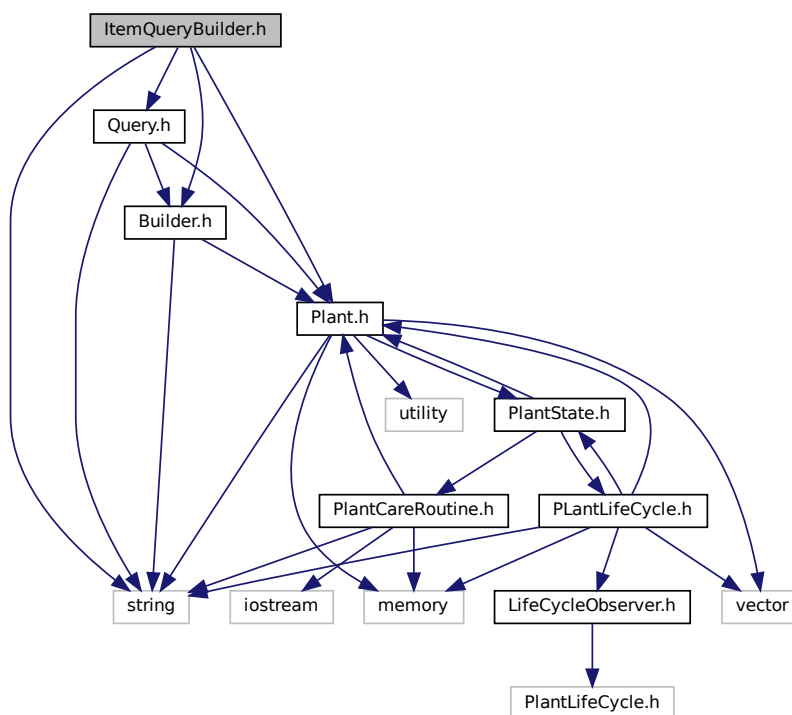
5.24 ItemQueryBuilder.h File Reference

Concrete builder for creating item-specific [Query](#) objects.

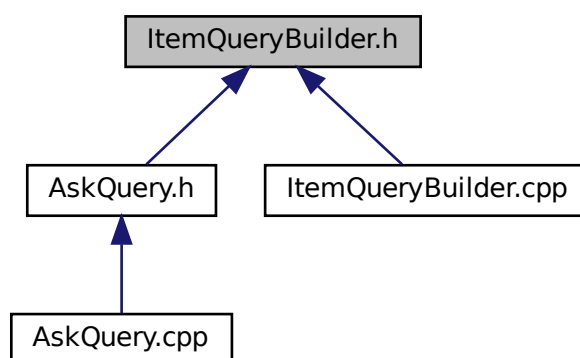
```
#include "Plant.h"
#include "Builder.h"
#include "Query.h"
```

```
#include <string>
```

Include dependency graph for ItemQueryBuilder.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [ItemQueryBuilder](#)
Builder class for constructing queries about specific plants.

5.24.1 Detailed Description

Concrete builder for creating item-specific [Query](#) objects.

[ItemQueryBuilder](#) constructs [Query](#) instances that reference a specific [Plant](#) item, such as information or care routine queries.

See also

[Builder](#)

[Query](#)

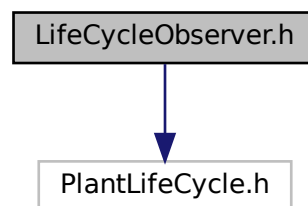
[Plant](#)

5.25 LifeCycleObserver.h File Reference

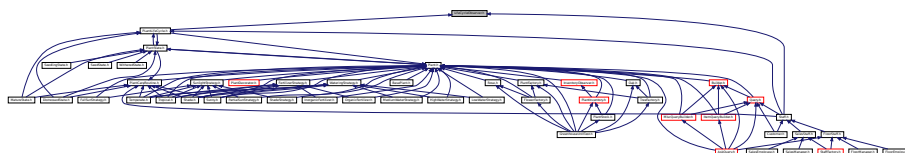
Abstract base class for observers monitoring plant life cycle changes.

```
#include "PlantLifeCycle.h"
```

Include dependency graph for LifeCycleObserver.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [LifeCycleObserver](#)
Observer interface for plant life cycle changes.

5.25.1 Detailed Description

Abstract base class for observers monitoring plant life cycle changes.

[LifeCycleObserver](#) defines the interface for objects that need to be notified when changes occur in a plant's life cycle. Observers can track growth stages, health changes, and other life cycle events.

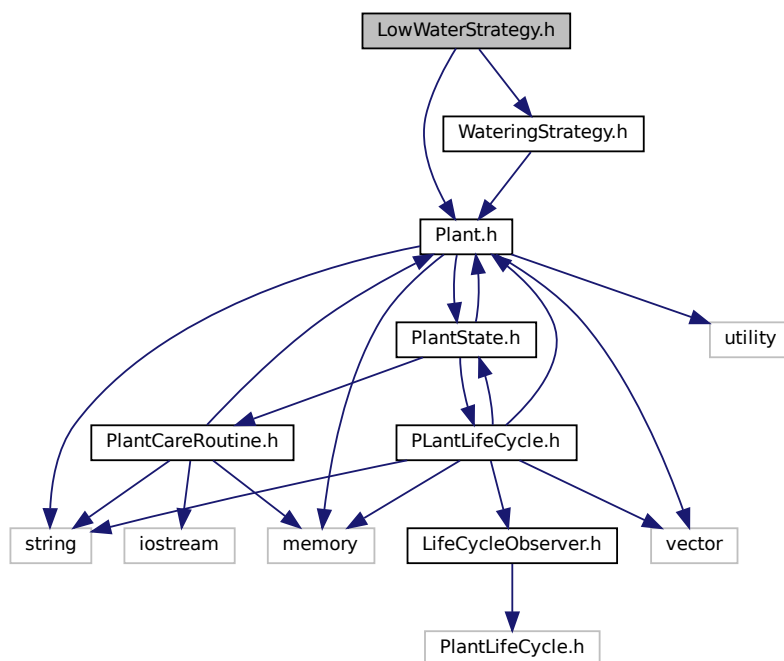
See also

[PlantLifeCycle](#)

5.26 LowWaterStrategy.h File Reference

Defines the [LowWaterStrategy](#) class used for minimal watering.

```
#include "Plant.h"
#include "WateringStrategy.h"
Include dependency graph for LowWaterStrategy.h:
```



Classes

- class [LowWaterStrategy](#)
Strategy for plants requiring minimal water intake.

5.26.1 Detailed Description

Defines the [LowWaterStrategy](#) class used for minimal watering.

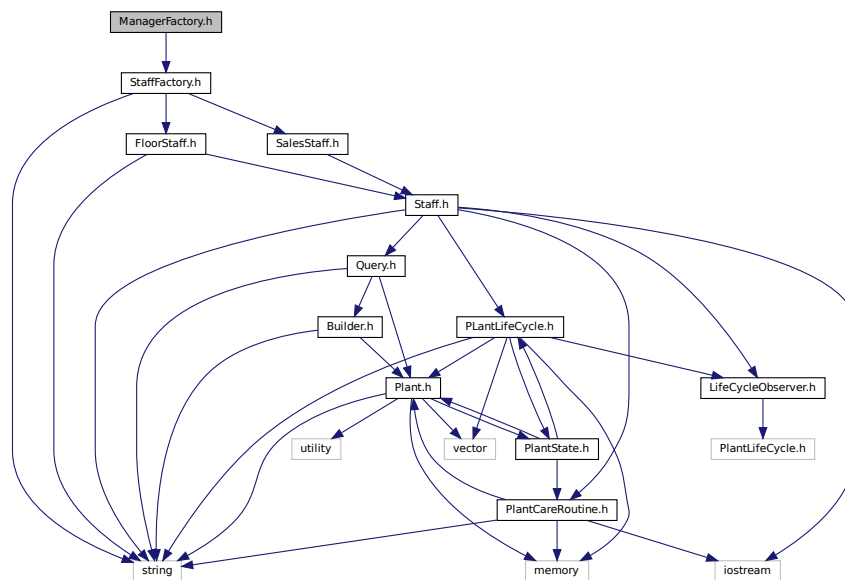
This strategy increases a plant's water level by a small, controlled amount, ensuring it doesn't exceed its maximum capacity.

5.27 ManagerFactory.h File Reference

Defines the factory for creating manager-level staff.

```
#include "StaffFactory.h"
```

Include dependency graph for ManagerFactory.h:



Classes

- class [Managerfactory](#)

5.27.1 Detailed Description

Defines the factory for creating manager-level staff.

The [ManagerFactory](#) is responsible for instantiating [FloorManager](#) and [SalesManager](#) objects using the [StaffFactory](#) interface.

5.28 MatureState.h File Reference

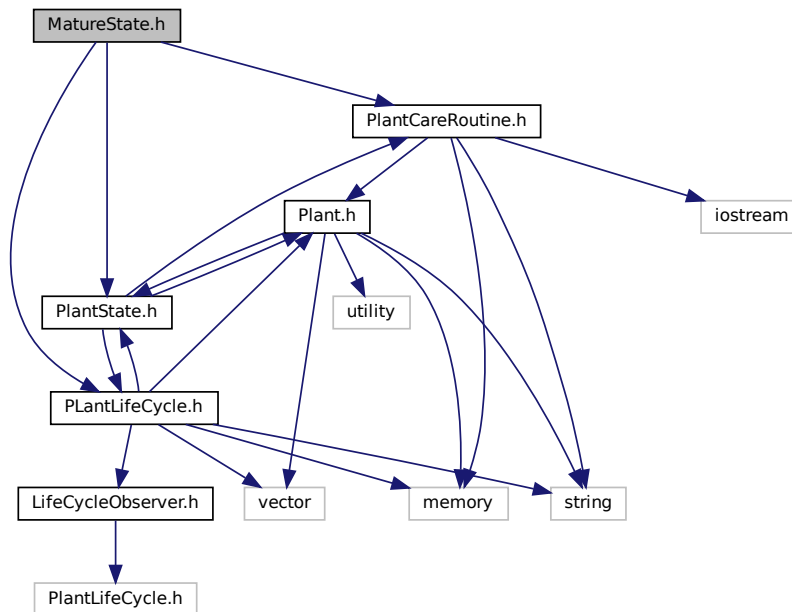
Defines the [MatureState](#) class representing a plant's healthy phase.

```
#include "PlantCareRoutine.h"
```

```
#include "PLantLifeCycle.h"
```

```
#include "PlantState.h"
```

Include dependency graph for MatureState.h:



Classes

- class [MatureState](#)

Represents a thriving stage in the plant's life cycle.

5.28.1 Detailed Description

Defines the [MatureState](#) class representing a plant's healthy phase.

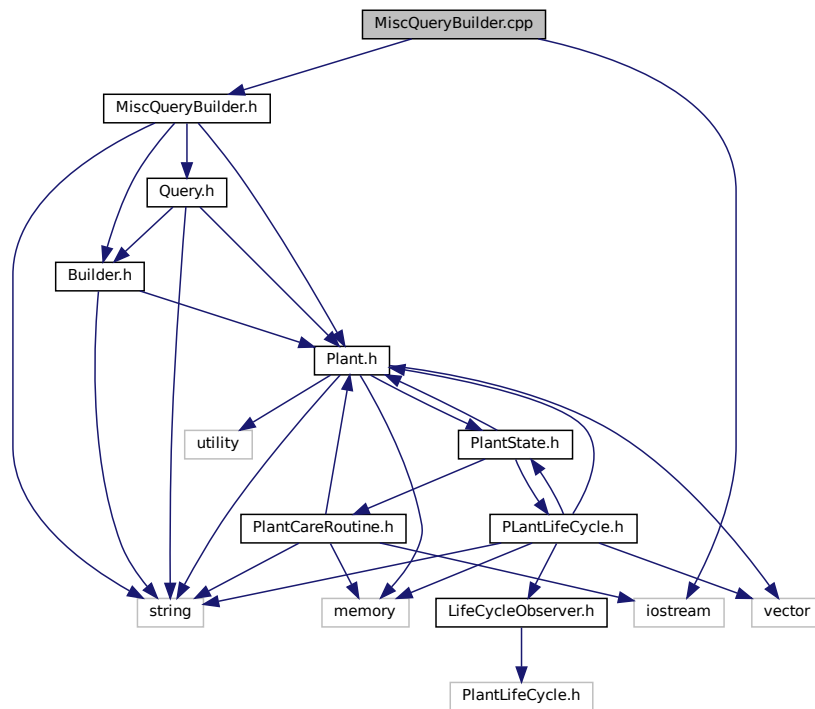
In this state, the plant thrives under regular care and sustenance. If its resources fall below minimum thresholds, it transitions to [DistressedState](#).

5.30 MiscQueryBuilder.cpp File Reference

Implementation of [MiscQueryBuilder](#) methods.

```
#include "MiscQueryBuilder.h"
#include <iostream>
```

Include dependency graph for MiscQueryBuilder.cpp:



5.30.1 Detailed Description

Implementation of [MiscQueryBuilder](#) methods.

Implements setters for the type/question. setItem is intentionally a no-op because miscellaneous queries do not target a specific [Plant](#).

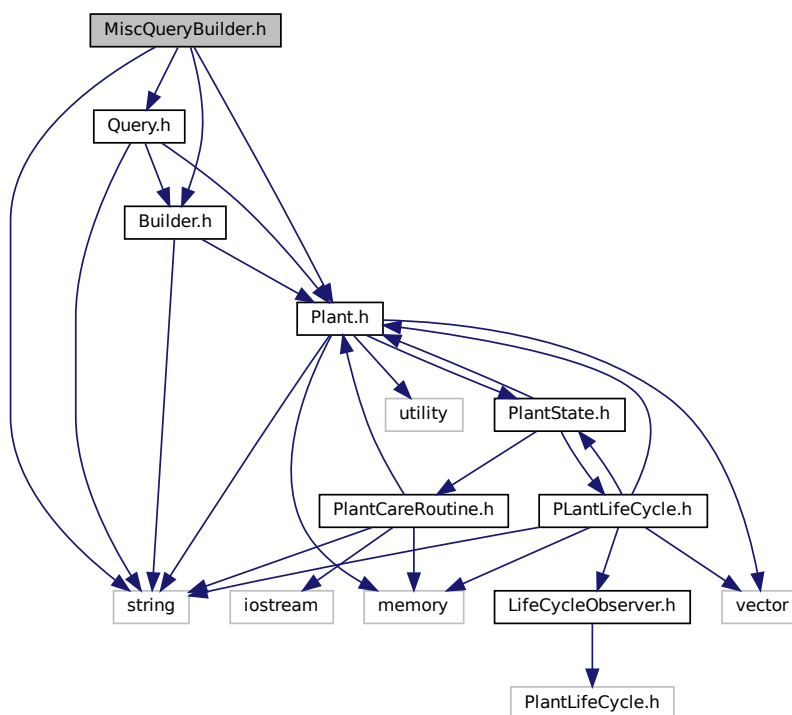
5.31 MiscQueryBuilder.h File Reference

[Builder](#) for creating general (non-item-specific) queries.

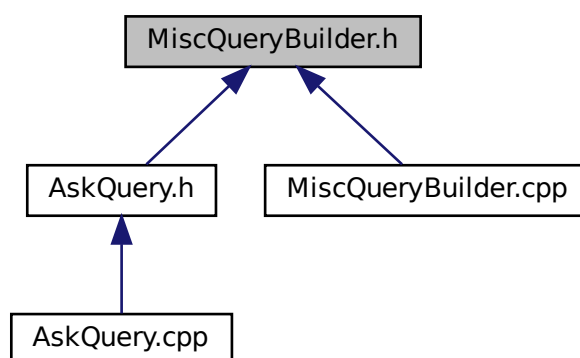
```
#include "Plant.h"
#include "Builder.h"
#include "Query.h"
```

```
#include <string>
```

Include dependency graph for MiscQueryBuilder.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [MiscQueryBuilder](#)
Concrete builder for non-item queries.

Classes

- class [Oak](#)

Represents an [Oak](#) tree plant.

5.32.1 Detailed Description

Concrete [Plant](#) class representing an [Oak](#) tree.

The [Oak](#) class defines a specific type of [Plant](#) characterized as an [Oak](#) tree. It inherits from the [Plant](#) base class and provides descriptive attributes relevant to this plant species.

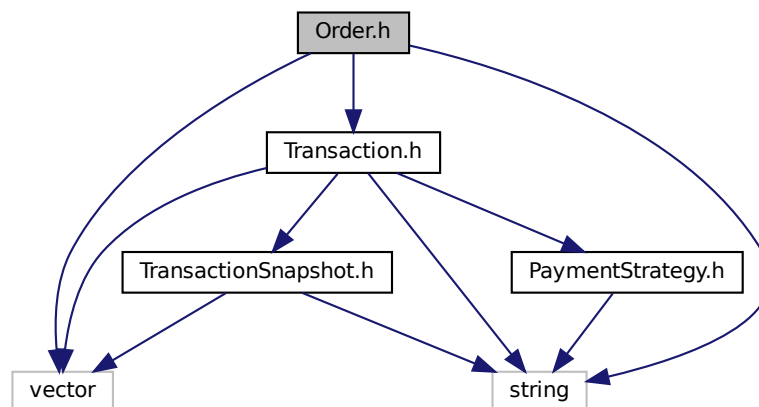
See also

[Plant](#)

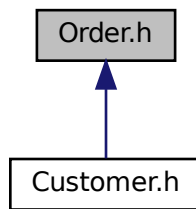
5.33 Order.h File Reference

Represents a customer's order containing multiple transactions.

```
#include <string>
#include <vector>
#include "Transaction.h"
Include dependency graph for Order.h:
```



This graph shows which files directly or indirectly include this file:



Classes

- class [Order](#)
Represents a customer's purchase order.

5.33.1 Detailed Description

Represents a customer's order containing multiple transactions.

The [Order](#) class tracks the status, transactions, and associated customer for each order placed within the system.

See also

[Transaction](#)

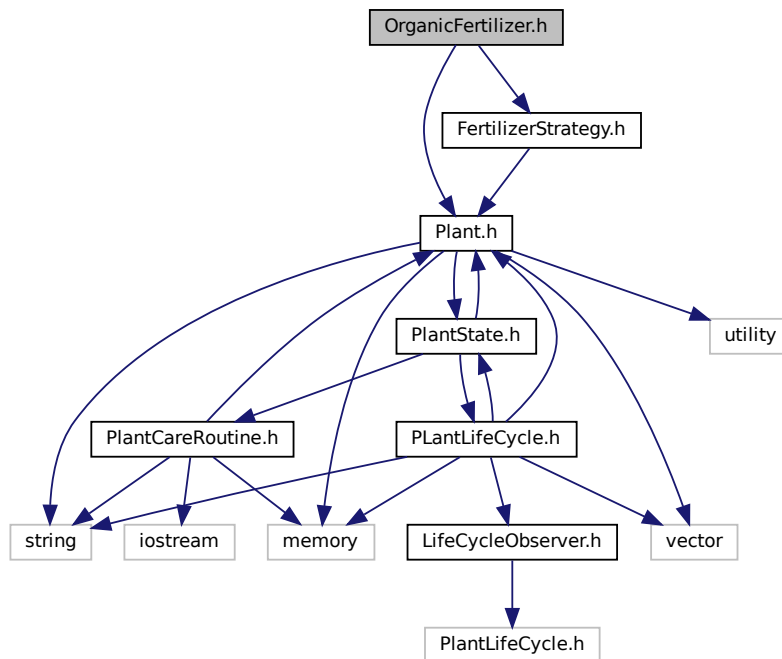
[Customer](#)

5.34 OrganicFertilizer.h File Reference

Concrete fertilizer strategy using organic nutrients.

```
#include "Plant.h"  
#include "FertilizerStrategy.h"
```

Include dependency graph for OrganicFertilizer.h:



Classes

- class [OrganicFertilizer](#)
Strategy for applying organic fertilizer to a plant.

5.34.1 Detailed Description

Concrete fertilizer strategy using organic nutrients.

[OrganicFertilizer](#) increases a plant's nutrient level naturally using organic compounds. It implements the [FertilizerStrategy](#) interface.

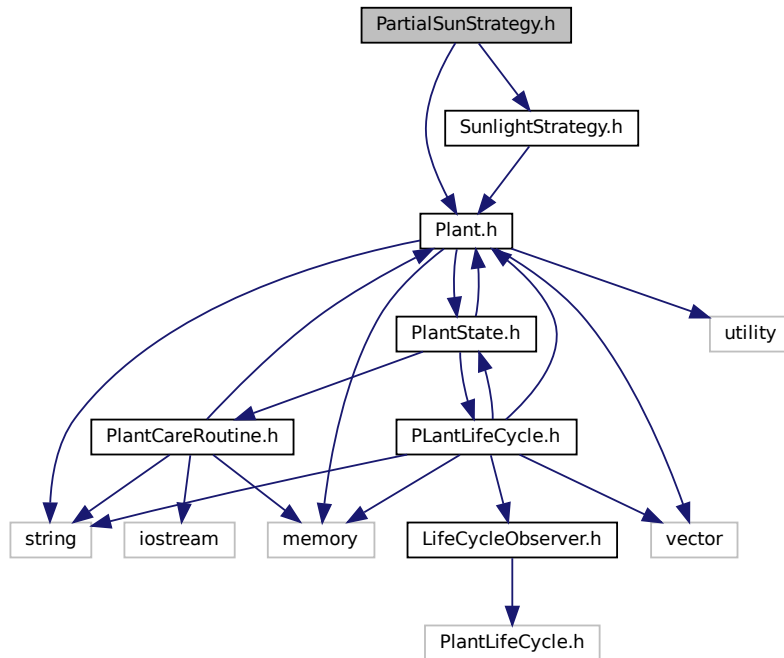
See also

[FertilizerStrategy](#)
[Plant](#)

5.35 PartialSunStrategy.h File Reference

Concrete sunlight strategy providing moderate sunlight exposure.

```
#include "Plant.h"
#include "SunlightStrategy.h"
Include dependency graph for PartialSunStrategy.h:
```



Classes

- class [PartialSunStrategy](#)
Strategy for applying partial sunlight to a plant.

5.35.1 Detailed Description

Concrete sunlight strategy providing moderate sunlight exposure.

[PartialSunStrategy](#) increases a plant's sunlight exposure moderately, suitable for plants that thrive in partial sun conditions.

See also

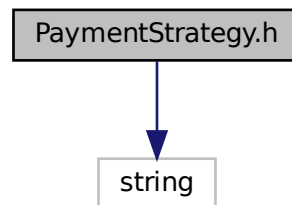
[SunlightStrategy](#)
[Plant](#)

5.36 PaymentStrategy.h File Reference

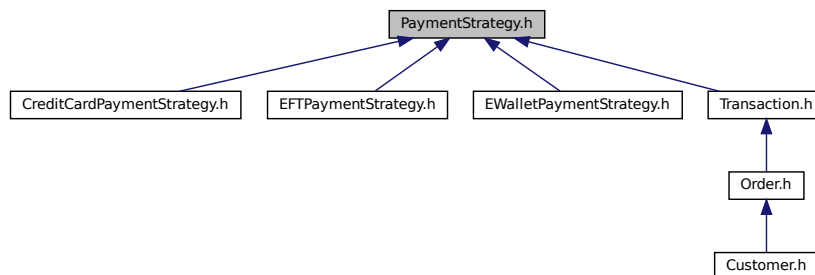
Abstract interface for various payment strategies.

```
#include <string>
```

Include dependency graph for PaymentStrategy.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [PaymentStrategy](#)
Abstract strategy for processing payments.

5.36.1 Detailed Description

Abstract interface for various payment strategies.

[PaymentStrategy](#) defines the interface for all payment mechanisms, allowing flexibility through polymorphism (Strategy Pattern).

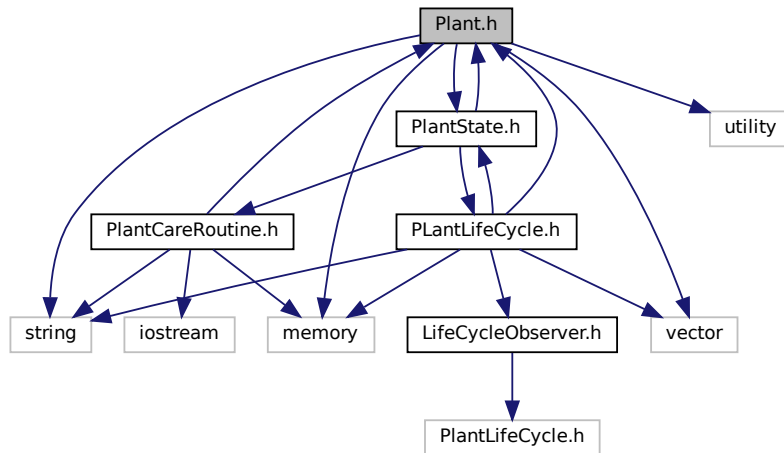
See also

[EFTPaymentStrategy](#)
[EWalletPaymentStrategy](#)

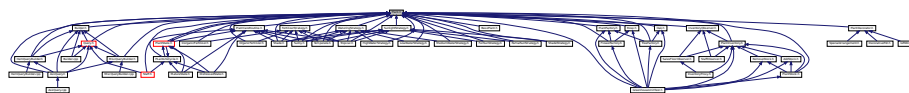
5.37 Plant.h File Reference

Abstract base class representing a plant in the nursery system.

```
#include "PlantState.h"
#include <string>
#include <vector>
#include <utility>
#include <memory>
Include dependency graph for Plant.h:
```



This graph shows which files directly or indirectly include this file:



Classes

- class [Plant](#)

Abstract representation of a plant with care and lifecycle management.

5.37.1 Detailed Description

Abstract base class representing a plant in the nursery system.

The [Plant](#) class models a plant's properties, lifecycle state, and care strategies. It serves as the core of the system, implementing key design patterns such as State and Observer for managing growth and life cycle notifications.

See also

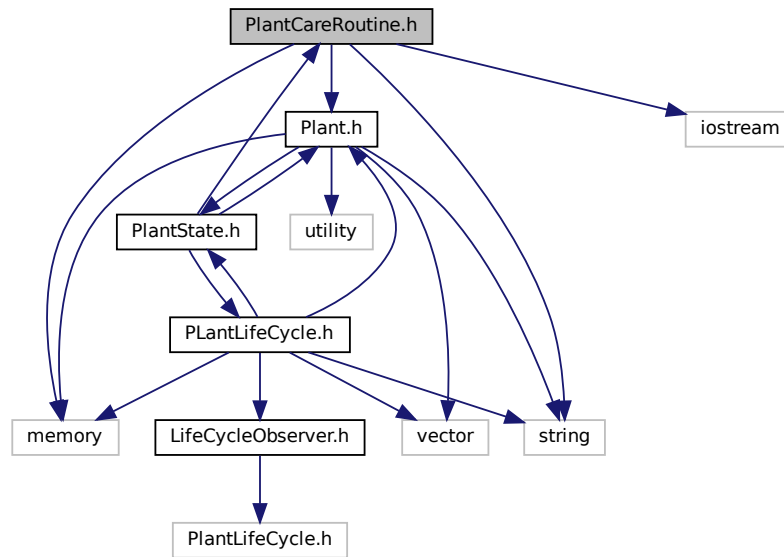
[PlantState](#)
[WateringStrategy](#)
[SunlightStrategy](#)
[FertilizerStrategy](#)
[LifeCycleObserver](#)

5.38 PlantCareRoutine.h File Reference

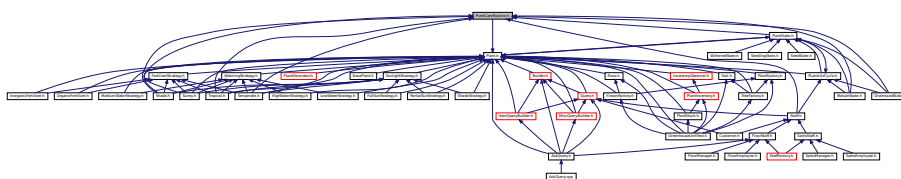
Defines the abstract interface for plant care routines.

```
#include <string>
#include "Plant.h"
#include <iostream>
#include <memory>
```

Include dependency graph for PlantCareRoutine.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [PlantCareRoutine](#)
Abstract base class defining a plant care routine interface.

5.38.1 Detailed Description

Defines the abstract interface for plant care routines.

The [PlantCareRoutine](#) class provides a template for applying care operations (watering, fertilizing, and sunlight exposure) to plants. It supports dynamic creation of specific care routines.

See also

[Plant](#)

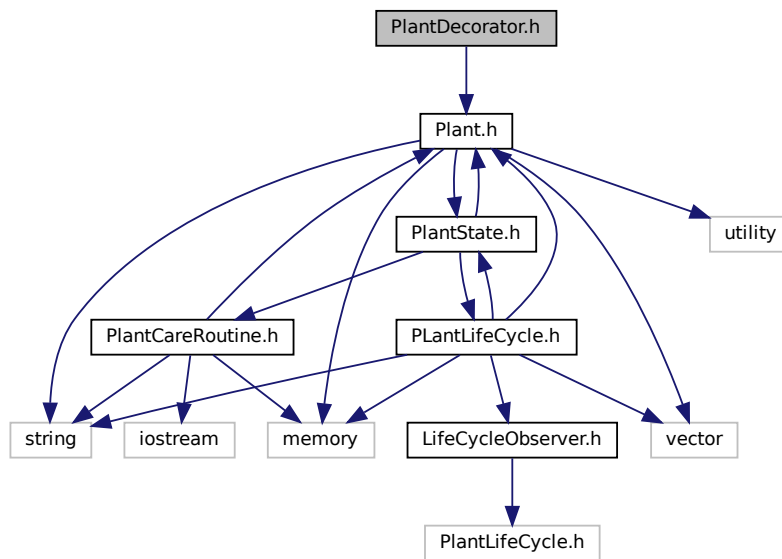
[PlantLifeCycle](#)

5.39 PlantDecorator.h File Reference

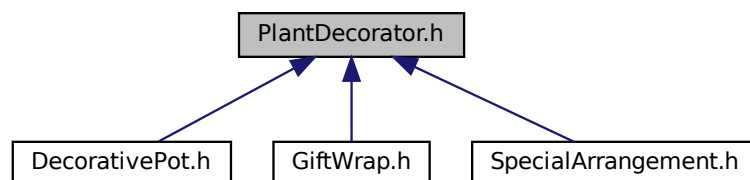
Abstract decorator for extending [Plant](#) behavior dynamically.

```
#include "Plant.h"
```

Include dependency graph for PlantDecorator.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [PlantDecorator](#)

Abstract decorator for dynamically extending [Plant](#) behavior.

5.39.1 Detailed Description

Abstract decorator for extending [Plant](#) behavior dynamically.

This class implements the Decorator pattern, allowing plant objects to be extended with new features (e.g., pots, wrapping) without modifying the original [Plant](#) class.

See also

[Plant](#)

[DecorativePot](#)

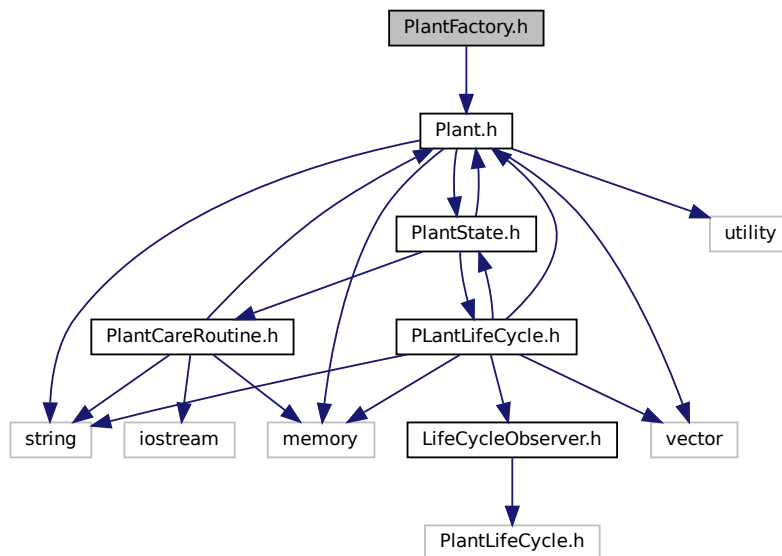
[GiftWrap](#)

5.40 PlantFactory.h File Reference

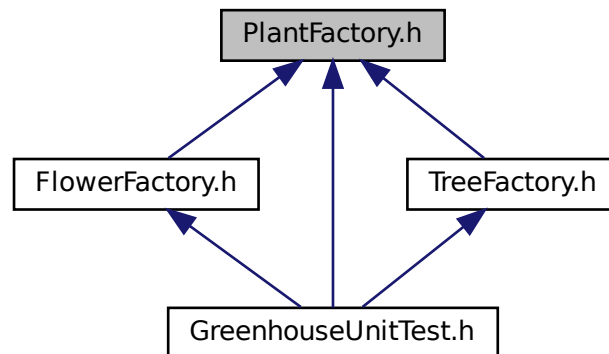
Abstract factory interface for creating [Plant](#) objects.

```
#include "Plant.h"
```

Include dependency graph for PlantFactory.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [PlantFactory](#)
Abstract base class for plant creation factories.

5.40.1 Detailed Description

Abstract factory interface for creating [Plant](#) objects.

The [PlantFactory](#) defines a consistent interface for creating different types of plant objects. Concrete subclasses (e.g., [FlowerFactory](#)) must implement the creation logic.

See also

[FlowerFactory](#)
[Plant](#)

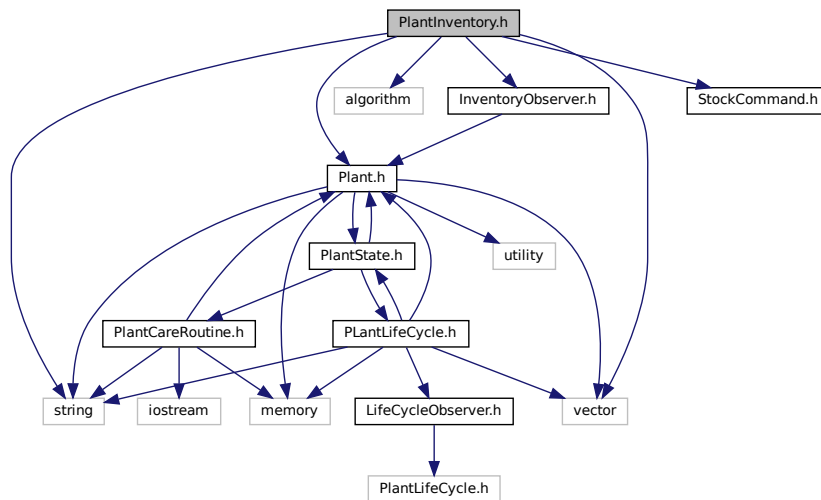
5.41 PlantInventory.h File Reference

Manages plant inventory using Singleton and Observer patterns.

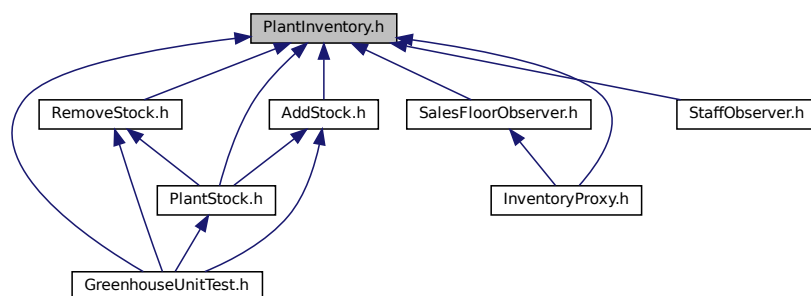
```
#include <vector>
#include <string>
#include <algorithm>
#include "InventoryObserver.h"
#include "Plant.h"
```

```
#include "StockCommand.h"
```

Include dependency graph for PlantInventory.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [PlantInventory](#)
Singleton class managing plant inventory and observers.

5.41.1 Detailed Description

Manages plant inventory using Singleton and Observer patterns.

The [PlantInventory](#) class maintains a record of all plants in stock. It supports attaching observers and executing commands that modify stock quantities. Implements the Singleton pattern for shared access.

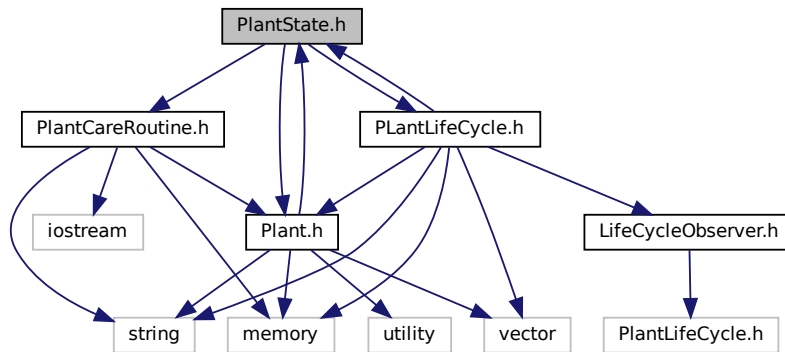
See also

[InventoryObserver](#)
[PlantStock](#)
[StockCommand](#)

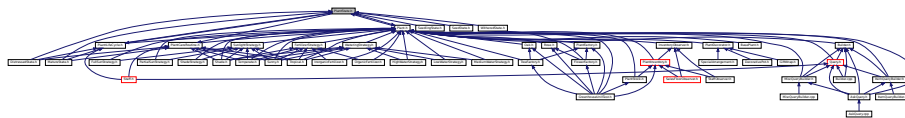
5.42 PlantState.h File Reference

Defines the abstract interface for plant life cycle states.

```
#include "Plant.h"
#include "PlantCareRoutine.h"
#include "PLantLifeCycle.h"
Include dependency graph for PlantState.h:
```



This graph shows which files directly or indirectly include this file:



Classes

- class [PlantState](#)
Abstract base class representing a state in a plant's life cycle.

5.42.1 Detailed Description

Defines the abstract interface for plant life cycle states.

The [PlantState](#) class represents a specific state in a plant's life cycle (e.g., Growth, Distressed, Blooming). Each state defines its behavior for applying care and transitioning.

See also

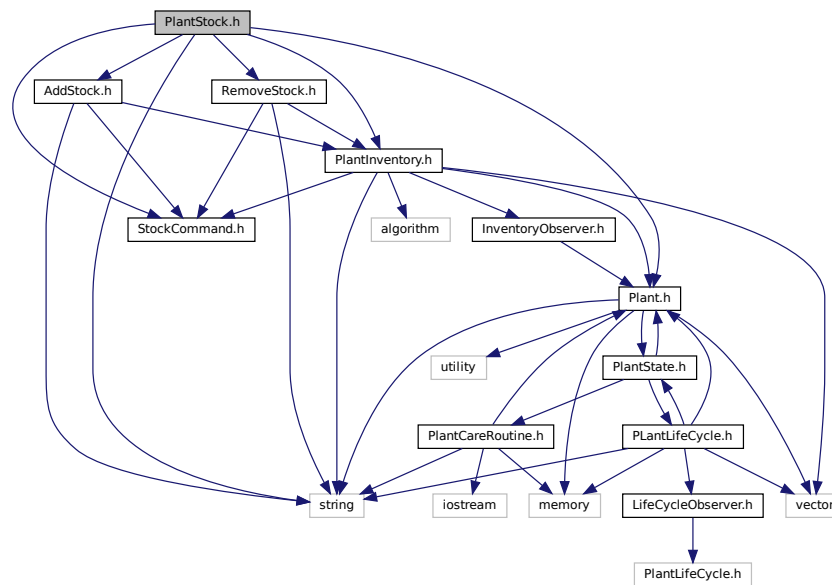
[PlantLifeCycle](#)
[DistressedState](#)

5.43 PlantStock.h File Reference

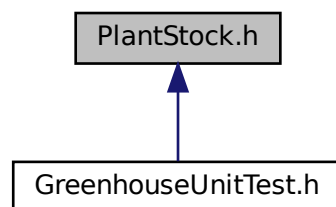
Invoker for executing stock-related commands.

```
#include "StockCommand.h"
#include "AddStock.h"
#include "RemoveStock.h"
#include "Plant.h"
#include "PlantInventory.h"
#include <string>
```

Include dependency graph for PlantStock.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [PlantStock](#)

Acts as the Invoker in the Command pattern.

5.43.1 Detailed Description

Invoker for executing stock-related commands.

The [PlantStock](#) class creates and executes stock management commands such as adding or removing plants from inventory. Implements the Command pattern.

See also

[StockCommand](#)

[AddStock](#)

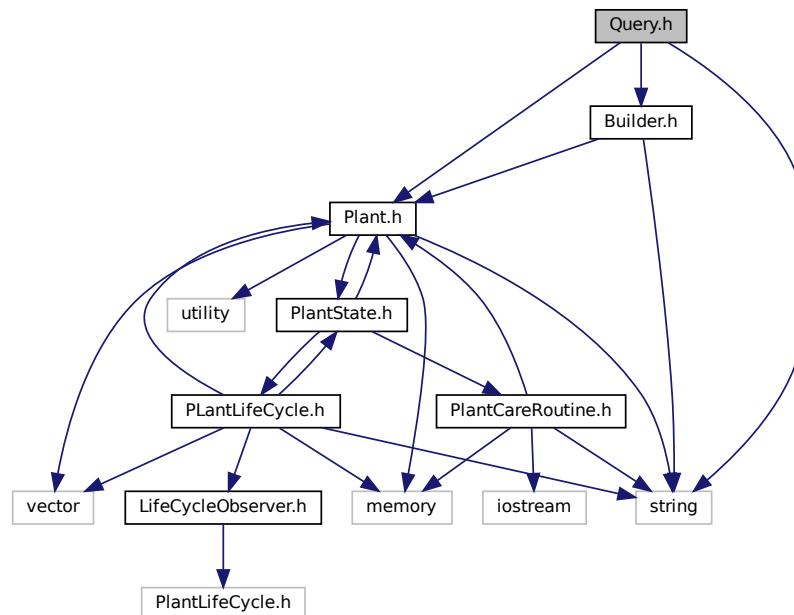
[RemoveStock](#)

[PlantInventory](#)

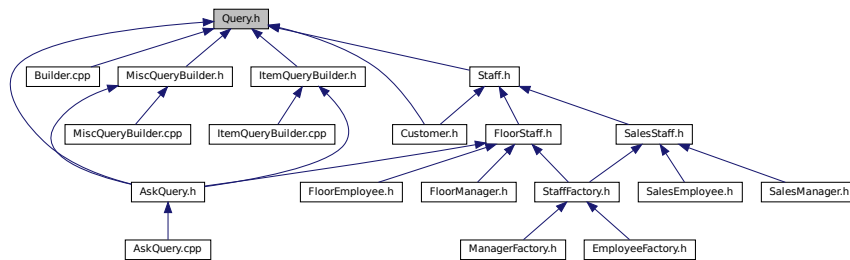
5.44 Query.h File Reference

Represents a query within the plant management system.

```
#include "Plant.h"
#include "Builder.h"
#include <string>
Include dependency graph for Query.h:
```



This graph shows which files directly or indirectly include this file:



Classes

- class [Query](#)

Encapsulates a query object constructed by a [Builder](#).

5.44.1 Detailed Description

Represents a query within the plant management system.

The [Query](#) class encapsulates a specific question or request related to a plant or inventory operation. It is built using the [Builder](#) pattern to allow flexible creation of query objects.

See also

[Builder](#)

[Plant](#)

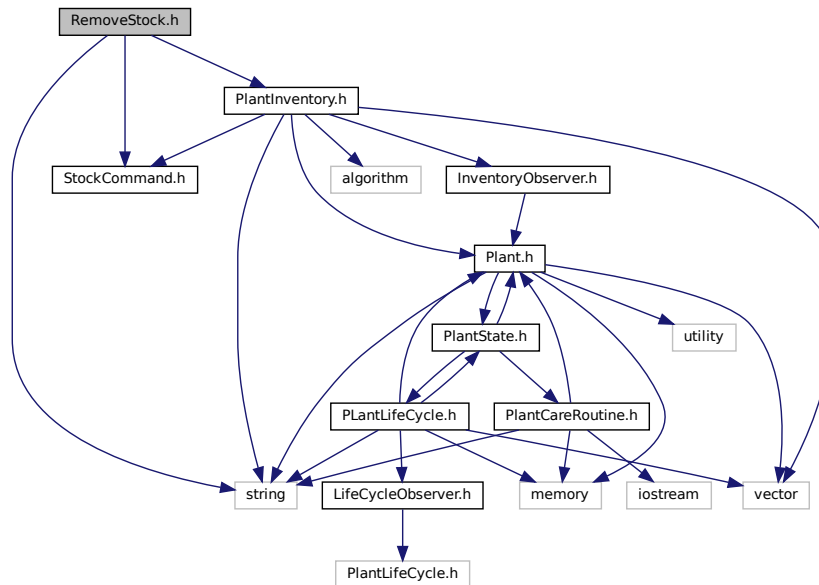
5.45 RemoveStock.h File Reference

Defines the [RemoveStock](#) command for decreasing inventory.

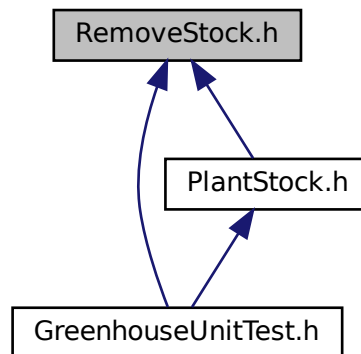
```
#include "StockCommand.h"
#include "PlantInventory.h"
```

```
#include <string>
```

Include dependency graph for RemoveStock.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [RemoveStock](#)

Concrete command that removes plant stock from the inventory.

5.45.1 Detailed Description

Defines the [RemoveStock](#) command for decreasing inventory.

Implements the Command pattern to remove a specified quantity of a given plant from the [PlantInventory](#).

See also

[StockCommand](#)

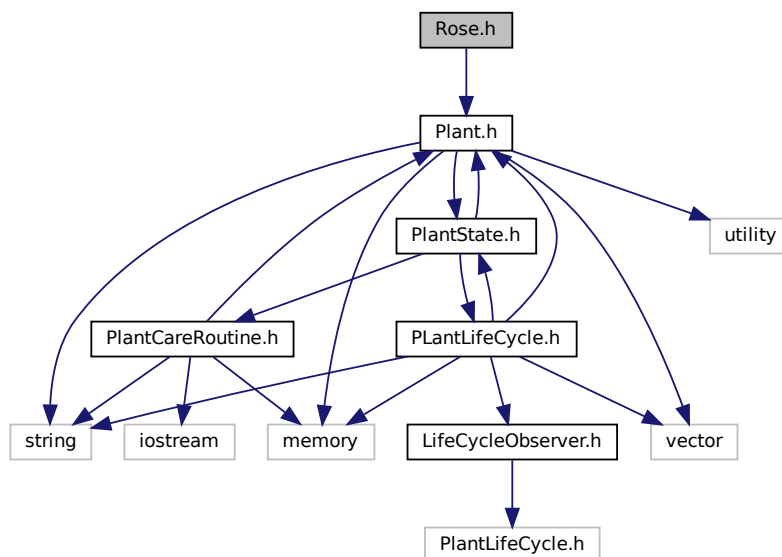
[PlantInventory](#)

[AddStock](#)

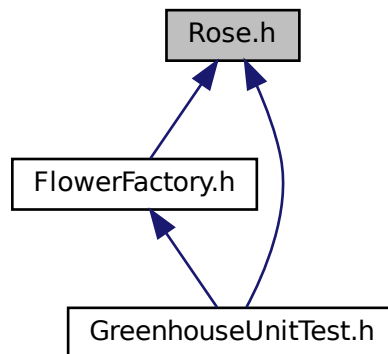
5.46 Rose.h File Reference

Defines a concrete [Rose](#) plant class.

```
#include "Plant.h"  
Include dependency graph for Rose.h:
```



This graph shows which files directly or indirectly include this file:



Classes

- class [Rose](#)
Concrete implementation of the [Plant](#) class for roses.

5.46.1 Detailed Description

Defines a concrete [Rose](#) plant class.

The [Rose](#) class represents a type of plant with a specified price and description. It inherits from the [Plant](#) base class.

See also

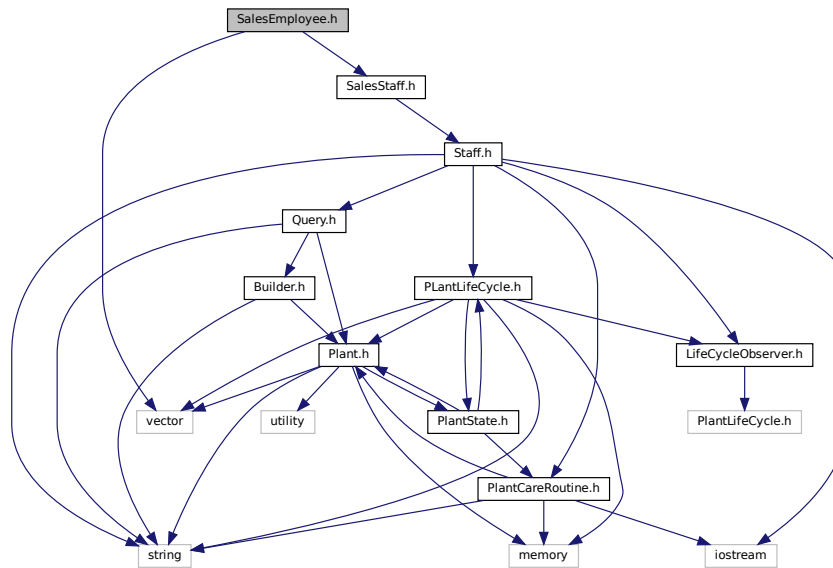
[Plant](#)

5.47 SalesEmployee.h File Reference

Defines a regular employee in the sales department.

```
#include "SalesStaff.h"  
#include <vector>
```

Include dependency graph for SalesEmployee.h:



Classes

- class [SalesEmployee](#)
Concrete class representing a regular sales employee.

5.47.1 Detailed Description

Defines a regular employee in the sales department.

The [SalesEmployee](#) class handles basic sales queries and supports chain-of-responsibility query delegation.

See also

[SalesStaff](#)
[SalesManager](#)

5.48 SalesFloorObserver.h File Reference

Defines the [SalesFloorObserver](#) class for inventory updates.

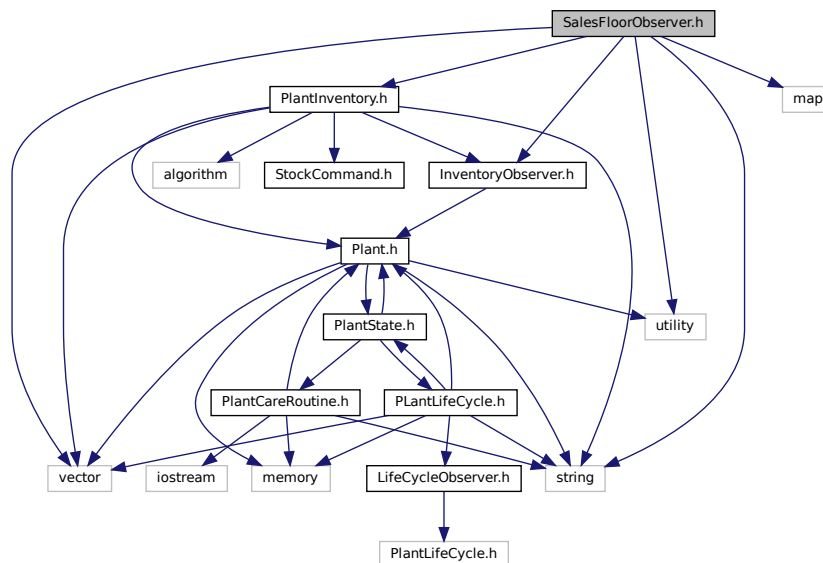
```

#include "InventoryObserver.h"
#include "PlantInventory.h"
#include <vector>
#include <utility>
#include <string>

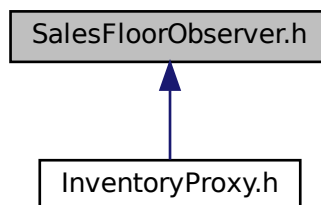
```

```
#include <map>
```

Include dependency graph for SalesFloorObserver.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [SalesFloorObserver](#)

Observes the [PlantInventory](#) and updates the sales floor display.

5.48.1 Detailed Description

Defines the [SalesFloorObserver](#) class for inventory updates.

The [SalesFloorObserver](#) monitors plant inventory and manages updates on the sales floor, notifying staff about changes.

Implements the Observer pattern.

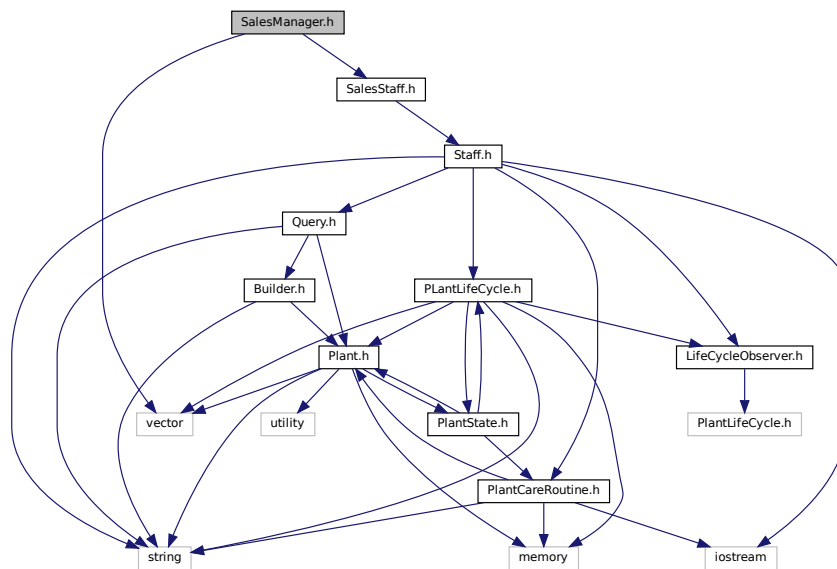
See also

[InventoryObserver](#)
[PlantInventory](#)
[Staff](#)

5.49 SalesManager.h File Reference

Defines the [SalesManager](#) class for the sales department.

```
#include "SalesStaff.h"
#include <vector>
Include dependency graph for SalesManager.h:
```



Classes

- class [SalesManager](#)
Represents a manager in the sales department.

5.49.1 Detailed Description

Defines the [SalesManager](#) class for the sales department.

The [SalesManager](#) class handles escalated queries and has authority for special deals and promotional decisions.

See also

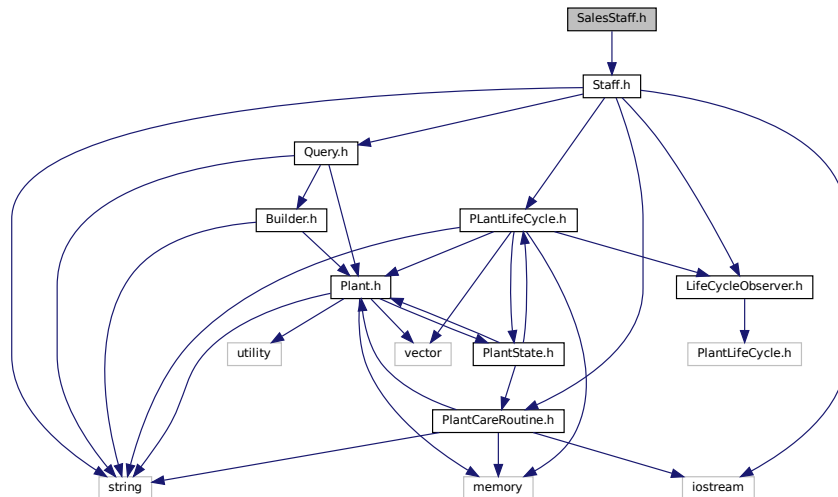
[SalesStaff](#)
[SalesEmployee](#)

5.50 SalesStaff.h File Reference

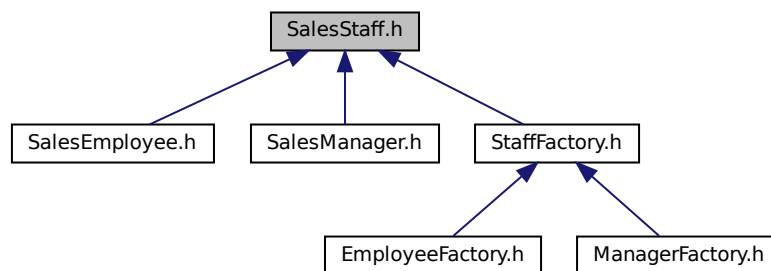
Abstract base class for staff in the sales department.

```
#include "Staff.h"
```

Include dependency graph for SalesStaff.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [SalesStaff](#)

Abstract base class for staff members in the sales department.

5.50.1 Detailed Description

Abstract base class for staff in the sales department.

The [SalesStaff](#) class provides a base for handling sales-related queries and supports the Chain of Responsibility pattern.

See also

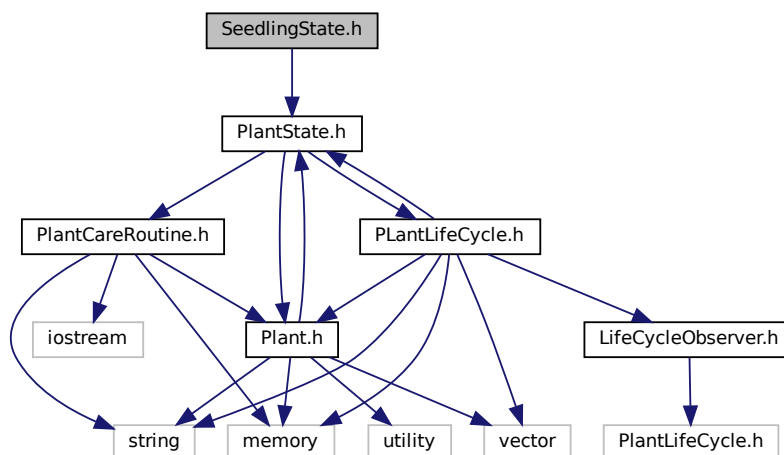
[Staff](#)
[SalesEmployee](#)
[SalesManager](#)

5.51 SeedlingState.h File Reference

Represents the seedling stage of a plant's life cycle.

```
#include "PlantState.h"
```

Include dependency graph for SeedlingState.h:



Classes

- class [SeedlingState](#)

Represents the initial growth stage of a plant's life cycle. In this state, the plant is vulnerable and requires careful nurturing to develop into a healthy mature plant.

5.51.1 Detailed Description

Represents the seedling stage of a plant's life cycle.

This state models a young plant that has germinated from a seed. In this stage, the plant is vulnerable and requires careful care to transition into a healthy mature plant.

See also

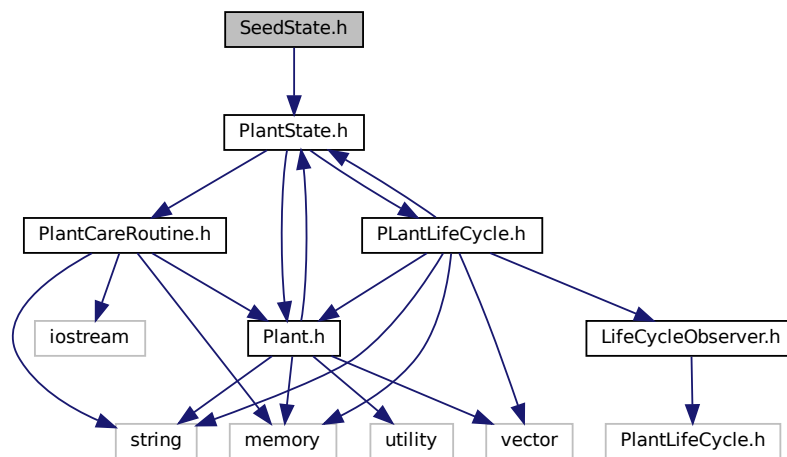
[PlantState](#)
[PlantLifeCycle](#)

5.52 SeedState.h File Reference

Represents the seed stage of a plant's life cycle.

```
#include "PlantState.h"
```

Include dependency graph for SeedState.h:



Classes

- class [SeedState](#)

Represents the initial state of a plant's life cycle, where it is a seed. In this state, the seed is dormant and requires specific conditions to germinate and grow into a seedling.

5.52.1 Detailed Description

Represents the seed stage of a plant's life cycle.

This state models a dormant seed that requires specific conditions to germinate and grow into a seedling.

See also

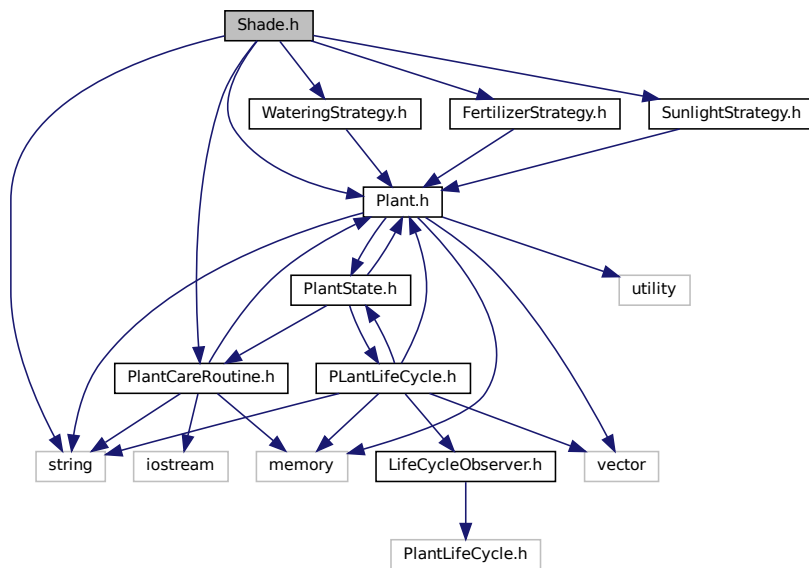
[PlantState](#)

[PlantLifeCycle](#)

5.53 Shade.h File Reference

[Plant](#) care routine implementing shade care.

```
#include <string>
#include "Plant.h"
#include "PlantCareRoutine.h"
#include "WateringStrategy.h"
#include "FertilizerStrategy.h"
#include "SunlightStrategy.h"
Include dependency graph for Shade.h:
```



Classes

- class [Shade](#)

5.53.1 Detailed Description

[Plant](#) care routine implementing shade care.

The [Shade](#) routine applies watering, fertilizing, and sunlight strategies suitable for shade-tolerant plants.

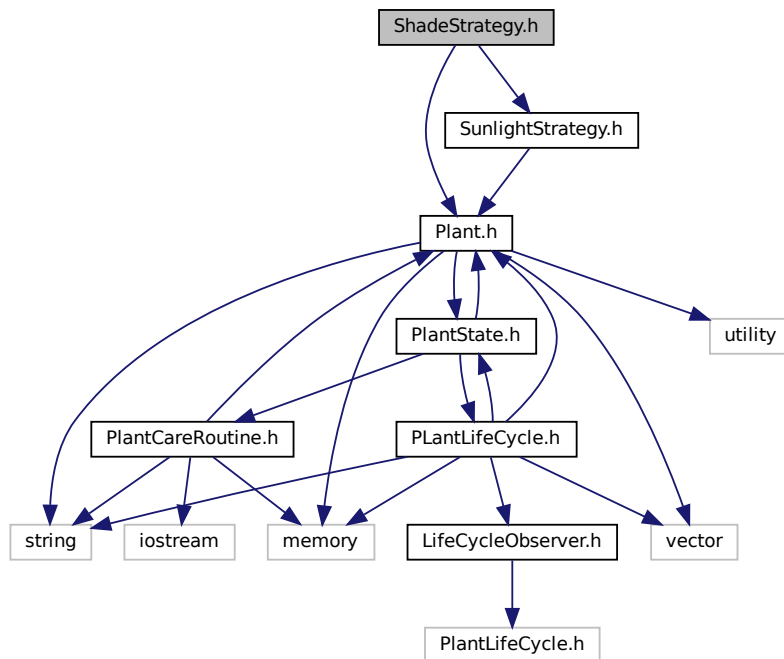
See also

[PlantCareRoutine](#)
[WateringStrategy](#)
[FertilizerStrategy](#)
[SunlightStrategy](#)

5.54 ShadeStrategy.h File Reference

Sunlight strategy for shade conditions.

```
#include "Plant.h"
#include "SunlightStrategy.h"
Include dependency graph for ShadeStrategy.h:
```



Classes

- class [ShadeStrategy](#)
[SunlightStrategy](#) implementation for shade.

5.54.1 Detailed Description

Sunlight strategy for shade conditions.

Applies the shade sunlight strategy to a plant, adjusting its sunlight exposure to simulate shaded conditions.

See also

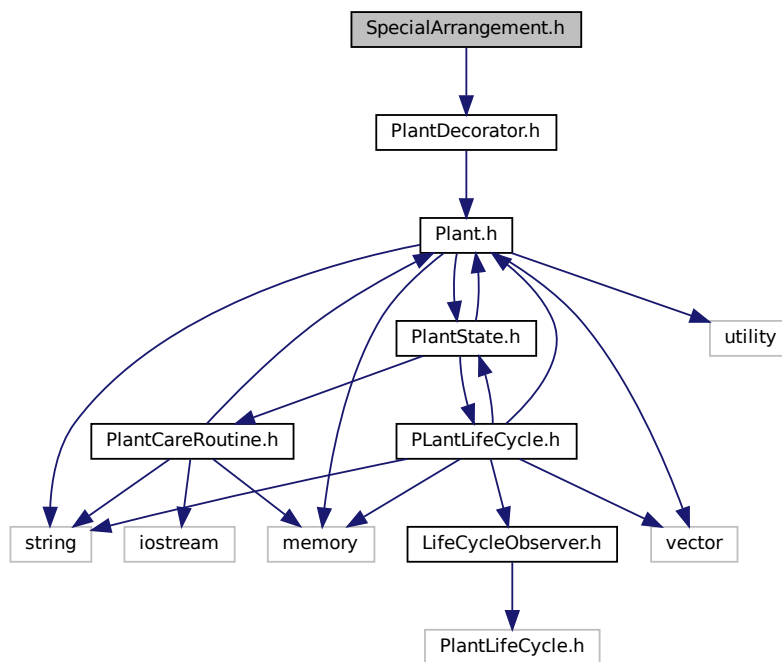
[SunlightStrategy](#)
[Plant](#)

5.55 SpecialArrangement.h File Reference

Decorator for plants that adds a special arrangement feature.

```
#include "PlantDecorator.h"
```

Include dependency graph for SpecialArrangement.h:



Classes

- class [SpecialArrangement](#)
Concrete decorator that adds a special arrangement to a plant.

5.55.1 Detailed Description

Decorator for plants that adds a special arrangement feature.

The [SpecialArrangement](#) class is a [PlantDecorator](#) that modifies the plant description and price to reflect special arrangements.

See also

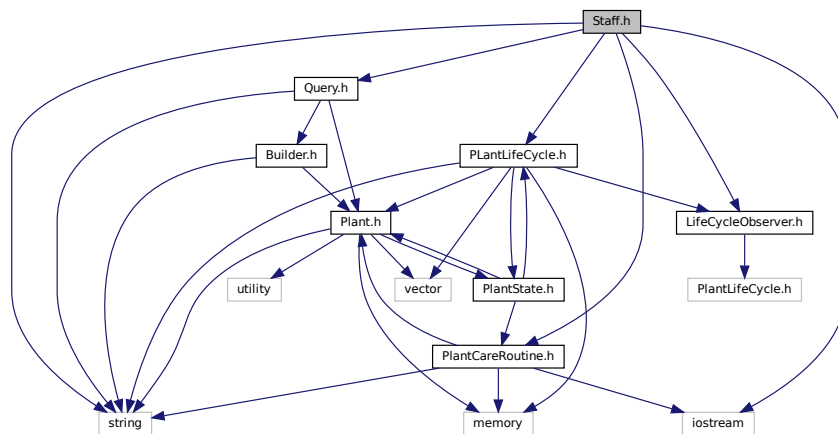
[PlantDecorator](#)
[Plant](#)

5.56 Staff.h File Reference

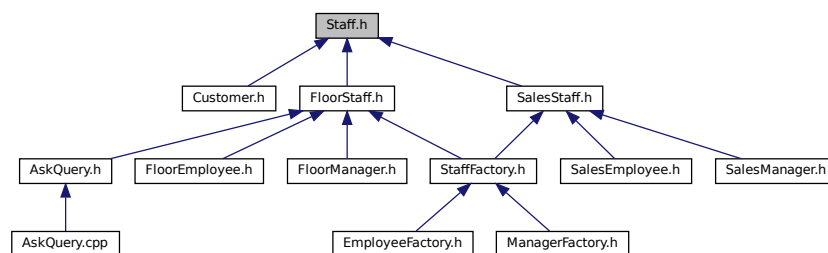
Abstract base class for staff members and lifecycle observers.

```
#include "PLantLifeCycle.h"
#include "LifeCycleObserver.h"
#include "PlantCareRoutine.h"
#include "Query.h"
#include <iostream>
#include <string>
```

Include dependency graph for Staff.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [Staff](#)

Base class representing a staff member and lifecycle observer.

5.56.1 Detailed Description

Abstract base class for staff members and lifecycle observers.

[Staff](#) observes [PlantLifeCycle](#) events, applies care routines, and optionally participates in a chain-of-responsibility for query handling.

See also

[LifeCycleObserver](#)

[PlantCareRoutine](#)

[Query](#)

5.57 StaffFactory.h File Reference

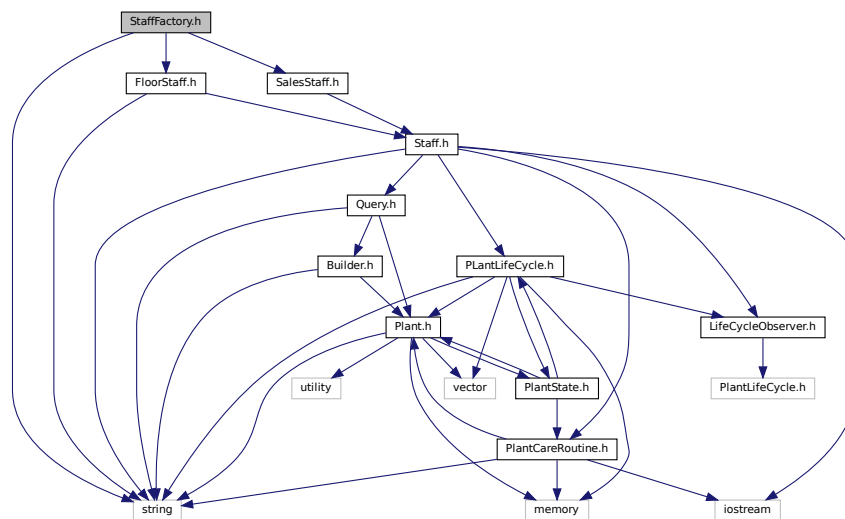
Abstract factory for creating staff members.

```
#include "FloorStaff.h"
```

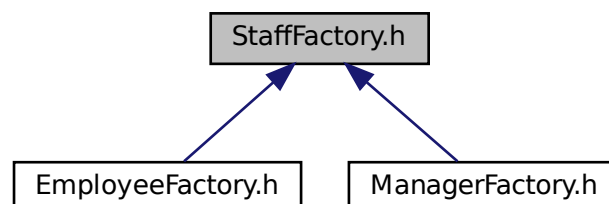
```
#include "SalesStaff.h"
```

```
#include <string>
```

Include dependency graph for StaffFactory.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [StaffFactory](#)

Abstract base factory for creating different types of staff members.

5.57.1 Detailed Description

Abstract factory for creating staff members.

Defines an interface for factories to produce [FloorStaff](#) and [SalesStaff](#). Concrete factories implement creation methods for employees or managers.

See also

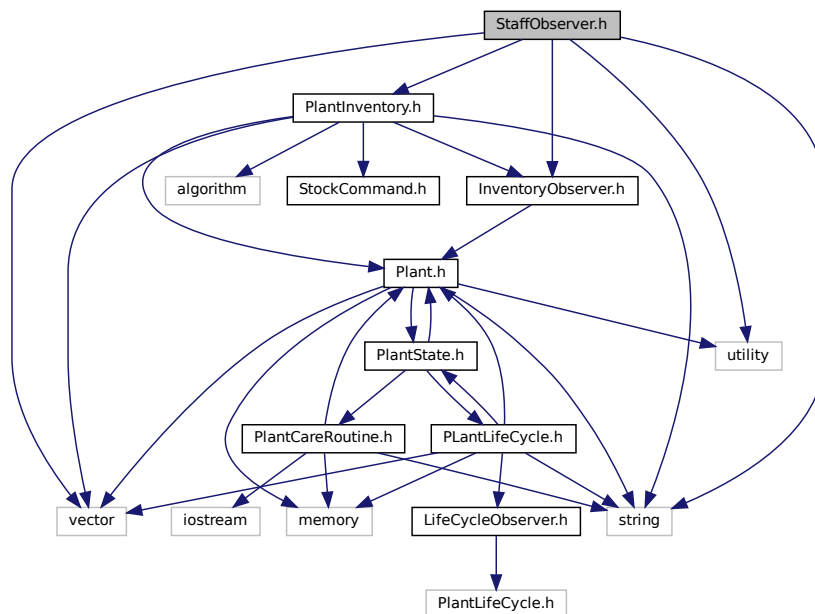
[FloorStaff](#)

[SalesStaff](#)

5.58 StaffObserver.h File Reference

Observer class for tracking inventory relevant to staff.

```
#include "InventoryObserver.h"
#include "PlantInventory.h"
#include <string>
#include <vector>
#include <utility>
Include dependency graph for StaffObserver.h:
```



Classes

- class [StaffObserver](#)
Concrete observer that tracks inventory for staff.

5.58.1 Detailed Description

Observer class for tracking inventory relevant to staff.

Monitors inventory changes and allows staff to respond appropriately.

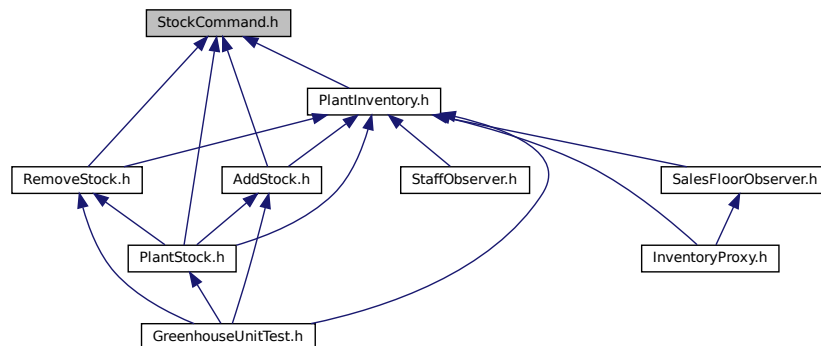
See also

[InventoryObserver](#)
[PlantInventory](#)

5.59 StockCommand.h File Reference

Abstract base class for stock management commands.

This graph shows which files directly or indirectly include this file:



Classes

- class [StockCommand](#)
Abstract base class for command pattern in stock management.

5.59.1 Detailed Description

Abstract base class for stock management commands.

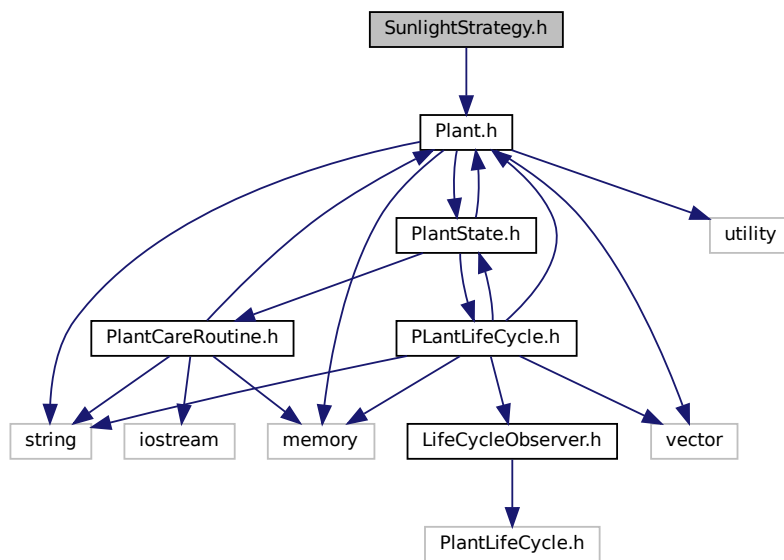
Defines the interface for command pattern used in inventory management.

5.60 SunlightStrategy.h File Reference

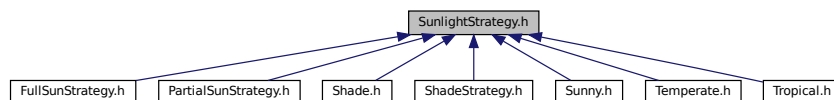
Abstract base class for sunlight strategies.

```
#include "Plant.h"
```

Include dependency graph for SunlightStrategy.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [SunlightStrategy](#)

5.60.1 Detailed Description

Abstract base class for sunlight strategies.

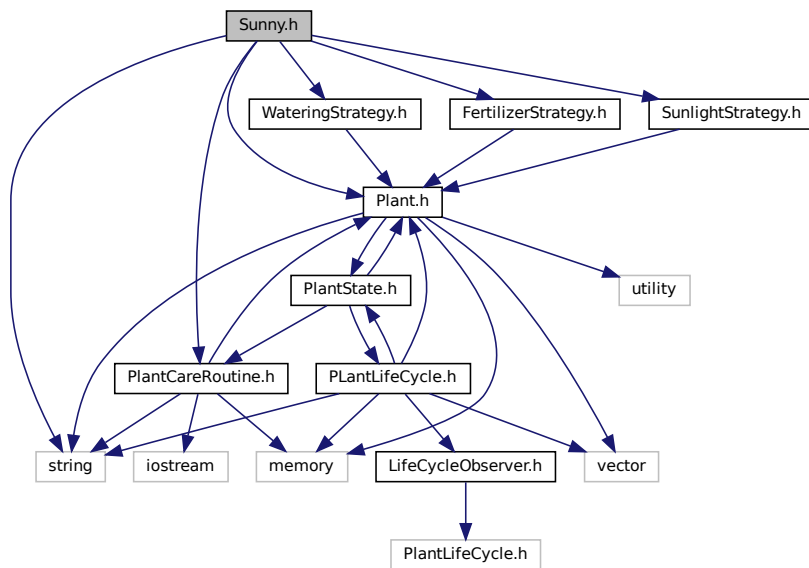
Defines the interface for strategies that apply sunlight to plants.

5.61 Sunny.h File Reference

[Plant](#) care routine for sunny conditions.

```
#include <string>
#include "Plant.h"
#include "PlantCareRoutine.h"
#include "WateringStrategy.h"
#include "FertilizerStrategy.h"
#include "SunlightStrategy.h"
```

Include dependency graph for Sunny.h:



Classes

- class [Sunny](#)
Implements the care routine for sun-loving plants.

5.61.1 Detailed Description

[Plant](#) care routine for sunny conditions.

Implements watering, fertilizing, and sunlight strategies suitable for sun-loving plants.

See also

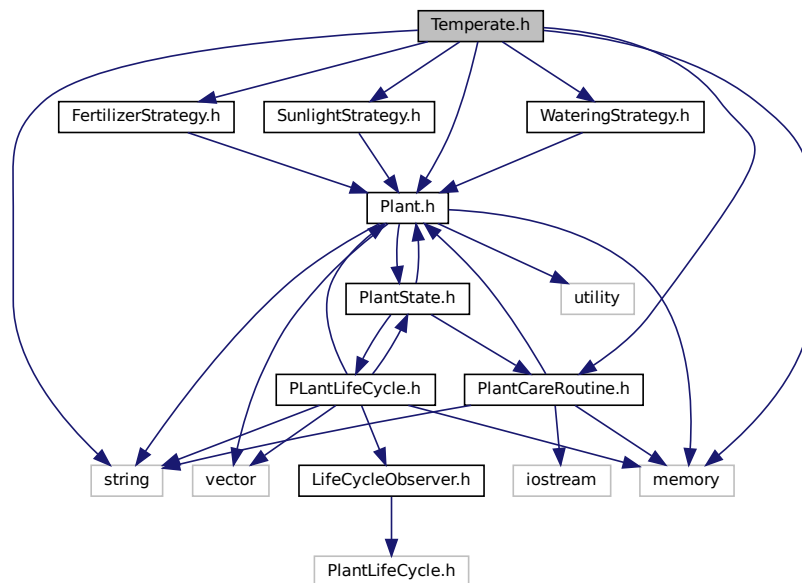
[PlantCareRoutine](#)
[WaterStrategy](#)
[FertilizerStrategy](#)
[SunlightStrategy](#)

5.62 Temperate.h File Reference

[Plant](#) care routine for temperate conditions.

```
#include <string>
#include "Plant.h"
#include "PlantCareRoutine.h"
#include "WateringStrategy.h"
#include "FertilizerStrategy.h"
#include "SunlightStrategy.h"
#include <memory>
```

Include dependency graph for Temperate.h:



Classes

- class [Temperate](#)
Implements plant care routine for temperate climate plants.

5.62.1 Detailed Description

[Plant](#) care routine for temperate conditions.

Uses strategies for watering, fertilizing, and sunlight optimized for temperate climates.

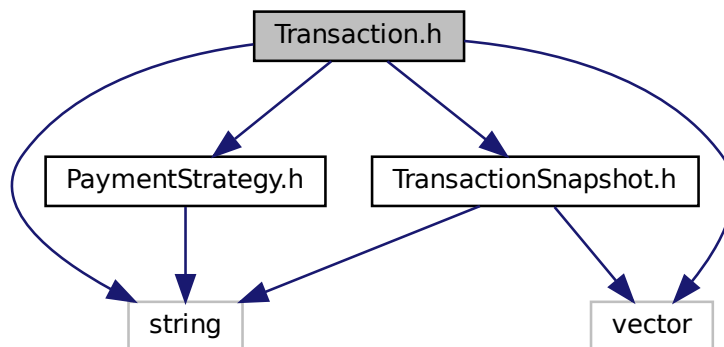
See also

[PlantCareRoutine](#)

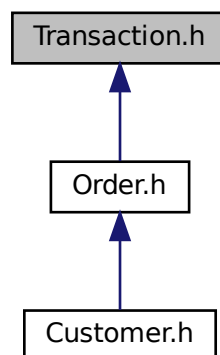
5.63 Transaction.h File Reference

Represents a customer transaction and payment.

```
#include <string>
#include <vector>
#include "TransactionSnapshot.h"
#include "PaymentStrategy.h"
Include dependency graph for Transaction.h:
```



This graph shows which files directly or indirectly include this file:



Classes

- class [Transaction](#)

Encapsulates all details of a plant purchase transaction.

5.63.1 Detailed Description

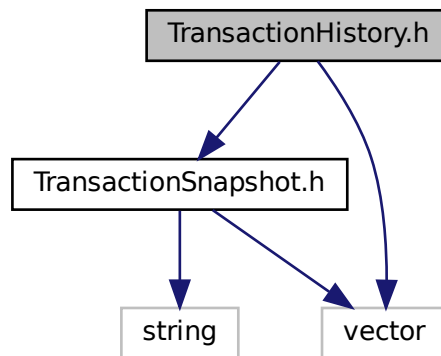
Represents a customer transaction and payment.

Supports snapshot creation and restoration for Memento pattern.

5.64 TransactionHistory.h File Reference

Maintains a record of transaction snapshots for order tracking and history retrieval.

```
#include "TransactionSnapshot.h"  
#include <vector>  
Include dependency graph for TransactionHistory.h:
```



Classes

- class [TransactionHistory](#)
Manages the storage and retrieval of transaction snapshots.

5.64.1 Detailed Description

Maintains a record of transaction snapshots for order tracking and history retrieval.

This class stores and manages a collection of [TransactionSnapshot](#) objects, allowing addition, retrieval, and inspection of past transactions.

See also

[TransactionSnapshot](#)

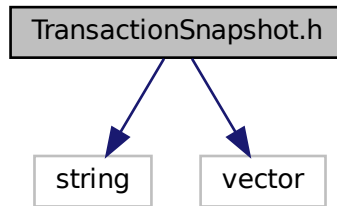
5.65 TransactionSnapshot.h File Reference

Defines the [TransactionSnapshot](#) class for storing details of a single order transaction.

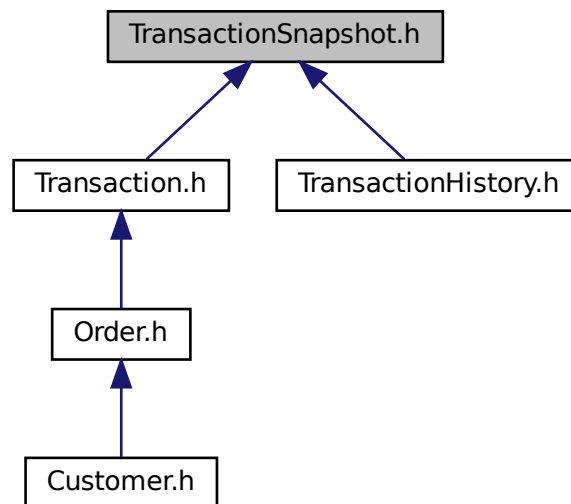
```
#include <string>
```

```
#include <vector>
```

Include dependency graph for TransactionSnapshot.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [TransactionSnapshot](#)

Represents a snapshot of a completed transaction.

5.65.1 Detailed Description

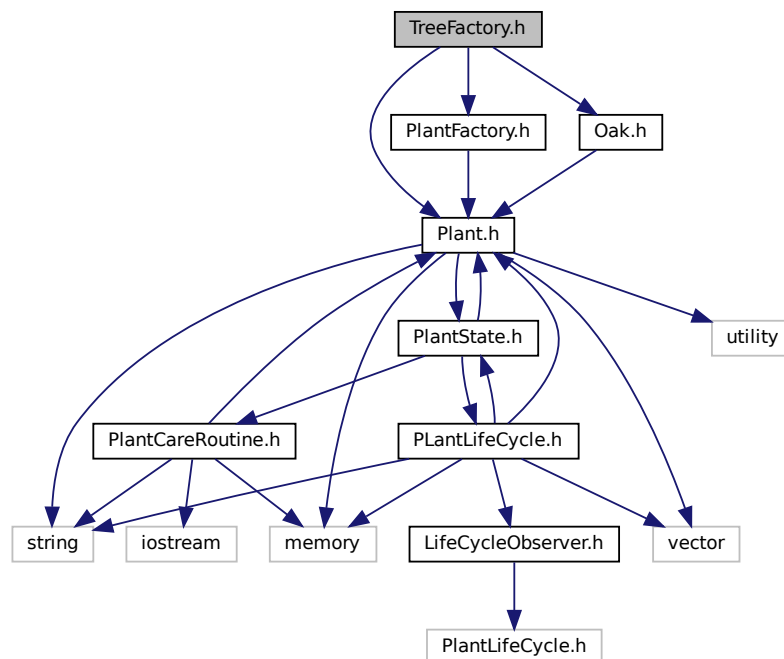
Defines the [TransactionSnapshot](#) class for storing details of a single order transaction.

Each snapshot captures order details such as order number, total amount, quantity, and optional decorations applied to the purchased plant.

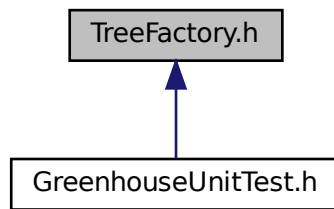
5.66 TreeFactory.h File Reference

Declares the [TreeFactory](#) class responsible for creating tree-type plants.

```
#include "PlantFactory.h"
#include "Plant.h"
#include "Oak.h"
Include dependency graph for TreeFactory.h:
```



This graph shows which files directly or indirectly include this file:



Classes

- class [TreeFactory](#)
Concrete factory for creating tree plants.

5.66.1 Detailed Description

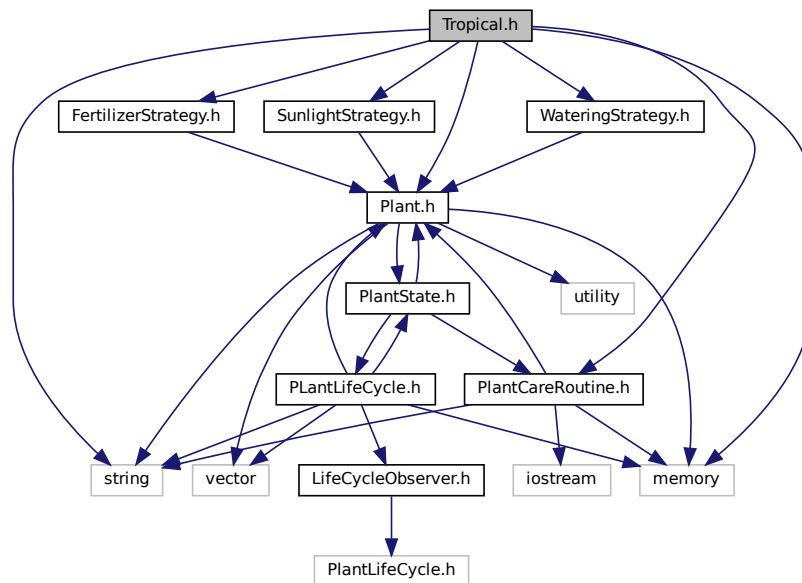
Declares the [TreeFactory](#) class responsible for creating tree-type plants.

5.67 Tropical.h File Reference

Declares the [Tropical](#) care routine for tropical plant types.

```
#include <string>
#include "Plant.h"
#include "PlantCareRoutine.h"
#include "WateringStrategy.h"
#include "FertilizerStrategy.h"
#include "SunlightStrategy.h"
#include <memory>
```

Include dependency graph for Tropical.h:



Classes

- class [Tropical](#)

Represents the plant care routine for tropical plants.

5.67.1 Detailed Description

Declares the [Tropical](#) care routine for tropical plant types.

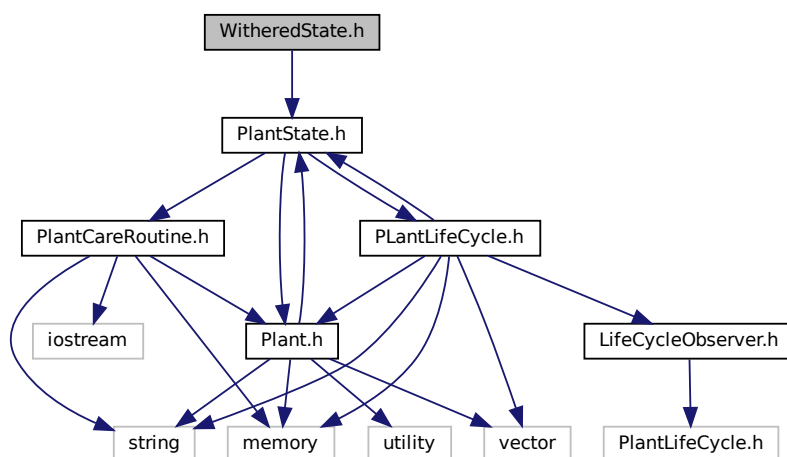
This class defines the care strategy for tropical plants by implementing watering, sunlight, and fertilizing strategies suitable for warm, humid climates.

5.68 WitheredState.h File Reference

Declares the [WitheredState](#) class representing a plant's final life stage.

```
#include "PlantState.h"
```

Include dependency graph for WitheredState.h:



Classes

- class [WitheredState](#)

Represents the final state of a plant's life cycle, where it has withered and died. In this state, the plant is beyond recovery and will not respond to care.

5.68.1 Detailed Description

Declares the [WitheredState](#) class representing a plant's final life stage.

Index

- ~Staff
 - Staff, [138](#)
- ~StockCommand
 - StockCommand, [145](#)
- addDecoration
 - Transaction, [153](#)
- addSnapshot
 - TransactionHistory, [157](#)
- AddStock, [11](#)
 - AddStock, [12](#)
 - execute, [12](#)
- addStock
 - InventoryProxy, [59](#)
 - PlantInventory, [101](#)
 - PlantStock, [107](#)
- addTransaction
 - Order, [80](#)
- applyCare
 - DistressedState, [28](#)
 - MatureState, [71](#)
 - PlantState, [105](#)
 - SeedlingState, [128](#)
 - SeedState, [131](#)
 - WitheredState, [167](#)
- applyFertilizer
 - FertilizerStrategy, [37](#)
 - InorganicFertilizer, [56](#)
 - OrganicFertilizer, [82](#)
- applySunlight
 - FullSunStrategy, [49](#)
 - PartialSunStrategy, [84](#)
 - ShadeStrategy, [134](#)
 - SunlightStrategy, [147](#)
- applyWater
 - HighWaterStrategy, [54](#)
 - LowWaterStrategy, [65](#)
 - MediumWaterStrategy, [74](#)
 - WaterStrategy, [165](#)
- AskQuery, [13](#)
 - careRoutine, [13](#)
 - describe, [14](#)
- AskQuery.cpp, [169](#)
- AskQuery.h, [170](#)
- attach
 - Plant, [89](#)
 - PlantInventory, [101](#)
- attachStaff
 - SalesFloorObserver, [119](#)
- BasePlant, [15](#)
 - getDescription, [16](#)
 - getPrice, [16](#)
- build
 - Builder, [18](#)
- Builder, [17](#)
 - build, [18](#)
 - item, [19](#)
 - question, [19](#)
 - setItem, [18](#)
 - setQuestion, [19](#)
 - setType, [19](#)
 - type, [19](#)
- Builder.cpp, [171](#)
- Builder.h, [172](#)
- buyPlant
 - InventoryProxy, [59](#)
- careRoutine
 - AskQuery, [13](#)
- clone
 - DistressedState, [28](#)
 - Transaction, [154](#)
 - WitheredState, [167](#)
- createFloorStaff
 - EmployeeFactory, [32](#)
 - Managerfactory, [66, 69](#)
 - StaffFactory, [141](#)
- createPlant
 - FlowerFactory, [47](#)
 - PlantFactory, [99](#)
 - TreeFactory, [161](#)
- createSalesStaff
 - EmployeeFactory, [33](#)
 - Managerfactory, [68, 69](#)
 - StaffFactory, [142](#)
- createSnapshot
 - Transaction, [154](#)
- CreditCardPaymentStrategy, [20](#)
 - CreditCardPaymentStrategy, [21](#)
 - getName, [21](#)
 - pay, [21](#)
- CreditCardPaymentStrategy.h, [173](#)
- currentPlant
 - FertilizerStrategy, [37](#)
 - SunlightStrategy, [147](#)
- Customer, [22](#)
 - Customer, [23](#)
 - getName, [23](#)
 - makeQuery, [23](#)

- placeOrder, 23
 - setName, 24
- Customer.h, 174
- DecorativePot, 24
 - DecorativePot, 26
 - getDescription, 26
 - getPrice, 26
- DecorativePot.h, 174
- describe
 - AskQuery, 14
- detach
 - Plant, 89
 - PlantInventory, 101
- detachStaff
 - SalesFloorObserver, 120
- DistressedState, 27
 - applyCare, 28
 - clone, 28
 - evaluate, 28
 - getName, 29
- DistressedState.h, 175
- EFTPaymentStrategy, 29
 - EFTPaymentStrategy, 30
 - getName, 31
 - pay, 31
- EFTPaymentStrategy.h, 177
- EmployeeFactory, 31
 - createFloorStaff, 32
 - createSalesStaff, 33
- EmployeeFactory.h, 177
- evaluate
 - DistressedState, 28
 - MatureState, 72
 - PlantState, 106
 - SeedlingState, 129
 - SeedState, 131
 - WitheredState, 167
- EWalletPaymentStrategy, 33
 - EWalletPaymentStrategy, 34
 - getName, 35
 - pay, 35
- EWalletPaymentStrategy.h, 178
- execute
 - AddStock, 12
 - RemoveStock, 113
 - StockCommand, 145
- executeCommand
 - PlantInventory, 102
- FertilizerStrategy, 36
 - applyFertilizer, 37
 - currentPlant, 37
 - getStrategyName, 37
- FertilizerStrategy.h, 179
- Fertilizing
 - Sunny, 149
 - Temperate, 151
 - Tropical, 163
- findPlantIndex
 - PlantInventory, 102
- FloorEmployee, 38
 - FloorEmployee, 39
 - reply, 40
 - staffCanHandle, 40
- FloorEmployee.h, 181
- FloorManager, 41
 - FloorManager, 42
 - reply, 42
 - staffCanHandle, 43
- FloorManager.h, 181
- FloorStaff, 43
 - FloorStaff, 45
 - handleQuery, 45
 - reply, 45
 - staffCanHandle, 46
- FloorStaff.h, 182
- FlowerFactory, 46
 - createPlant, 47
- FlowerFactory.h, 184
- FullSunStrategy, 48
 - applySunlight, 49
 - getStrategyName, 50
- FullSunStrategy.h, 185
- getAmount
 - Transaction, 154
 - TransactionSnapshot, 159
- getAvailableItems
 - SalesFloorObserver, 120
- getCategory
 - Plant, 89
- getColor
 - Plant, 89
- getCurrentNutrients
 - Plant, 89
- getCurrentSunlight
 - Plant, 90
- getCurrentWater
 - Plant, 90
- getCustomer
 - Order, 80
- getDecorations
 - Transaction, 154
 - TransactionSnapshot, 159
- getDescription
 - BasePlant, 16
 - DecorativePot, 26
 - GiftWrap, 52
 - Plant, 90
- getGrowthProgress
 - Plant, 90
- getHistorySize
 - TransactionHistory, 157
- getInstance
 - PlantInventory, 102
- getInventoryReference

- PlantInventory, 103
- getInventoryView
 - PlantInventory, 103
- getItem
 - Query, 110
- getMaxNutrients
 - Plant, 91
- getMaxSunlight
 - Plant, 91
- getMaxWater
 - Plant, 91
- getMinNutrients
 - Plant, 91
- getMinSunlight
 - Plant, 92
- getMinWater
 - Plant, 92
- getName
 - CreditCardPaymentStrategy, 21
 - Customer, 23
 - DistressedState, 29
 - EFTPaymentStrategy, 31
 - EWalletPaymentStrategy, 35
 - MatureState, 72
 - PaymentStrategy, 85
 - Plant, 92
 - PlantState, 106
 - SeedlingState, 129
 - SeedState, 131
 - Staff, 138
 - WitheredState, 168
- getNext
 - Staff, 138
- getNotificationHistory
 - SalesFloorObserver, 120
- getOrderId
 - Order, 80
- getOrderNum
 - TransactionSnapshot, 159
- getPaymentMethod
 - Transaction, 155
- getPlantQuantity
 - SalesFloorObserver, 120
- getPrice
 - BasePlant, 16
 - DecorativePot, 26
 - GiftWrap, 52
 - Plant, 92
- getQuantity
 - PlantInventory, 103
 - Transaction, 155
 - TransactionSnapshot, 159
- getQuestion
 - Query, 111
- getSnapshot
 - TransactionHistory, 157
- getState
 - Plant, 93
- getStatus
 - Order, 80
- getStock
 - Staff, 138
- getStrategyName
 - FertilizerStrategy, 37
 - FullSunStrategy, 50
 - HighWaterStrategy, 54
 - InorganicFertilizer, 56
 - LowWaterStrategy, 65
 - MediumWaterStrategy, 74
 - OrganicFertilizer, 82
 - PartialSunStrategy, 84
 - ShadeStrategy, 134
 - SunlightStrategy, 147
 - WaterStrategy, 165
- getTransactionId
 - Transaction, 155
- getType
 - Query, 111
- GiftWrap, 50
 - getDescription, 52
 - getPrice, 52
 - GiftWrap, 52
- GiftWrap.h, 186
- GreenhouseUnitTest, 53
- handleQuery
 - FloorStaff, 45
 - SalesStaff, 126
 - Staff, 139
- HighWaterStrategy, 53
 - applyWater, 54
 - getStrategyName, 54
- HighWaterStrategy.h, 187
- InorganicFertilizer, 55
 - applyFertilizer, 56
 - getStrategyName, 56
- InorganicFertilizer.h, 188
- InventoryObserver, 57
 - InventoryObserver, 58
- InventoryObserver.h, 189
- InventoryProxy, 58
 - addStock, 59
 - buyPlant, 59
 - InventoryProxy, 59
 - removeStock, 60
- InventoryProxy.h, 190
- isHealthy
 - Staff, 139
- isPlantAvailable
 - SalesFloorObserver, 121
- item
 - Builder, 19
- ItemQueryBuilder, 60
 - setItem, 61
 - setQuestion, 62
 - setType, 62

- ItemQueryBuilder.cpp, 192
- ItemQueryBuilder.h, 192
- LifeCycleObserver, 62
 - update, 63
- LifeCycleObserver.h, 194
- LowWaterStrategy, 64
 - applyWater, 65
 - getStrategyName, 65
- LowWaterStrategy.h, 195
- makeQuery
 - Customer, 23
- Managerfactory, 66, 68
 - createFloorStaff, 66, 69
 - createSalesStaff, 68, 69
- ManagerFactory.h, 196
- MatureState, 70
 - applyCare, 71
 - evaluate, 72
 - getName, 72
- MatureState.h, 197
- MediumWaterStrategy, 73
 - applyWater, 74
 - getStrategyName, 74
- MediumWaterStrategy.h, 198
- MiscQueryBuilder, 75
 - setItem, 76
 - setQuestion, 76
 - setType, 77
- MiscQueryBuilder.cpp, 199
- MiscQueryBuilder.h, 199
- notifyStaff
 - SalesFloorObserver, 121
- Oak, 77
 - Oak, 78
- Oak.h, 201
- Order, 79
 - addTransaction, 80
 - getCustomer, 80
 - getOrderId, 80
 - getStatus, 80
 - Order, 79
- Order.h, 202
- OrganicFertilizer, 81
 - applyFertilizer, 82
 - getStrategyName, 82
- OrganicFertilizer.h, 203
- PartialSunStrategy, 83
 - applySunlight, 84
 - getStrategyName, 84
- PartialSunStrategy.h, 204
- pay
 - CreditCardPaymentStrategy, 21
 - EFTPaymentStrategy, 31
 - EWalletPaymentStrategy, 35
 - PaymentStrategy, 85
- PaymentStrategy, 85
 - getName, 85
 - pay, 85
- PaymentStrategy.h, 206
- placeOrder
 - Customer, 23
- Plant, 86
 - attach, 89
 - detach, 89
 - getCategory, 89
 - getColor, 89
 - getCurrentNutrients, 89
 - getCurrentSunlight, 90
 - getCurrentWater, 90
 - getDescription, 90
 - getGrowthProgress, 90
 - getMaxNutrients, 91
 - getMaxSunlight, 91
 - getMaxWater, 91
 - getMinNutrients, 91
 - getMinSunlight, 92
 - getMinWater, 92
 - getName, 92
 - getPrice, 92
 - getState, 93
 - Plant, 88
 - setCategory, 93
 - setColor, 93
 - setCurrentNutrients, 94
 - setCurrentSunlight, 94
 - setCurrentWater, 94
 - setGrowthProgress, 94
 - setMaxNutrients, 95
 - setMaxSunlight, 95
 - setMaxWater, 95
 - setState, 96
- Plant.h, 207
- PlantCare
 - PlantCareRoutine, 97
- PlantCareRoutine, 96
 - PlantCare, 97
- PlantCareRoutine.h, 208
- PlantDecorator, 97
- PlantDecorator.h, 209
- PlantFactory, 99
 - createPlant, 99
- PlantFactory.h, 210
- PlantInventory, 100
 - addStock, 101
 - attach, 101
 - detach, 101
 - executeCommand, 102
 - findPlantIndex, 102
 - getInstance, 102
 - getInventoryReference, 103
 - getInventoryView, 103
 - getQuantity, 103

- removeStock, 104
- PlantInventory.h, 211
- PlantLifeCycle, 104
- PlantState, 105
 - applyCare, 105
 - evaluate, 106
 - getName, 106
- PlantState.h, 213
- PlantStock, 107
 - addStock, 107
 - PlantStock, 107
 - removeStock, 109
- PlantStock.h, 214
- Query, 109
 - getItem, 110
 - getQuestion, 111
 - getType, 111
 - Query, 110
- Query.h, 215
- question
 - Builder, 19
- RemoveStock, 112
 - execute, 113
 - RemoveStock, 113
- removeStock
 - InventoryProxy, 60
 - PlantInventory, 104
 - PlantStock, 109
- RemoveStock.h, 216
- reply
 - FloorEmployee, 40
 - FloorManager, 42
 - FloorStaff, 45
 - SalesEmployee, 117
 - SalesManager, 123
 - SalesStaff, 127
- restoreSnapshot
 - Transaction, 155
- Rose, 114
 - Rose, 115
- Rose.h, 218
- SalesEmployee, 115
 - reply, 117
 - SalesEmployee, 116
 - staffCanHandle, 117
- SalesEmployee.h, 219
- SalesFloorObserver, 118
 - attachStaff, 119
 - detachStaff, 120
 - getAvailableItems, 120
 - getNotificationHistory, 120
 - getPlantQuantity, 120
 - isPlantAvailable, 121
 - notifyStaff, 121
 - SalesFloorObserver, 119
- SalesFloorObserver.h, 220
- SalesManager, 121
 - reply, 123
 - SalesManager, 123
 - staffCanHandle, 124
- SalesManager.h, 222
- SalesStaff, 124
 - handleQuery, 126
 - reply, 127
 - SalesStaff, 126
 - staffCanHandle, 127
- SalesStaff.h, 223
- SeedlingState, 127
 - applyCare, 128
 - evaluate, 129
 - getName, 129
- SeedlingState.h, 224
- SeedState, 130
 - applyCare, 131
 - evaluate, 131
 - getName, 131
- SeedState.h, 225
- setCategory
 - Plant, 93
- setColor
 - Plant, 93
- setCurrentNutrients
 - Plant, 94
- setCurrentSunlight
 - Plant, 94
- setCurrentWater
 - Plant, 94
- setGrowthProgress
 - Plant, 94
- setItem
 - Builder, 18
 - ItemQueryBuilder, 61
 - MiscQueryBuilder, 76
- setMaxNutrients
 - Plant, 95
- setMaxSunlight
 - Plant, 95
- setMaxWater
 - Plant, 95
- setName
 - Customer, 24
- setNext
 - Staff, 140
- setPaymentStrategy
 - Transaction, 156
- setQuestion
 - Builder, 19
 - ItemQueryBuilder, 62
 - MiscQueryBuilder, 76
- setState
 - Plant, 96
- setTransaction
 - Transaction, 156
- setType

- Builder, 19
- ItemQueryBuilder, 62
- MiscQueryBuilder, 77
- Shade, 132
- Shade.h, 226
- ShadeStrategy, 133
 - applySunlight, 134
 - getStrategyName, 134
- ShadeStrategy.h, 227
- SpecialArrangement, 135
- SpecialArrangement.h, 228
- Staff, 136
 - ~Staff, 138
 - getName, 138
 - getNext, 138
 - getStock, 138
 - handleQuery, 139
 - isHealthy, 139
 - setNext, 140
 - Staff, 138
- Staff.h, 229
- staffCanHandle
 - FloorEmployee, 40
 - FloorManager, 43
 - FloorStaff, 46
 - SalesEmployee, 117
 - SalesManager, 124
 - SalesStaff, 127
- StaffFactory, 140
 - createFloorStaff, 141
 - createSalesStaff, 142
- StaffFactory.h, 230
- StaffObserver, 142
 - StaffObserver, 144
- StaffObserver.h, 231
- StockCommand, 144
 - ~StockCommand, 145
 - execute, 145
- StockCommand.h, 232
- Sunlight
 - Sunny, 149
 - Temperate, 151
 - Tropical, 163
- SunlightStrategy, 146
 - applySunlight, 147
 - currentPlant, 147
 - getStrategyName, 147
- SunlightStrategy.h, 233
- Sunny, 148
 - Fertilizing, 149
 - Sunlight, 149
 - Watering, 149
- Sunny.h, 234
- Temperate, 150
 - Fertilizing, 151
 - Sunlight, 151
 - Watering, 152
- Temperate.h, 235
- Transaction, 152
 - addDecoration, 153
 - clone, 154
 - createSnapshot, 154
 - getAmount, 154
 - getDecorations, 154
 - getPaymentMethod, 155
 - getQuantity, 155
 - getTransactionId, 155
 - restoreSnapshot, 155
 - setPaymentStrategy, 156
 - setTransaction, 156
 - Transaction, 153
- Transaction.h, 236
- TransactionHistory, 156
 - addSnapshot, 157
 - getHistorySize, 157
 - getSnapshot, 157
- TransactionHistory.h, 237
- TransactionSnapshot, 158
 - getAmount, 159
 - getDecorations, 159
 - getOrderNum, 159
 - getQuantity, 159
 - TransactionSnapshot, 158
- TransactionSnapshot.h, 238
- TreeFactory, 160
 - createPlant, 161
- TreeFactory.h, 239
- Tropical, 161
 - Fertilizing, 163
 - Sunlight, 163
 - Watering, 163
- Tropical.h, 240
- type
 - Builder, 19
- update
 - LifeCycleObserver, 63
- Watering
 - Sunny, 149
 - Temperate, 152
 - Tropical, 163
- WaterStrategy, 164
 - applyWater, 165
 - getStrategyName, 165
- WitheredState, 166
 - applyCare, 167
 - clone, 167
 - evaluate, 167
 - getName, 168
- WitheredState.h, 241